

knn

April 21, 2026

```
[1]: # This mounts your Google Drive to the Colab VM.
from google.colab import drive
drive.mount('/content/drive')

# TODO: Enter the foldername in your Drive where you have saved the unzipped
# assignment folder, e.g. 'cs231n/assignments/assignment1/'
FOLDERNAME = 'cs231n/assignments/assignment1/'
assert FOLDERNAME is not None, "[!] Enter the foldername."

# Now that we've mounted your Drive, this ensures that
# the Python interpreter of the Colab VM can load
# python files from within it.
import sys
sys.path.append('/content/drive/My Drive/{}'.format(FOLDERNAME))

# This downloads the CIFAR-10 dataset to your Drive
# if it doesn't already exist.
# %cd /content/drive/My\ Drive/$FOLDERNAME/cs231n/datasets/
# !bash get_datasets.sh
%cd /content/drive/My\ Drive/$FOLDERNAME
```

Mounted at /content/drive
/content/drive/My Drive/cs231n/assignments/assignment1

1 k-Nearest Neighbor (kNN) exercise

Complete and hand in this completed worksheet (including its outputs and any supporting code outside of the worksheet) with your assignment submission. For more details see the [assignments page](#) on the course website.

The kNN classifier consists of two stages:

- During training, the classifier takes the training data and simply remembers it
- During testing, kNN classifies every test image by comparing to all training images and transferring the labels of the k most similar training examples
- The value of k is cross-validated

In this exercise you will implement these steps and understand the basic Image Classification pipeline, cross-validation, and gain proficiency in writing efficient, vectorized code.

```
[ ]: # !pip install --upgrade IPython
```

```
Requirement already satisfied: IPython in /usr/local/lib/python3.12/dist-  
packages (7.34.0)  
Collecting IPython  
  Downloading ipython-9.12.0-py3-none-any.whl.metadata (4.5 kB)  
Collecting decorator>=5.1.0 (from IPython)  
  Downloading decorator-5.2.1-py3-none-any.whl.metadata (3.9 kB)  
Collecting ipython-pygments-lexers>=1.0.0 (from IPython)  
  Downloading ipython_pygments_lexers-1.1.1-py3-none-any.whl.metadata (1.1 kB)  
Collecting jedi>=0.18.2 (from IPython)  
  Downloading jedi-0.19.2-py2.py3-none-any.whl.metadata (22 kB)  
Requirement already satisfied: matplotlib-inline>=0.1.6 in  
/usr/local/lib/python3.12/dist-packages (from IPython) (0.2.1)  
Requirement already satisfied: pexpect>4.6 in /usr/local/lib/python3.12/dist-  
packages (from IPython) (4.9.0)  
Requirement already satisfied: prompt_toolkit<3.1.0,>=3.0.41 in  
/usr/local/lib/python3.12/dist-packages (from IPython) (3.0.52)  
Requirement already satisfied: pygments>=2.14.0 in  
/usr/local/lib/python3.12/dist-packages (from IPython) (2.20.0)  
Collecting stack_data>=0.6.0 (from IPython)  
  Downloading stack_data-0.6.3-py3-none-any.whl.metadata (18 kB)  
Collecting traitlets>=5.13.0 (from IPython)  
  Downloading traitlets-5.14.3-py3-none-any.whl.metadata (10 kB)  
Requirement already satisfied: parso<0.9.0,>=0.8.4 in  
/usr/local/lib/python3.12/dist-packages (from jedi>=0.18.2->IPython) (0.8.6)  
Requirement already satisfied: ptyprocess>=0.5 in  
/usr/local/lib/python3.12/dist-packages (from pexpect>4.6->IPython) (0.7.0)  
Requirement already satisfied: wcwidth in /usr/local/lib/python3.12/dist-  
packages (from prompt_toolkit<3.1.0,>=3.0.41->IPython) (0.6.0)  
Collecting executing>=1.2.0 (from stack_data>=0.6.0->IPython)  
  Downloading executing-2.2.1-py2.py3-none-any.whl.metadata (8.9 kB)  
Collecting asttokens>=2.1.0 (from stack_data>=0.6.0->IPython)  
  Downloading asttokens-3.0.1-py3-none-any.whl.metadata (4.9 kB)  
Collecting pure_eval (from stack_data>=0.6.0->IPython)  
  Downloading pure_eval-0.2.3-py3-none-any.whl.metadata (6.3 kB)  
Downloading ipython-9.12.0-py3-none-any.whl (625 kB)  
625.7/625.7 kB  
13.1 MB/s eta 0:00:00  
Downloading decorator-5.2.1-py3-none-any.whl (9.2 kB)  
Downloading ipython_pygments_lexers-1.1.1-py3-none-any.whl (8.1 kB)  
Downloading jedi-0.19.2-py2.py3-none-any.whl (1.6 MB)  
1.6/1.6 MB  
33.9 MB/s eta 0:00:00  
Downloading stack_data-0.6.3-py3-none-any.whl (24 kB)  
Downloading traitlets-5.14.3-py3-none-any.whl (85 kB)  
85.4/85.4 kB
```

7.9 MB/s eta 0:00:00

Downloading asttokens-3.0.1-py3-none-any.whl (27 kB)

Downloading executing-2.2.1-py2.py3-none-any.whl (28 kB)

Downloading pure_eval-0.2.3-py3-none-any.whl (11 kB)

Installing collected packages: pure-eval, traitlets, jedi, ipython-pygments-lexers, executing, decorator, asttokens, stack_data, IPython

Attempting uninstall: traitlets

Found existing installation: traitlets 5.7.1

Uninstalling traitlets-5.7.1:

Successfully uninstalled traitlets-5.7.1

Attempting uninstall: decorator

Found existing installation: decorator 4.4.2

Uninstalling decorator-4.4.2:

Successfully uninstalled decorator-4.4.2

Attempting uninstall: IPython

Found existing installation: ipython 7.34.0

Uninstalling ipython-7.34.0:

Successfully uninstalled ipython-7.34.0

ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.

google-colab 1.0.0 requires ipython==7.34.0, but you have ipython 9.12.0 which is incompatible.

moviepy 1.0.3 requires decorator<5.0,>=4.0.2, but you have decorator 5.2.1 which is incompatible.

Successfully installed IPython-9.12.0 asttokens-3.0.1 decorator-5.2.1 executing-2.2.1 ipython-pygments-lexers-1.1.1 jedi-0.19.2 pure-eval-0.2.3 stack_data-0.6.3 traitlets-5.14.3

```
[ ]: # Run some setup code for this notebook.

import random
import numpy as np
from cs231n.data_utils import load_CIFAR10
import matplotlib.pyplot as plt

# This is a bit of magic to make matplotlib figures appear inline in the
↳ notebook
# rather than in a new window.
%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'
```

```
# Some more magic so that the notebook will reload external python modules;
# see http://stackoverflow.com/questions/1907993/
↳ autoreload-of-modules-in-ipython
%load_ext autoreload
%autoreload 2
```

```
[ ]: # Load the raw CIFAR-10 data.
cifar10_dir = 'cs231n/datasets/cifar-10-batches-py'

# Cleaning up variables to prevent loading data multiple times (which may cause
↳ memory issue)
try:
    del X_train, y_train
    del X_test, y_test
    print('Clear previously loaded data.')
except:
    pass

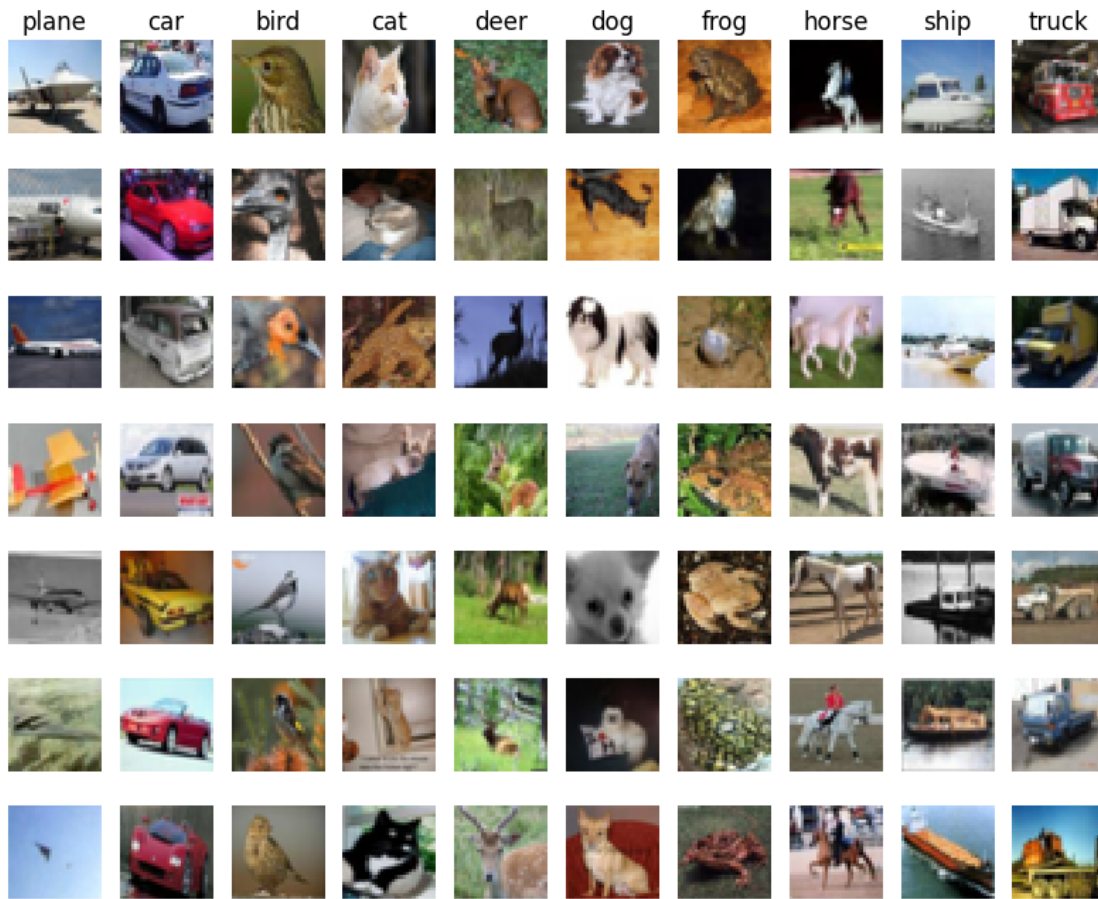
X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

# As a sanity check, we print out the size of the training and test data.
print('Training data shape: ', X_train.shape)
print('Training labels shape: ', y_train.shape)
print('Test data shape: ', X_test.shape)
print('Test labels shape: ', y_test.shape)
```

```
Training data shape: (50000, 32, 32, 3)
Training labels shape: (50000,)
Test data shape: (10000, 32, 32, 3)
Test labels shape: (10000,)
```

```
[ ]: # Visualize some examples from the dataset.
# We show a few examples of training images from each class.
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse',
↳ 'ship', 'truck']
num_classes = len(classes)
samples_per_class = 7
for y, cls in enumerate(classes):
    idxs = np.flatnonzero(y_train == y)
    idxs = np.random.choice(idxs, samples_per_class, replace=False)
    for i, idx in enumerate(idxs):
        plt_idx = i * num_classes + y + 1
        plt.subplot(samples_per_class, num_classes, plt_idx)
        plt.imshow(X_train[idx].astype('uint8'))
        plt.axis('off')
    if i == 0:
        plt.title(cls)
```

```
plt.show()
```



```
[ ]: # Subsample the data for more efficient code execution in this exercise
num_training = 5000
mask = list(range(num_training))
X_train = X_train[mask]
y_train = y_train[mask]

num_test = 500
mask = list(range(num_test))
X_test = X_test[mask]
y_test = y_test[mask]

# Reshape the image data into rows
X_train = np.reshape(X_train, (X_train.shape[0], -1))
X_test = np.reshape(X_test, (X_test.shape[0], -1))
print(X_train.shape, X_test.shape)
```

```
(5000, 3072) (500, 3072)
```

```
[ ]: from cs231n.classifiers import KNearestNeighbor

# Create a kNN classifier instance.
# Remember that training a kNN classifier is a noop:
# the Classifier simply remembers the data and does no further processing
classifier = KNearestNeighbor()
classifier.train(X_train, y_train)
```

We would now like to classify the test data with the kNN classifier. Recall that we can break down this process into two steps:

1. First we must compute the distances between all test examples and all train examples.
2. Given these distances, for each test example we find the k nearest examples and have them vote for the label

Lets begin with computing the distance matrix between all training and test examples. For example, if there are N_{tr} training examples and N_{te} test examples, this stage should result in a $N_{te} \times N_{tr}$ matrix where each element (i,j) is the distance between the i-th test and j-th train example.

Note: For the three distance computations that we require you to implement in this notebook, you may not use the `np.linalg.norm()` function that numpy provides.

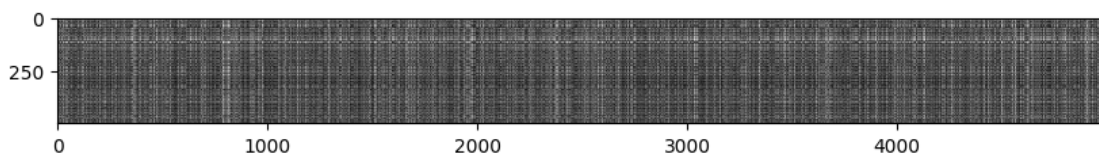
First, open `cs231n/classifiers/k_nearest_neighbor.py` and implement the function `compute_distances_two_loops` that uses a (very inefficient) double loop over all pairs of (test, train) examples and computes the distance matrix one element at a time.

```
[ ]: # Open cs231n/classifiers/k_nearest_neighbor.py and implement
# compute_distances_two_loops.

# Test your implementation:
dists = classifier.compute_distances_two_loops(X_test)
print(dists.shape)
```

(500, 5000)

```
[ ]: # We can visualize the distance matrix: each row is a single test example and
# its distances to training examples
plt.imshow(dists, interpolation='none')
plt.show()
```



```
[ ]: dists_one = classifier.compute_distances_one_loop(X_test)
print(dists_one.shape)
```

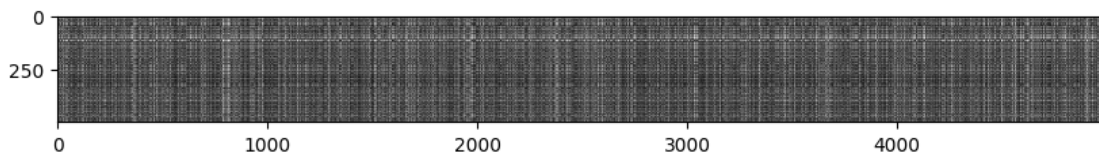
```
[autoreload of cs231n.classifiers.k_nearest_neighbor failed: Traceback (most
recent call last):
  File "/usr/local/lib/python3.11/dist-
packages/IPython/extensions/autoreload.py", line 245, in check
    superreload(m, reload, self.old_objects)
  File "/usr/local/lib/python3.11/dist-
packages/IPython/extensions/autoreload.py", line 394, in superreload
    module = reload(module)
    ~~~~~~
  File "/usr/lib/python3.11/imp.py", line 315, in reload
    return importlib.reload(module)
    ~~~~~~
  File "/usr/lib/python3.11/importlib/__init__.py", line 169, in reload
    _bootstrap._exec(spec, module)
  File "<frozen importlib._bootstrap>", line 621, in _exec
  File "<frozen importlib._bootstrap_external>", line 936, in exec_module
  File "<frozen importlib._bootstrap_external>", line 1074, in get_code
  File "<frozen importlib._bootstrap_external>", line 1004, in source_to_code
  File "<frozen importlib._bootstrap>", line 241, in _call_with_frames_removed
  File "/content/drive/My
Drive/cs231n/assignments/assignment1/cs231n/classifiers/k_nearest_neighbor.py",
line 167
    closest_y = np.
~
SyntaxError: invalid syntax
]
```

```
(500, 5000)
```

```
[ ]: dists_no = classifier.compute_distances_no_loops(X_test)
      print(dists_no.shape)
```

```
(500, 1)
(5000,)
(500, 5000)
(500, 5000)
```

```
[ ]: plt.imshow(dists_no, interpolation='none')
      plt.show()
```



[]:

Inline Question 1

Notice the structured patterns in the distance matrix, where some rows or columns are visibly brighter. (Note that with the default color scheme black indicates low distances while white indicates high distances.)

- What in the data is the cause behind the distinctly bright rows?
- What causes the columns?

Your Answer :

- Lí do có những hàng sáng : những ảnh trong tập test mà có khoảng cách xa với tất cả các ảnh trong tập train
- Lí do có những cột sáng : những ảnh trong tập train mà có khoảng cách xa với tất cả các ảnh trong tập test

```
[ ]: # Now implement the function predict_labels and run the code below:
# We use k = 1 (which is Nearest Neighbor).
y_test_pred = classifier.predict_labels(dists, k=1)

# Compute and print the fraction of correctly predicted examples
num_correct = np.sum(y_test_pred == y_test)
accuracy = float(num_correct) / num_test
print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, accuracy))
```

Got 137 / 500 correct => accuracy: 0.274000

You should expect to see approximately 27% accuracy. Now lets try out a larger k, say k = 5:

```
[ ]: y_test_pred = classifier.predict_labels(dists, k=5)
num_correct = np.sum(y_test_pred == y_test)
accuracy = float(num_correct) / num_test
print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, accuracy))
```

Got 139 / 500 correct => accuracy: 0.278000

You should expect to see a slightly better performance than with k = 1.

Inline Question 2

We can also use other distance metrics such as L1 distance. For pixel values $p_{ij}^{(k)}$ at location (i, j) of some image I_k ,

the mean μ across all pixels over all images is

$$\mu = \frac{1}{nhw} \sum_{k=1}^n \sum_{i=1}^h \sum_{j=1}^w p_{ij}^{(k)}$$

And the pixel-wise mean μ_{ij} across all images is

$$\mu_{ij} = \frac{1}{n} \sum_{k=1}^n p_{ij}^{(k)}.$$

The general standard deviation σ and pixel-wise standard deviation σ_{ij} is defined similarly.

Which of the following preprocessing steps will not change the performance of a Nearest Neighbor classifier that uses L1 distance? Select all that apply. To clarify, both training and test examples are preprocessed in the same way.

1. Subtracting the mean μ ($\tilde{p}_{ij}^{(k)} = p_{ij}^{(k)} - \mu$.)
2. Subtracting the per pixel mean μ_{ij} ($\tilde{p}_{ij}^{(k)} = p_{ij}^{(k)} - \mu_{ij}$.)
3. Subtracting the mean μ and dividing by the standard deviation σ .
4. Subtracting the pixel-wise mean μ_{ij} and dividing by the pixel-wise standard deviation σ_{ij} .
5. Rotating the coordinate axes of the data, which means rotating all the images by the same angle. Empty regions in the image caused by rotation are padded with a same pixel value and no interpolation is performed.

Your Answer :

Câu 1,2,3 đúng

Your Explanation :

Câu 1 đúng là do hiệu giữa 2 pixel tại (i,j) của ảnh k và k' không đổi ($p_{ij}^{(k)}$ cùng trừ đi μ)

Câu 2 đúng là do hiệu giữa 2 pixel tại (i,j) của ảnh k và k' không đổi ($p_{ij}^{(k)}$ cùng trừ đi μ_{ij})

Câu 3 đúng là do Câu 1 đúng và do hiệu giữa 2 pixel tương ứng tại 2 ảnh được chia cho σ

Câu 4 sai vì khi này tổng của $\sigma_{ij} * |p_{ijk} - p_{ijk}'|$ hoặc $\sigma_{ij} * (p_{ijk} - p_{ijk}')^2$ là tổng có trọng số, với các σ_{ij} khác nhau.

Câu 5 sai vì đối với cách tính L1, thì phép xoay góc không bảo toàn hiệu của nó, vì khi đó, phần padding sẽ làm mất tổng khoảng cách cũ giữa các pixel tương ứng

```
[ ]: # Now lets speed up distance matrix computation by using partial vectorization
# with one loop. Implement the function compute_distances_one_loop and run the
# code below:
dists_one = classifier.compute_distances_one_loop(X_test)

# To ensure that our vectorized implementation is correct, we make sure that it
# agrees with the naive implementation. There are many ways to decide whether
# two matrices are similar; one of the simplest is the Frobenius norm. In case
# you haven't seen it before, the Frobenius norm of two matrices is the square
# root of the squared sum of differences of all elements; in other words,
↳ reshape
# the matrices into vectors and compute the Euclidean distance between them.
difference = np.linalg.norm(dists - dists_one, ord='fro')
print('One loop difference was: %f' % (difference, ))
if difference < 0.001:
    print('Good! The distance matrices are the same')
else:
    print('Uh-oh! The distance matrices are different')
```

One loop difference was: 0.000000
Good! The distance matrices are the same

```
[ ]: # Now implement the fully vectorized version inside compute_distances_no_loops
# and run the code
dists_two = classifier.compute_distances_no_loops(X_test)

# check that the distance matrix agrees with the one we computed before:
difference = np.linalg.norm(dists - dists_two, ord='fro')
print('No loop difference was: %f' % (difference, ))
if difference < 0.001:
    print('Good! The distance matrices are the same')
else:
    print('Uh-oh! The distance matrices are different')
```

(500, 1)
(5000,)
(500, 5000)
No loop difference was: 0.000000
Good! The distance matrices are the same

```
[ ]: # Let's compare how fast the implementations are
def time_function(f, *args):
    """
    Call a function f with args and return the time (in seconds) that it took
    to execute.
    """
    import time
    tic = time.time()
    f(*args)
    toc = time.time()
    return toc - tic

two_loop_time = time_function(classifier.compute_distances_two_loops, X_test)
print('Two loop version took %f seconds' % two_loop_time)

one_loop_time = time_function(classifier.compute_distances_one_loop, X_test)
print('One loop version took %f seconds' % one_loop_time)

no_loop_time = time_function(classifier.compute_distances_no_loops, X_test)
print('No loop version took %f seconds' % no_loop_time)

# You should see significantly faster performance with the fully vectorized
implementation!

# NOTE: depending on what machine you're using,
# you might not see a speedup when you go from two loops to one loop,
```

```
# and might even see a slow-down.
```

```
Two loop version took 36.731819 seconds
One loop version took 52.386179 seconds
(500, 1)
(5000,)
(500, 5000)
No loop version took 0.958332 seconds
```

1.0.1 Cross-validation

We have implemented the k-Nearest Neighbor classifier but we set the value $k = 5$ arbitrarily. We will now determine the best value of this hyperparameter with cross-validation.

```
[ ]: # num_folds = 5
      # k_choices = [1, 3, 5, 8, 10, 12, 15, 20, 50, 100]

      # X_train_folds = []
      # y_train_folds = []
      #
      ↪ #####
      # # TODO:
      ↪ #
      # # Split up the training data into folds. After splitting, X_train_folds and
      ↪ #
      # # y_train_folds should each be lists of length num_folds, where
      ↪ #
      # # y_train_folds[i] is the label vector for the points in X_train_folds[i].
      ↪ #
      # # Hint: Look up the numpy array_split function.
      ↪ #
      #
      ↪ #####
      # X_train_folds = np.array_split(X_train, num_folds)
      # y_train_folds = np.array_split(y_train, num_folds)

      # print(len(X_train_folds[0][0]))
```

3072

```
[ ]: num_folds = 5
      k_choices = [1, 3, 5, 8, 10, 12, 15, 20, 50, 100]

      X_train_folds = []
      y_train_folds = []
      #####
      # TODO:
      #
```

```

# Split up the training data into folds. After splitting, X_train_folds and #
# y_train_folds should each be lists of length num_folds, where #
# y_train_folds[i] is the label vector for the points in X_train_folds[i]. #
# Hint: Look up the numpy array_split function. #
#####
X_train_folds = np.array_split(X_train, num_folds)
y_train_folds = np.array_split(y_train, num_folds)

# A dictionary holding the accuracies for different values of k that we find
# when running cross-validation. After running cross-validation,
# k_to_accuracies[k] should be a list of length num_folds giving the different
# accuracy values that we found when using that value of k.
k_to_accuracies = {}

#####
# TODO: #
# Perform k-fold cross validation to find the best value of k. For each #
# possible value of k, run the k-nearest-neighbor algorithm num_folds times, #
# where in each case you use all but one of the folds as training data and the #
# last fold as a validation set. Store the accuracies for all fold and all #
# values of k in the k_to_accuracies dictionary. #
#####
for k in k_choices:
    k_to_accuracies[k] = []
    for i in range(num_folds) :
        # print(10*"=")
        # print(len(X_train_folds))
        X_train_new = np.concatenate([X_train_folds[j] for j in range (num_folds)
↪if j != i])
        y_train_new = np.concatenate([y_train_folds[j] for j in range (num_folds)
↪if j != i])
        X_val = X_train_folds[i]
        y_val = y_train_folds[i]
        classifier.train(X_train_new, y_train_new)
        dists_val = classifier.compute_distances_no_loops(X_val)
        y_val_pred = classifier.predict_labels(dists_val, k)

        accuracy = float(np.sum(y_val_pred == y_val)) / num_folds
        k_to_accuracies[k].append(accuracy)
# Print out the computed accuracies
for k in sorted(k_to_accuracies):
    for accuracy in k_to_accuracies[k]:
        print('k = %d, accuracy = %f' % (k, accuracy))

```

(1000, 1)

[illegible]

[illegible]

[illegible]


```

(4000,)
(1000, 4000)
(1000, 1)
(4000,)
(1000, 4000)
k = 1, accuracy = 52.600000
k = 1, accuracy = 51.400000
k = 1, accuracy = 52.800000
k = 1, accuracy = 55.600000
k = 1, accuracy = 53.200000
k = 3, accuracy = 47.800000
k = 3, accuracy = 49.800000
k = 3, accuracy = 48.000000
k = 3, accuracy = 53.200000
k = 3, accuracy = 50.800000
k = 5, accuracy = 49.600000
k = 5, accuracy = 53.200000
k = 5, accuracy = 56.000000
k = 5, accuracy = 58.400000
k = 5, accuracy = 56.000000
k = 8, accuracy = 52.400000
k = 8, accuracy = 56.400000
k = 8, accuracy = 54.600000
k = 8, accuracy = 58.000000
k = 8, accuracy = 54.600000
k = 10, accuracy = 53.000000
k = 10, accuracy = 59.200000
k = 10, accuracy = 55.200000
k = 10, accuracy = 56.800000
k = 10, accuracy = 56.000000
k = 12, accuracy = 52.000000
k = 12, accuracy = 59.000000
k = 12, accuracy = 55.800000
k = 12, accuracy = 56.600000
k = 12, accuracy = 56.000000
k = 15, accuracy = 50.400000
k = 15, accuracy = 57.800000
k = 15, accuracy = 55.600000
k = 15, accuracy = 56.400000
k = 15, accuracy = 54.800000
k = 20, accuracy = 54.000000
k = 20, accuracy = 55.800000
k = 20, accuracy = 55.800000
k = 20, accuracy = 56.400000
k = 20, accuracy = 57.000000
k = 50, accuracy = 54.200000
k = 50, accuracy = 57.600000
k = 50, accuracy = 55.600000

```

```

k = 50, accuracy = 53.800000
k = 50, accuracy = 53.200000
k = 100, accuracy = 51.200000
k = 100, accuracy = 54.000000
k = 100, accuracy = 52.600000
k = 100, accuracy = 51.200000
k = 100, accuracy = 52.600000

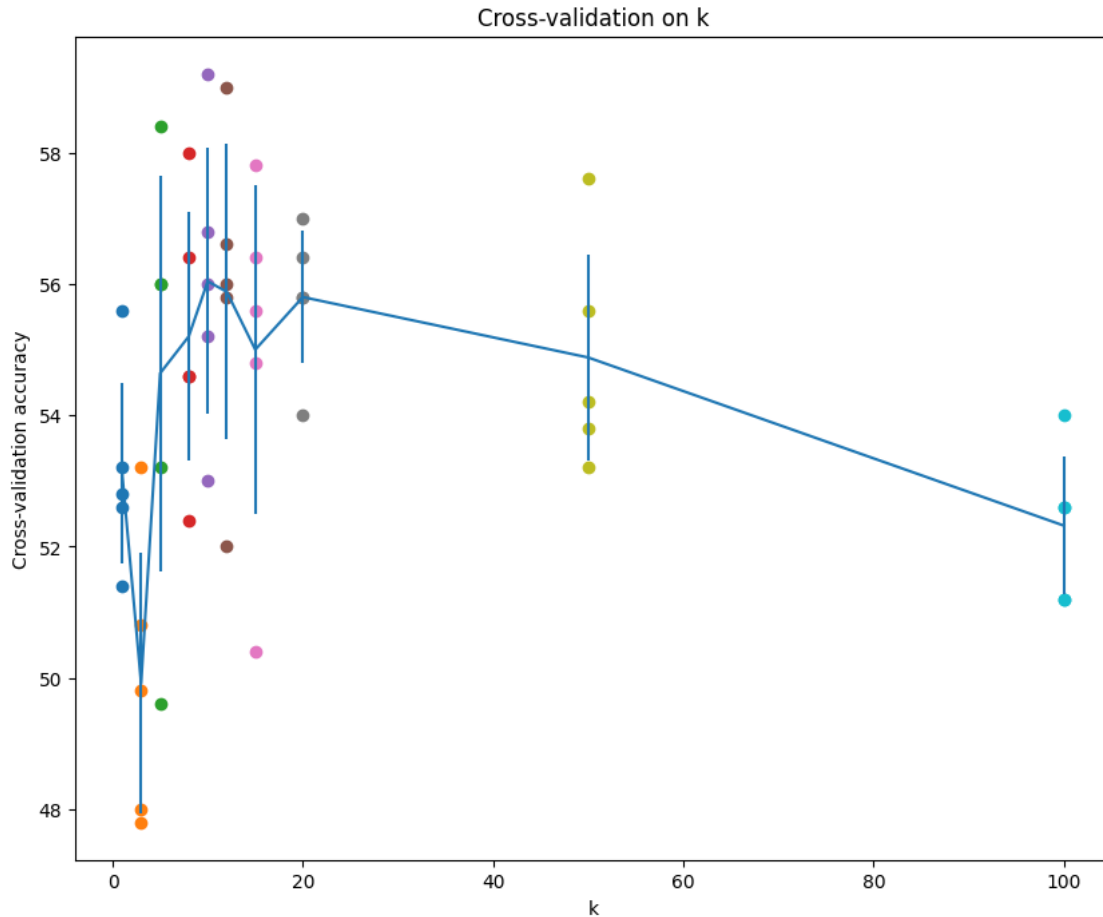
```

```

[ ]: # plot the raw observations
for k in k_choices:
    accuracies = k_to_accuracies[k]
    plt.scatter([k] * len(accuracies), accuracies)

# plot the trend line with error bars that correspond to standard deviation
accuracies_mean = np.array([np.mean(v) for k,v in sorted(k_to_accuracies.
    ↪items())])
accuracies_std = np.array([np.std(v) for k,v in sorted(k_to_accuracies.
    ↪items())])
plt.errorbar(k_choices, accuracies_mean, yerr=accuracies_std)
plt.title('Cross-validation on k')
plt.xlabel('k')
plt.ylabel('Cross-validation accuracy')
plt.show()

```



```
[ ]: # Based on the cross-validation results above, choose the best value for k,
# retrain the classifier using all the training data, and test it on the test
# data. You should be able to get above 28% accuracy on the test data.
best_k = 10

classifier = KNearestNeighbor()
classifier.train(X_train, y_train)
y_test_pred = classifier.predict(X_test, k=best_k)

# Compute and display the accuracy
num_correct = np.sum(y_test_pred == y_test)
accuracy = float(num_correct) / num_test
print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, accuracy))
```

Got 141 / 500 correct => accuracy: 0.282000

Inline Question 3

Which of the following statements about k -Nearest Neighbor (k -NN) are true in a classification

setting, and for all k ? Select all that apply. 1. The decision boundary of the k -NN classifier is linear. 2. The training error of a 1-NN will always be lower than or equal to that of 5-NN. 3. The test error of a 1-NN will always be lower than that of a 5-NN. 4. The time needed to classify a test example with the k -NN classifier grows with the size of the training set. 5. None of the above.

Your Answer :

Câu 2,4 đúng, các câu còn lại là sai

Your Explanation :

Câu 1 sai vì : boundary của k -NN classifier có dạng $\text{argmax}(1 * \text{distance}(\text{test_point}, \text{train_data}))$, với hàm distance là phi tuyến

Câu 2 đúng vì đối với 1 điểm thuộc training data :

- 1NN lấy điểm có khoảng cách nhỏ nhất trong training data $\rightarrow$ chính là điểm đó $\rightarrow$ đúng nhãn.
- 5-NN sẽ xét 5 điểm có khoảng cách nhỏ nhất trong training data $\rightarrow$ có thể phần lớn các điểm không có cùng nhãn.

Câu 3 sai vì 1-NN sẽ kém hơn 5-NN khi dữ liệu train có nhiều

Câu 4 đúng vì k -NN phải chọn label xuất hiện nhiều nhất trong top- k training data gần với test data cho trước, với độ phức tạp là $O(k)$ $\rightarrow$ tăng khi k tăng

Câu 5 sai vì có câu đúng

softmax

April 21, 2026

```
[1]: # This mounts your Google Drive to the Colab VM.
from google.colab import drive
drive.mount('/content/drive',force_remount=True)

# TODO: Enter the foldername in your Drive where you have saved the unzipped
# assignment folder, e.g. 'cs231n/assignments/assignment1/'
FOLDERNAME = 'cs231n/assignments/assignment1/'
assert FOLDERNAME is not None, "[!] Enter the foldername."

# Now that we've mounted your Drive, this ensures that
# the Python interpreter of the Colab VM can load
# python files from within it.
import sys
sys.path.append('/content/drive/My Drive/{}'.format(FOLDERNAME))

# This downloads the CIFAR-10 dataset to your Drive
# if it doesn't already exist.
# %cd /content/drive/My\ Drive/$FOLDERNAME/cs231n/datasets/
# !bash get_datasets.sh
%cd /content/drive/My\ Drive/$FOLDERNAME
```

Mounted at /content/drive
/content/drive/My Drive/cs231n/assignments/assignment1

```
[ ]: # !pip show IPython
```

```
Name: ipython
Version: 7.34.0
Summary: IPython: Productive Interactive Computing
Home-page: https://ipython.org
Author: The IPython Development Team
Author-email: ipython-dev@python.org
License: BSD
Location: /usr/local/lib/python3.11/dist-packages
Requires: backcall, decorator, jedi, matplotlib-inline, pexpect, pickleshare,
prompt-toolkit, pygments, setuptools, traitlets
Required-by: bigquery-magics, cufflinks, datascience, google-colab, ipykernel,
ipyparallel, ipython-sql, ipywidgets, jupyter-console, spanner-graph-notebook
```

```
[ ]: # !pip install --upgrade IPython
```

```
Requirement already satisfied: IPython in /usr/local/lib/python3.11/dist-  
packages (7.34.0)  
Collecting IPython  
  Downloading ipython-9.10.1-py3-none-any.whl.metadata (4.6 kB)  
Requirement already satisfied: decorator>=4.3.2 in  
/usr/local/lib/python3.11/dist-packages (from IPython) (4.4.2)  
Collecting ipython-pygments-lexers>=1.0.0 (from IPython)  
  Downloading ipython_pygments_lexers-1.1.1-py3-none-any.whl.metadata (1.1 kB)  
Collecting jedi>=0.18.1 (from IPython)  
  Downloading jedi-0.19.2-py2.py3-none-any.whl.metadata (22 kB)  
Requirement already satisfied: matplotlib-inline>=0.1.5 in  
/usr/local/lib/python3.11/dist-packages (from IPython) (0.1.7)  
Requirement already satisfied: pexpect>4.3 in /usr/local/lib/python3.11/dist-  
packages (from IPython) (4.9.0)  
Requirement already satisfied: prompt_toolkit<3.1.0,>=3.0.41 in  
/usr/local/lib/python3.11/dist-packages (from IPython) (3.0.51)  
Requirement already satisfied: pygments>=2.11.0 in  
/usr/local/lib/python3.11/dist-packages (from IPython) (2.19.2)  
Collecting stack_data>=0.6.0 (from IPython)  
  Downloading stack_data-0.6.3-py3-none-any.whl.metadata (18 kB)  
Collecting traitlets>=5.13.0 (from IPython)  
  Downloading traitlets-5.14.3-py3-none-any.whl.metadata (10 kB)  
Requirement already satisfied: typing_extensions>=4.6 in  
/usr/local/lib/python3.11/dist-packages (from IPython) (4.14.1)  
Requirement already satisfied: parso<0.9.0,>=0.8.4 in  
/usr/local/lib/python3.11/dist-packages (from jedi>=0.18.1->IPython) (0.8.4)  
Requirement already satisfied: ptyprocess>=0.5 in  
/usr/local/lib/python3.11/dist-packages (from pexpect>4.3->IPython) (0.7.0)  
Requirement already satisfied: wcwidth in /usr/local/lib/python3.11/dist-  
packages (from prompt_toolkit<3.1.0,>=3.0.41->IPython) (0.2.13)  
Collecting executing>=1.2.0 (from stack_data>=0.6.0->IPython)  
  Downloading executing-2.2.1-py2.py3-none-any.whl.metadata (8.9 kB)  
Collecting asttokens>=2.1.0 (from stack_data>=0.6.0->IPython)  
  Downloading asttokens-3.0.1-py3-none-any.whl.metadata (4.9 kB)  
Collecting pure_eval (from stack_data>=0.6.0->IPython)  
  Downloading pure_eval-0.2.3-py3-none-any.whl.metadata (6.3 kB)  
Downloading ipython-9.10.1-py3-none-any.whl (622 kB)  
622.8/622.8 kB  
16.8 MB/s eta 0:00:00  
Downloading ipython_pygments_lexers-1.1.1-py3-none-any.whl (8.1 kB)  
Downloading jedi-0.19.2-py2.py3-none-any.whl (1.6 MB)  
1.6/1.6 MB  
31.8 MB/s eta 0:00:00  
Downloading stack_data-0.6.3-py3-none-any.whl (24 kB)  
Downloading traitlets-5.14.3-py3-none-any.whl (85 kB)
```

85.4/85.4 kB

5.6 MB/s eta 0:00:00

Downloading asttokens-3.0.1-py3-none-any.whl (27 kB)

Downloading executing-2.2.1-py2.py3-none-any.whl (28 kB)

Downloading pure_eval-0.2.3-py3-none-any.whl (11 kB)

Installing collected packages: pure-eval, traitlets, jedi, ipython-pygments-lexers, executing, asttokens, stack_data, IPython

Attempting uninstall: traitlets

Found existing installation: traitlets 5.7.1

Uninstalling traitlets-5.7.1:

Successfully uninstalled traitlets-5.7.1

Attempting uninstall: IPython

Found existing installation: ipython 7.34.0

Uninstalling ipython-7.34.0:

Successfully uninstalled ipython-7.34.0

ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.

google-colab 1.0.0 requires ipython==7.34.0, but you have ipython 9.10.1 which is incompatible.

Successfully installed IPython-9.10.1 asttokens-3.0.1 executing-2.2.1

ipython-pygments-lexers-1.1.1 jedi-0.19.2 pure-eval-0.2.3 stack_data-0.6.3

traitlets-5.14.3

1 Softmax Classifier exercise

Complete and hand in this completed worksheet (including its outputs and any supporting code outside of the worksheet) with your assignment submission. For more details see the [assignments page](#) on the course website.

In this exercise you will:

- implement a fully-vectorized **loss function** for the Softmax classifier.
- implement the fully-vectorized expression for its **analytic gradient**
- **check your implementation** using numerical gradient
- use a validation set to **tune the learning rate and regularization strength**
- **optimize** the loss function with **SGD**
- **visualize** the final learned weights

```
[2]: # Run some setup code for this notebook.
import random
import numpy as np
from cs231n.data_utils import load_CIFAR10
import matplotlib.pyplot as plt

# This is a bit of magic to make matplotlib figures appear inline in the
```

```

# notebook rather than in a new window.
%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# Some more magic so that the notebook will reload external python modules;
# see http://stackoverflow.com/questions/1907993/
↳ autoreload-of-modules-in-ipython
%load_ext autoreload
%autoreload 2

```

1.1 CIFAR-10 Data Loading and Preprocessing

```

[3]: # Load the raw CIFAR-10 data.
cifar10_dir = 'cs231n/datasets/cifar-10-batches-py'

# Cleaning up variables to prevent loading data multiple times (which may cause
↳ memory issue)
try:
    del X_train, y_train
    del X_test, y_test
    print('Clear previously loaded data.')
except:
    pass

X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

# As a sanity check, we print out the size of the training and test data.
print('Training data shape: ', X_train.shape)
print('Training labels shape: ', y_train.shape)
print('Test data shape: ', X_test.shape)
print('Test labels shape: ', y_test.shape)

```

```

Training data shape: (50000, 32, 32, 3)
Training labels shape: (50000,)
Test data shape: (10000, 32, 32, 3)
Test labels shape: (10000,)

```

```

[ ]: # !pip install --upgrade IPython

```

```

Requirement already satisfied: IPython in /usr/local/lib/python3.11/dist-
packages (7.34.0)
Collecting IPython
  Downloading ipython-9.10.1-py3-none-any.whl.metadata (4.6 kB)
Requirement already satisfied: decorator>=4.3.2 in
/usr/local/lib/python3.11/dist-packages (from IPython) (4.4.2)

```



```

Collecting ipython-pygments-lexers>=1.0.0 (from IPython)
  Downloading ipython_pygments_lexers-1.1.1-py3-none-any.whl.metadata (1.1 kB)
Collecting jedi>=0.18.1 (from IPython)
  Downloading jedi-0.19.2-py2.py3-none-any.whl.metadata (22 kB)
Requirement already satisfied: matplotlib-inline>=0.1.5 in
/usr/local/lib/python3.11/dist-packages (from IPython) (0.1.7)
Requirement already satisfied: pexpect>4.3 in /usr/local/lib/python3.11/dist-
packages (from IPython) (4.9.0)
Requirement already satisfied: prompt_toolkit<3.1.0,>=3.0.41 in
/usr/local/lib/python3.11/dist-packages (from IPython) (3.0.51)
Requirement already satisfied: pygments>=2.11.0 in
/usr/local/lib/python3.11/dist-packages (from IPython) (2.19.2)
Collecting stack_data>=0.6.0 (from IPython)
  Downloading stack_data-0.6.3-py3-none-any.whl.metadata (18 kB)
Collecting traitlets>=5.13.0 (from IPython)
  Downloading traitlets-5.14.3-py3-none-any.whl.metadata (10 kB)
Requirement already satisfied: typing_extensions>=4.6 in
/usr/local/lib/python3.11/dist-packages (from IPython) (4.14.1)
Requirement already satisfied: parso<0.9.0,>=0.8.4 in
/usr/local/lib/python3.11/dist-packages (from jedi>=0.18.1->IPython) (0.8.4)
Requirement already satisfied: ptyprocess>=0.5 in
/usr/local/lib/python3.11/dist-packages (from pexpect>4.3->IPython) (0.7.0)
Requirement already satisfied: wcwidth in /usr/local/lib/python3.11/dist-
packages (from prompt_toolkit<3.1.0,>=3.0.41->IPython) (0.2.13)
Collecting executing>=1.2.0 (from stack_data>=0.6.0->IPython)
  Downloading executing-2.2.1-py2.py3-none-any.whl.metadata (8.9 kB)
Collecting asttokens>=2.1.0 (from stack_data>=0.6.0->IPython)
  Downloading asttokens-3.0.1-py3-none-any.whl.metadata (4.9 kB)
Collecting pure-eval (from stack_data>=0.6.0->IPython)
  Downloading pure_eval-0.2.3-py3-none-any.whl.metadata (6.3 kB)
Downloading ipython-9.10.1-py3-none-any.whl (622 kB)
      622.8/622.8 kB
11.9 MB/s eta 0:00:00
Downloading ipython_pygments_lexers-1.1.1-py3-none-any.whl (8.1 kB)
Downloading jedi-0.19.2-py2.py3-none-any.whl (1.6 MB)
      1.6/1.6 MB
43.2 MB/s eta 0:00:00
Downloading stack_data-0.6.3-py3-none-any.whl (24 kB)
Downloading traitlets-5.14.3-py3-none-any.whl (85 kB)
      85.4/85.4 kB
9.9 MB/s eta 0:00:00
Downloading asttokens-3.0.1-py3-none-any.whl (27 kB)
Downloading executing-2.2.1-py2.py3-none-any.whl (28 kB)
Downloading pure_eval-0.2.3-py3-none-any.whl (11 kB)
Installing collected packages: pure-eval, traitlets, jedi, ipython-pygments-
lexers, executing, asttokens, stack_data, IPython
  Attempting uninstall: traitlets
    Found existing installation: traitlets 5.7.1

```

```

Uninstalling traitlets-5.7.1:
  Successfully uninstalled traitlets-5.7.1
Attempting uninstall: IPython
  Found existing installation: ipython 7.34.0
  Uninstalling ipython-7.34.0:
    Successfully uninstalled ipython-7.34.0
ERROR: pip's dependency resolver does not currently take into account all
the packages that are installed. This behaviour is the source of the following
dependency conflicts.

google-colab 1.0.0 requires ipython==7.34.0, but you have ipython 9.10.1 which
is incompatible.

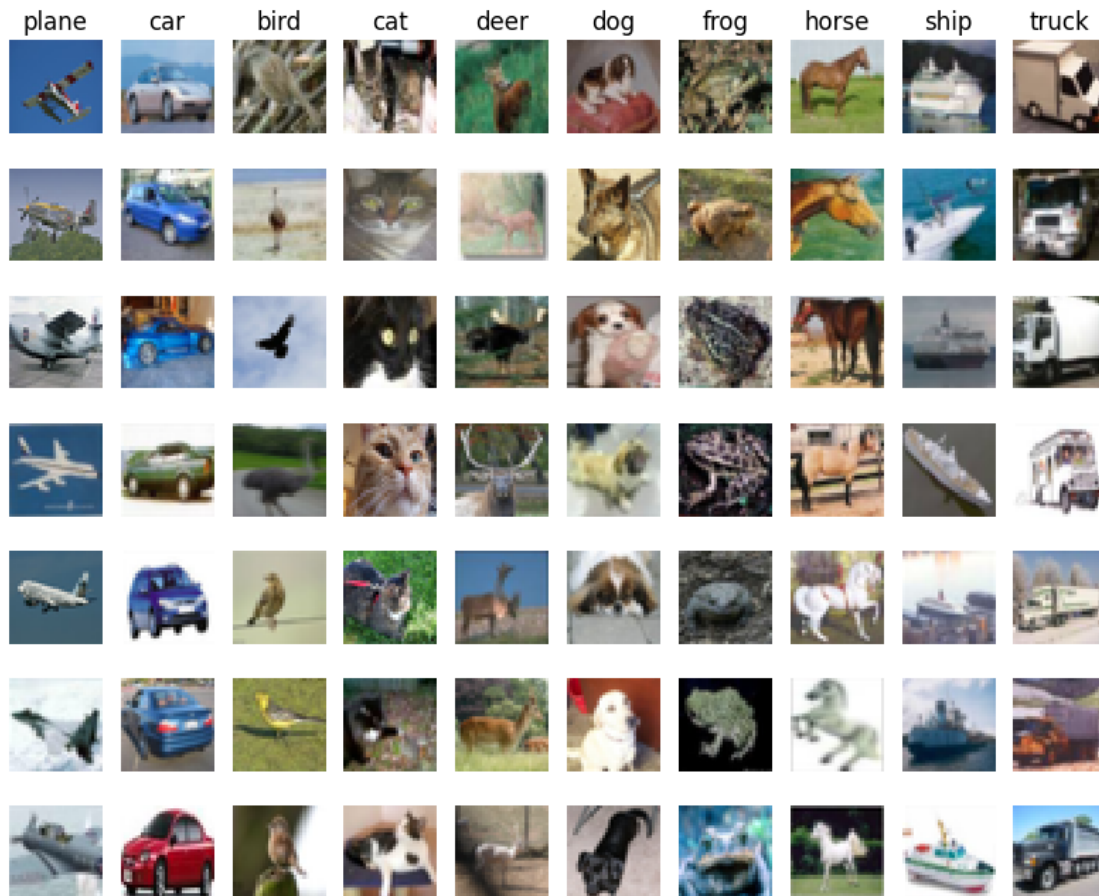
Successfully installed IPython-9.10.1 asttokens-3.0.1 executing-2.2.1
ipython-pygments-lexers-1.1.1 jedi-0.19.2 pure-eval-0.2.3 stack_data-0.6.3
traitlets-5.14.3

```

```

[4]: # Visualize some examples from the dataset.
     # We show a few examples of training images from each class.
     classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']
     num_classes = len(classes)
     samples_per_class = 7
     for y, cls in enumerate(classes):
         idxs = np.flatnonzero(y_train == y)
         idxs = np.random.choice(idxs, samples_per_class, replace=False)
         for i, idx in enumerate(idxs):
             plt_idx = i * num_classes + y + 1
             plt.subplot(samples_per_class, num_classes, plt_idx)
             plt.imshow(X_train[idx].astype('uint8'))
             plt.axis('off')
             if i == 0:
                 plt.title(cls)
     plt.show()

```



```
[5]: # Split the data into train, val, and test sets. In addition we will
# create a small development set as a subset of the training data;
# we can use this for development so our code runs faster.
num_training = 49000
num_validation = 1000
num_test = 1000
num_dev = 500

# Our validation set will be num_validation points from the original
# training set.
mask = range(num_training, num_training + num_validation)
X_val = X_train[mask]
y_val = y_train[mask]

# Our training set will be the first num_train points from the original
# training set.
mask = range(num_training)
X_train = X_train[mask]
y_train = y_train[mask]
```

```

# We will also make a development set, which is a small subset of
# the training set.
mask = np.random.choice(num_training, num_dev, replace=False)
X_dev = X_train[mask]
y_dev = y_train[mask]

# We use the first num_test points of the original test set as our
# test set.
mask = range(num_test)
X_test = X_test[mask]
y_test = y_test[mask]

print('Train data shape: ', X_train.shape)
print('Train labels shape: ', y_train.shape)
print('Validation data shape: ', X_val.shape)
print('Validation labels shape: ', y_val.shape)
print('Test data shape: ', X_test.shape)
print('Test labels shape: ', y_test.shape)

```

```

Train data shape: (49000, 32, 32, 3)
Train labels shape: (49000,)
Validation data shape: (1000, 32, 32, 3)
Validation labels shape: (1000,)
Test data shape: (1000, 32, 32, 3)
Test labels shape: (1000,)

```

```

[6]: # Preprocessing: reshape the image data into rows
X_train = np.reshape(X_train, (X_train.shape[0], -1))
X_val = np.reshape(X_val, (X_val.shape[0], -1))
X_test = np.reshape(X_test, (X_test.shape[0], -1))
X_dev = np.reshape(X_dev, (X_dev.shape[0], -1))

# As a sanity check, print out the shapes of the data
print('Training data shape: ', X_train.shape)
print('Validation data shape: ', X_val.shape)
print('Test data shape: ', X_test.shape)
print('dev data shape: ', X_dev.shape)

```

```

Training data shape: (49000, 3072)
Validation data shape: (1000, 3072)
Test data shape: (1000, 3072)
dev data shape: (500, 3072)

```

```

[7]: # Preprocessing: subtract the mean image
# first: compute the image mean based on the training data
mean_image = np.mean(X_train, axis=0)

```

```

print(mean_image[:10]) # print a few of the elements
plt.figure(figsize=(4,4))
plt.imshow(mean_image.reshape((32,32,3)).astype('uint8')) # visualize the mean_
    ↪image
plt.show()

# second: subtract the mean image from train and test data
X_train -= mean_image
X_val -= mean_image
X_test -= mean_image
X_dev -= mean_image

# third: append the bias dimension of ones (i.e. bias trick) so that our_
    ↪classifier
# only has to worry about optimizing a single weight matrix W.
X_train = np.hstack([X_train, np.ones((X_train.shape[0], 1))])
X_val = np.hstack([X_val, np.ones((X_val.shape[0], 1))])
X_test = np.hstack([X_test, np.ones((X_test.shape[0], 1))])
X_dev = np.hstack([X_dev, np.ones((X_dev.shape[0], 1))])

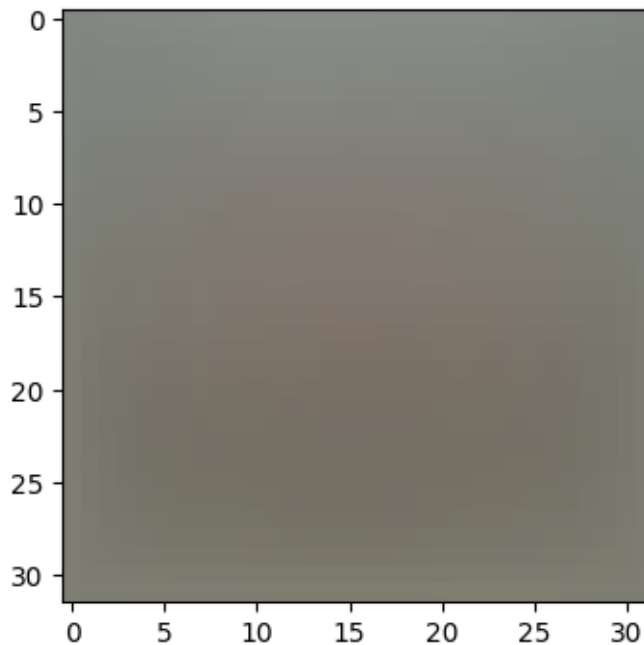
print(X_train.shape, X_val.shape, X_test.shape, X_dev.shape)

```

```

[130.64189796 135.98173469 132.47391837 130.05569388 135.34804082
 131.75402041 130.96055102 136.14328571 132.47636735 131.48467347]

```



(49000, 3073) (1000, 3073) (1000, 3073) (500, 3073)

1.2 Softmax Classifier

Your code for this section will all be written inside `cs231n/classifiers/softmax.py`.

As you can see, we have prefilled the function `softmax_loss_naive` which uses for loops to evaluate the softmax loss function.

```
[8]: # Evaluate the naive implementation of the loss we provided for you:
from cs231n.classifiers.softmax import softmax_loss_naive
import time

# generate a random Softmax classifier weight matrix of small numbers
W = np.random.randn(3073, 10) * 0.0001

loss, grad = softmax_loss_naive(W, X_dev, y_dev, 0.000005)
print('loss: %f' % (loss, ))

# As a rough sanity check, our loss should be something close to -log(0.1).
print('loss: %f' % loss)
print('sanity check: %f' % (-np.log(0.1)))
```

```
loss: 2.358833
loss: 2.358833
sanity check: 2.302585
```

Inline Question 1

Why do we expect our loss to be close to $-\log(0.1)$? Explain briefly.**

Your Answer : Fill this in

Hàm loss của softmax là lấy trung bình giữa các $y_i * \log(p_{\{y_i\}})$ nên loss ban đầu chính là trung bình của các $\log(p_{\{y_i\}})$.

Có 10 class

→ Sau khi khởi tạo với trọng số ngẫu nhiên nhỏ khiến giá trị của từng $p_{\{class\}}$ là như nhau → khả năng $p_{\{y_i\}}$ chỉ quanh quanh 0.1

The `grad` returned from the function above is right now all zero. Derive and implement the gradient for the softmax loss function and implement it inline inside the function `softmax_loss_naive`. You will find it helpful to interleave your new code inside the existing function.

To check that you have correctly implemented the gradient, you can numerically estimate the gradient of the loss function and compare the numeric estimate to the gradient that you computed. We have provided code that does this for you:

```
[29]: # Once you've implemented the gradient, recompute it with the code below
# and gradient check it with the function we provided for you

# Compute the loss and its gradient at W.
```

```

loss, grad = softmax_loss_naive(W, X_dev, y_dev, 0.0)

# Numerically compute the gradient along several randomly chosen dimensions, and
# compare them with your analytically computed gradient. The numbers should
↪match
# almost exactly along all dimensions.
from cs231n.gradient_check import grad_check_sparse
f = lambda w: softmax_loss_naive(w, X_dev, y_dev, 0.0)[0]
grad_numerical = grad_check_sparse(f, W, grad)

# do the gradient check once again with regularization turned on
# you didn't forget the regularization gradient did you?
loss, grad = softmax_loss_naive(W, X_dev, y_dev, 5e1)
f = lambda w: softmax_loss_naive(w, X_dev, y_dev, 5e1)[0]
grad_numerical = grad_check_sparse(f, W, grad)

```

```

numerical: -1.352467 analytic: -1.352467, relative error: 9.990241e-09
numerical: 2.346673 analytic: 2.346673, relative error: 2.906986e-09
numerical: -2.426967 analytic: -2.426967, relative error: 2.356905e-08
numerical: -0.930560 analytic: -0.930560, relative error: 2.324334e-09
numerical: -1.936973 analytic: -1.936973, relative error: 1.099188e-08
numerical: 0.633299 analytic: 0.633299, relative error: 8.895969e-08
numerical: 0.986051 analytic: 0.986051, relative error: 1.198038e-07
numerical: -0.450097 analytic: -0.450097, relative error: 1.662930e-08
numerical: -0.429554 analytic: -0.429554, relative error: 4.403477e-08
numerical: 1.470964 analytic: 1.470964, relative error: 2.211943e-08
numerical: 3.813504 analytic: 3.813503, relative error: 2.573270e-08
numerical: 3.658487 analytic: 3.658487, relative error: 1.308049e-08
numerical: 0.814181 analytic: 0.814181, relative error: 9.127460e-08
numerical: -3.925308 analytic: -3.925308, relative error: 2.307766e-08
numerical: 2.361946 analytic: 2.361946, relative error: 3.061259e-08
numerical: -1.928113 analytic: -1.928113, relative error: 3.993687e-09
numerical: -0.333560 analytic: -0.333560, relative error: 1.237340e-08
numerical: 0.328549 analytic: 0.328549, relative error: 4.141665e-08
numerical: 4.079781 analytic: 4.079781, relative error: 1.272005e-08
numerical: 0.462583 analytic: 0.462583, relative error: 5.134649e-08

```

Inline Question 2

Although gradcheck is reliable softmax loss, it is possible that for SVM loss, once in a while, a dimension in the gradcheck will not match exactly. What could such a discrepancy be caused by? Is it a reason for concern? What is a simple example in one dimension where a svm loss gradient check could fail? How would change the margin affect of the frequency of this happening?

Note that SVM loss for a sample (x_i, y_i) is defined as:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + \Delta)$$

where j iterates over all classes except the correct class y_i and s_j denotes the classifier score for

j^{th} class. Δ is a scalar margin. For more information, refer to ‘Multiclass Support Vector Machine loss’ on [this](#) page.

Hint: the SVM loss function is not strictly speaking differentiable.

Your Answer : fill this in.

1. gradcheck sử dụng công thức xấp xỉ $(f(w+h) - f(w-h))/2h$ để xấp xỉ gradient tại điểm w . Tuy nhiên với các w mà khiến cho

$$(s_j - s_{y_i} + \Delta)$$

sát 0 (bên 0+ hoặc 0-), tức khoảng cách của nó đến 0 còn nhỏ hơn cả h , thì phép xấp xỉ trên không đúng. Lí do là hàm $f(x) = \max(0, x)$ không có đạo hàm tại $x = 0$.

2. Không quá concern, vì xác suất rơi khiến

$$(s_j - s_{y_i} + \Delta)$$

sát 0 là thấp.

3. ví dụ 1 chiều, $f(x) = \max(0, x)$. $h = 1e-6$, $x = 1e-10$ (lúc này x nhỏ hơn h rất nhiều). đạo hàm xấp xỉ là $(1e-10 + 1e-6)/2(1e-6) = 1/2 + 1e-4$. Trong khi $f'(x)$ tại $x=1e-10$ thì bằng 1.
4. việc thay đổi margin không ảnh hưởng đến tần suất của điểm kinks. Nó chỉ dịch chuyển các điểm kinks này. Còn cái gây sai số xấp xỉ phụ thuộc chủ yếu vào h (h càng nhỏ càng tốt)/

```
[45]: # Next implement the function softmax_loss_vectorized; for now only compute the
      ↪ loss;
      # we will implement the gradient in a moment.
      tic = time.time()
      loss_naive, grad_naive = softmax_loss_naive(W, X_dev, y_dev, 0.000005)
      toc = time.time()
      print('Naive loss: %e computed in %fs' % (loss_naive, toc - tic))

      from cs231n.classifiers.softmax import softmax_loss_vectorized
      tic = time.time()
      loss_vectorized, _ = softmax_loss_vectorized(W, X_dev, y_dev, 0.000005)
      toc = time.time()
      print('Vectorized loss: %e computed in %fs' % (loss_vectorized, toc - tic))

      # The losses should match but your vectorized implementation should be much
      ↪ faster.
      print('difference: %f' % (loss_naive - loss_vectorized))
```

Naive loss: 2.358833e+00 computed in 0.090157s
(500,)

Vectorized loss: 2.358833e+00 computed in 0.010496s
difference: -0.000000

```
[48]: # Complete the implementation of softmax_loss_vectorized, and compute the
      ↪ gradient
      # of the loss function in a vectorized way.
```



```

# The naive implementation and the vectorized implementation should match, but
# the vectorized version should still be much faster.
tic = time.time()
_, grad_naive = softmax_loss_naive(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Naive loss and gradient: computed in %fs' % (toc - tic))

tic = time.time()
_, grad_vectorized = softmax_loss_vectorized(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Vectorized loss and gradient: computed in %fs' % (toc - tic))

# The loss is a single number, so it is easy to compare the values computed
# by the two implementations. The gradient on the other hand is a matrix, so
# we use the Frobenius norm to compare them.
print("grad_naive = %f \n " % (np.mean(grad_naive)))
print("grad_vectorized = %f \n" % (np.mean(grad_vectorized)))
difference = np.linalg.norm(grad_naive - grad_vectorized, ord='fro')
print('difference: %f' % difference)

```

```

Naive loss and gradient: computed in 0.090571s
Vectorized loss and gradient: computed in 0.011064s
grad_naive = 0.000000

grad_vectorized = 0.000000

difference: 0.000000

```

1.2.1 Stochastic Gradient Descent

We now have vectorized and efficient expressions for the loss, the gradient and our gradient matches the numerical gradient. We are therefore ready to do SGD to minimize the loss. Your code for this part will be written inside `cs231n/classifiers/linear_classifier.py`.

```

[49]: # In the file linear_classifier.py, implement SGD in the function
# LinearClassifier.train() and then run it with the code below.
from cs231n.classifiers import Softmax
softmax = Softmax()
tic = time.time()
loss_hist = softmax.train(X_train, y_train, learning_rate=1e-7, reg=2.5e4,
                          num_iters=1500, verbose=True)
toc = time.time()
print('That took %fs' % (toc - tic))

```

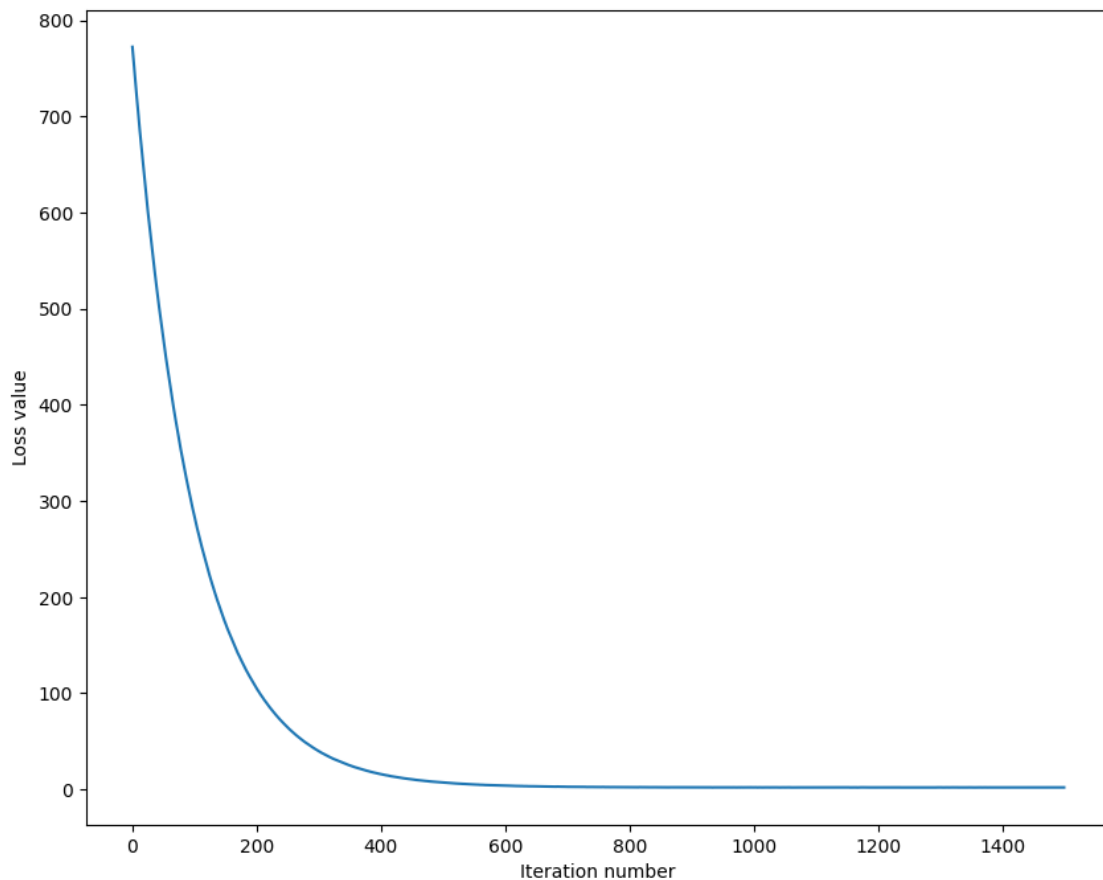
```

iteration 0 / 1500: loss 772.342014
iteration 100 / 1500: loss 283.734806
iteration 200 / 1500: loss 105.012560

```

```
iteration 300 / 1500: loss 39.807126
iteration 400 / 1500: loss 15.932476
iteration 500 / 1500: loss 7.193760
iteration 600 / 1500: loss 3.939022
iteration 700 / 1500: loss 2.712573
iteration 800 / 1500: loss 2.331121
iteration 900 / 1500: loss 2.237885
iteration 1000 / 1500: loss 2.091852
iteration 1100 / 1500: loss 2.081488
iteration 1200 / 1500: loss 2.074932
iteration 1300 / 1500: loss 2.127004
iteration 1400 / 1500: loss 2.100776
That took 14.492887s
```

```
[50]: # A useful debugging strategy is to plot the loss as a function of  
# iteration number:  
plt.plot(loss_hist)  
plt.xlabel('Iteration number')  
plt.ylabel('Loss value')  
plt.show()
```



```
[51]: # Write the LinearClassifier.predict function and evaluate the performance on
# both the training and validation set
# You should get validation accuracy of about 0.34 (> 0.33).
y_train_pred = softmax.predict(X_train)
print('training accuracy: %f' % (np.mean(y_train == y_train_pred), ))
y_val_pred = softmax.predict(X_val)
print('validation accuracy: %f' % (np.mean(y_val == y_val_pred), ))
```

```
training accuracy: 0.331265
validation accuracy: 0.352000
```

```
[52]: # Save the trained model for autograder.
softmax.save("softmax.npy")
```

softmax.npy saved.

```
[54]: # Use the validation set to tune hyperparameters (regularization strength and
# learning rate). You should experiment with different ranges for the learning
# rates and regularization strengths; if you are careful you should be able to
# get a classification accuracy of about 0.365 (> 0.36) on the validation set.

# Note: you may see runtime/overflow warnings during hyper-parameter search.
# This may be caused by extreme values, and is not a bug.

# results is dictionary mapping tuples of the form
# (learning_rate, regularization_strength) to tuples of the form
# (training_accuracy, validation_accuracy). The accuracy is simply the fraction
# of data points that are correctly classified.
results = {}
best_val = -1 # The highest validation accuracy that we have seen so far.
best_softmax = None # The Softmax object that achieved the highest validation
    ↪rate.

#####
# TODO: #
# Write code that chooses the best hyperparameters by tuning on the validation #
# set. For each combination of hyperparameters, train a Softmax on the. #
# training set, compute its accuracy on the training and validation sets, and #
# store these numbers in the results dictionary. In addition, store the best #
# validation accuracy in best_val and the Softmax object that achieves this. #
# accuracy in best_softmax. #
# #
# Hint: You should use a small value for num_iters as you develop your #
# validation code so that the classifiers don't take much time to train; once #
# you are confident that your validation code works, you should rerun the #
# code with a larger value for num_iters. #
```

```
#####

# Provided as a reference. You may or may not want to change these
↳ hyperparameters
learning_rates = [1e-7, 1e-6]
regularization_strengths = [2.5e4, 1e4]

for lr in learning_rates:
    for reg in regularization_strengths:
        softmax = Softmax()
        # W = np.random.randn(3073, 10) * 0.0001
        softmax.train(X_train, y_train, learning_rate=lr, reg=reg,
                      num_iters=1500, verbose=True)
        y_train_pred = softmax.predict(X_train)
        print('training accuracy: %f' % (np.mean(y_train == y_train_pred), ))
        y_val_pred = softmax.predict(X_val)
        print('validation accuracy: %f' % (np.mean(y_val == y_val_pred), ))
        results[(lr,reg)] = (np.mean(y_train == y_train_pred), np.mean(y_val ==
↳ y_val_pred))
        if np.mean(y_val == y_val_pred) > best_val:
            best_val = np.mean(y_val == y_val_pred)
            best_softmax = softmax

# Print out results.
for lr, reg in sorted(results):
    train_accuracy, val_accuracy = results[(lr, reg)]
    print('lr %e reg %e train accuracy: %f val accuracy: %f' % (
        lr, reg, train_accuracy, val_accuracy))

print('best validation accuracy achieved during cross-validation: %f' %
↳ best_val)
```

```
iteration 0 / 1500: loss 766.908248
iteration 100 / 1500: loss 281.871023
iteration 200 / 1500: loss 104.442712
iteration 300 / 1500: loss 39.545839
iteration 400 / 1500: loss 15.725316
iteration 500 / 1500: loss 7.118611
iteration 600 / 1500: loss 3.960074
iteration 700 / 1500: loss 2.774735
iteration 800 / 1500: loss 2.330816
iteration 900 / 1500: loss 2.196706
iteration 1000 / 1500: loss 2.184808
iteration 1100 / 1500: loss 2.185608
```

iteration 1200 / 1500: loss 2.053612
iteration 1300 / 1500: loss 2.038320
iteration 1400 / 1500: loss 2.156073
training accuracy: 0.326122
validation accuracy: 0.338000
iteration 0 / 1500: loss 317.067593
iteration 100 / 1500: loss 211.372632
iteration 200 / 1500: loss 141.809005
iteration 300 / 1500: loss 95.436847
iteration 400 / 1500: loss 64.380946
iteration 500 / 1500: loss 43.626726
iteration 600 / 1500: loss 29.811728
iteration 700 / 1500: loss 20.620528
iteration 800 / 1500: loss 14.481002
iteration 900 / 1500: loss 10.341956
iteration 1000 / 1500: loss 7.673530
iteration 1100 / 1500: loss 5.750907
iteration 1200 / 1500: loss 4.487285
iteration 1300 / 1500: loss 3.673547
iteration 1400 / 1500: loss 3.179245
training accuracy: 0.351612
validation accuracy: 0.362000
iteration 0 / 1500: loss 773.186055
iteration 100 / 1500: loss 2.091827
iteration 200 / 1500: loss 2.020803
iteration 300 / 1500: loss 2.133183
iteration 400 / 1500: loss 2.093173
iteration 500 / 1500: loss 2.043410
iteration 600 / 1500: loss 2.118333
iteration 700 / 1500: loss 2.111725
iteration 800 / 1500: loss 2.042610
iteration 900 / 1500: loss 2.031525
iteration 1000 / 1500: loss 2.096353
iteration 1100 / 1500: loss 2.152428
iteration 1200 / 1500: loss 2.062559
iteration 1300 / 1500: loss 2.140925
iteration 1400 / 1500: loss 2.105816
training accuracy: 0.303306
validation accuracy: 0.324000
iteration 0 / 1500: loss 311.093468
iteration 100 / 1500: loss 7.332537
iteration 200 / 1500: loss 2.110877
iteration 300 / 1500: loss 1.994017
iteration 400 / 1500: loss 1.997741
iteration 500 / 1500: loss 1.991063
iteration 600 / 1500: loss 2.035516
iteration 700 / 1500: loss 1.961851
iteration 800 / 1500: loss 2.004965

```

iteration 900 / 1500: loss 2.051754
iteration 1000 / 1500: loss 2.056548
iteration 1100 / 1500: loss 2.059154
iteration 1200 / 1500: loss 2.060752
iteration 1300 / 1500: loss 2.072749
iteration 1400 / 1500: loss 2.067661
training accuracy: 0.352857
validation accuracy: 0.370000
lr 1.000000e-07 reg 1.000000e+04 train accuracy: 0.351612 val accuracy: 0.362000
lr 1.000000e-07 reg 2.500000e+04 train accuracy: 0.326122 val accuracy: 0.338000
lr 1.000000e-06 reg 1.000000e+04 train accuracy: 0.352857 val accuracy: 0.370000
lr 1.000000e-06 reg 2.500000e+04 train accuracy: 0.303306 val accuracy: 0.324000
best validation accuracy achieved during cross-validation: 0.370000

```

```

[55]: # Visualize the cross-validation results
import math
import pdb

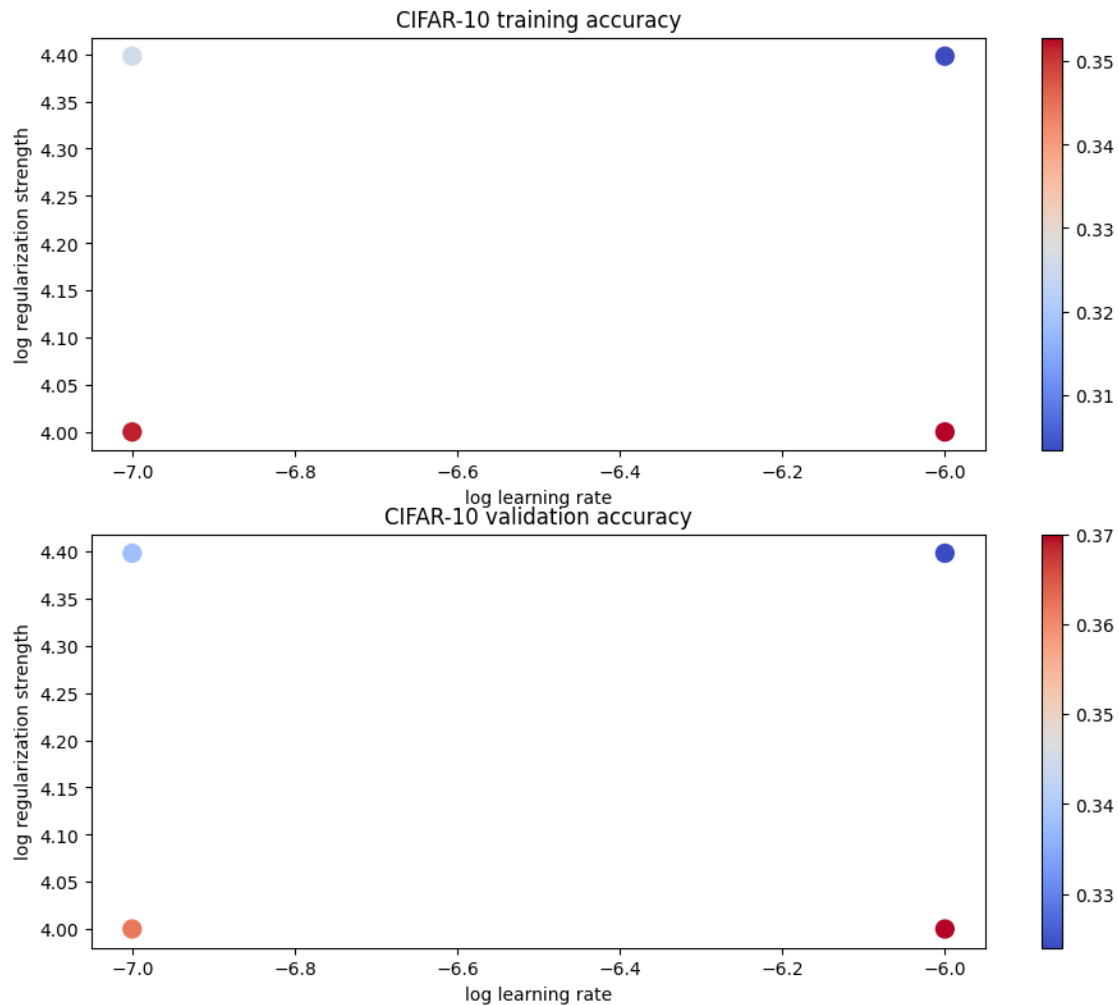
# pdb.set_trace()

x_scatter = [math.log10(x[0]) for x in results]
y_scatter = [math.log10(x[1]) for x in results]

# plot training accuracy
marker_size = 100
colors = [results[x][0] for x in results]
plt.subplot(2, 1, 1)
plt.tight_layout(pad=3)
plt.scatter(x_scatter, y_scatter, marker_size, c=colors, cmap=plt.cm.coolwarm)
plt.colorbar()
plt.xlabel('log learning rate')
plt.ylabel('log regularization strength')
plt.title('CIFAR-10 training accuracy')

# plot validation accuracy
colors = [results[x][1] for x in results] # default size of markers is 20
plt.subplot(2, 1, 2)
plt.scatter(x_scatter, y_scatter, marker_size, c=colors, cmap=plt.cm.coolwarm)
plt.colorbar()
plt.xlabel('log learning rate')
plt.ylabel('log regularization strength')
plt.title('CIFAR-10 validation accuracy')
plt.show()

```



```
[56]: # Evaluate the best softmax on test set
y_test_pred = best_softmax.predict(X_test)
test_accuracy = np.mean(y_test == y_test_pred)
print('Softmax classifier on raw pixels final test set accuracy: %f' %
      test_accuracy)
```

Softmax classifier on raw pixels final test set accuracy: 0.362000

```
[57]: # Save best softmax model
best_softmax.save("best_softmax.npy")
```

best_softmax.npy saved.

```
[58]: # Visualize the learned weights for each class.
# Depending on your choice of learning rate and regularization strength, these
      may
```

```
# or may not be nice to look at.
w = best_softmax.W[:-1,:] # strip out the bias
w = w.reshape(32, 32, 3, 10)
w_min, w_max = np.min(w), np.max(w)
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']
for i in range(10):
    plt.subplot(2, 5, i + 1)

    # Rescale the weights to be between 0 and 255
    wimg = 255.0 * (w[:, :, :, i].squeeze() - w_min) / (w_max - w_min)
    plt.imshow(wimg.astype('uint8'))
    plt.axis('off')
    plt.title(classes[i])
```



Inline question 3

Describe what your visualized Softmax classifier weights look like, and offer a brief explanation for why they look the way they do.

Your Answer : fill this in

weight cho từng class nhìn gần giống ảnh của class đó. Điều này là do Softmax khiến khuyến khích $\text{weight}[i] * x$ của từng class i , tức là véc tơ $\text{weight}[i]$ sẽ sát với x . Dẫn đến $\text{weight}[i]$ nhìn gần giống các ảnh nhãn i .

Inline Question 4 - *True or False*

Suppose the overall training loss is defined as the sum of the per-datapoint loss over all training examples. It is possible to add a new datapoint to a training set that would change the softmax loss, but leave the SVM loss unchanged.

Your Answer :

Có thể

Your Explanation :

Theo công thức thì nó sẽ thêm một số hạng mới khác 0 vào phần tổng của hàm loss softmax -> tổng này sẽ có thay đổi chút

Nếu dữ liệu training này khiến cho

$$s_j - s_{y_i} + \Delta < 0$$

với mọi j thì hàm loss sẽ không đổi do phần loss

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + \Delta) = 0$$

[]:

two_layer_net

April 21, 2026

```
[1]: # This mounts your Google Drive to the Colab VM.
from google.colab import drive
drive.mount('/content/drive',force_remount=True)

# TODO: Enter the foldername in your Drive where you have saved the unzipped
# assignment folder, e.g. 'cs231n/assignments/assignment1/'
FOLDERNAME = 'cs231n/assignments/assignment1/'
assert FOLDERNAME is not None, "[!] Enter the foldername."

# Now that we've mounted your Drive, this ensures that
# the Python interpreter of the Colab VM can load
# python files from within it.
import sys
sys.path.append('/content/drive/My Drive/{}'.format(FOLDERNAME))

# This downloads the CIFAR-10 dataset to your Drive
# if it doesn't already exist.
# %cd /content/drive/My\ Drive/$FOLDERNAME/cs231n/datasets/
# !bash get_datasets.sh
%cd /content/drive/My\ Drive/$FOLDERNAME
```

Mounted at /content/drive
/content/drive/.shortcut-targets-by-
id/16YEzBI860N8oXEabllZ2vheRhnpKWioQ/cs231n/assignments/assignment1

```
[5]: # !pip install --upgrade IPython
```

Requirement already satisfied: IPython in /usr/local/lib/python3.11/dist-packages (7.34.0)
Collecting IPython
 Downloading ipython-9.10.1-py3-none-any.whl.metadata (4.6 kB)
Requirement already satisfied: decorator>=4.3.2 in /usr/local/lib/python3.11/dist-packages (from IPython) (4.4.2)
Collecting ipython-pygments-lexers>=1.0.0 (from IPython)
 Downloading ipython_pygments_lexers-1.1.1-py3-none-any.whl.metadata (1.1 kB)
Collecting jedi>=0.18.1 (from IPython)
 Downloading jedi-0.19.2-py2.py3-none-any.whl.metadata (22 kB)
Requirement already satisfied: matplotlib-inline>=0.1.5 in

```

/usr/local/lib/python3.11/dist-packages (from IPython) (0.1.7)
Requirement already satisfied: pexpect>4.3 in /usr/local/lib/python3.11/dist-
packages (from IPython) (4.9.0)
Requirement already satisfied: prompt_toolkit<3.1.0,>=3.0.41 in
/usr/local/lib/python3.11/dist-packages (from IPython) (3.0.51)
Requirement already satisfied: pygments>=2.11.0 in
/usr/local/lib/python3.11/dist-packages (from IPython) (2.19.2)
Collecting stack_data>=0.6.0 (from IPython)
  Downloading stack_data-0.6.3-py3-none-any.whl.metadata (18 kB)
Collecting traitlets>=5.13.0 (from IPython)
  Downloading traitlets-5.14.3-py3-none-any.whl.metadata (10 kB)
Requirement already satisfied: typing_extensions>=4.6 in
/usr/local/lib/python3.11/dist-packages (from IPython) (4.14.1)
Requirement already satisfied: parso<0.9.0,>=0.8.4 in
/usr/local/lib/python3.11/dist-packages (from jedi>=0.18.1->IPython) (0.8.4)
Requirement already satisfied: ptyprocess>=0.5 in
/usr/local/lib/python3.11/dist-packages (from pexpect>4.3->IPython) (0.7.0)
Requirement already satisfied: wcwidth in /usr/local/lib/python3.11/dist-
packages (from prompt_toolkit<3.1.0,>=3.0.41->IPython) (0.2.13)
Collecting executing>=1.2.0 (from stack_data>=0.6.0->IPython)
  Downloading executing-2.2.1-py2.py3-none-any.whl.metadata (8.9 kB)
Collecting asttokens>=2.1.0 (from stack_data>=0.6.0->IPython)
  Downloading asttokens-3.0.1-py3-none-any.whl.metadata (4.9 kB)
Collecting pure-eval (from stack_data>=0.6.0->IPython)
  Downloading pure_eval-0.2.3-py3-none-any.whl.metadata (6.3 kB)
Downloading ipython-9.10.1-py3-none-any.whl (622 kB)
      622.8/622.8 kB
40.6 MB/s eta 0:00:00
Downloading ipython_pygments_lexers-1.1.1-py3-none-any.whl (8.1 kB)
Downloading jedi-0.19.2-py2.py3-none-any.whl (1.6 MB)
      1.6/1.6 MB
59.8 MB/s eta 0:00:00
Downloading stack_data-0.6.3-py3-none-any.whl (24 kB)
Downloading traitlets-5.14.3-py3-none-any.whl (85 kB)
      85.4/85.4 kB
6.4 MB/s eta 0:00:00
Downloading asttokens-3.0.1-py3-none-any.whl (27 kB)
Downloading executing-2.2.1-py2.py3-none-any.whl (28 kB)
Downloading pure_eval-0.2.3-py3-none-any.whl (11 kB)
Installing collected packages: pure-eval, traitlets, jedi, ipython-pygments-
lexers, executing, asttokens, stack_data, IPython
  Attempting uninstall: traitlets
    Found existing installation: traitlets 5.7.1
    Uninstalling traitlets-5.7.1:
      Successfully uninstalled traitlets-5.7.1
  Attempting uninstall: IPython
    Found existing installation: ipython 7.34.0
    Uninstalling ipython-7.34.0:

```

```
Successfully uninstalled ipython-7.34.0
ERROR: pip's dependency resolver does not currently take into account all
the packages that are installed. This behaviour is the source of the following
dependency conflicts.
google-colab 1.0.0 requires ipython==7.34.0, but you have ipython 9.10.1 which
is incompatible.

Successfully installed IPython-9.10.1 asttokens-3.0.1 executing-2.2.1
ipython-pygments-lexers-1.1.1 jedi-0.19.2 pure-eval-0.2.3 stack_data-0.6.3
traitlets-5.14.3
```

1 Fully-Connected Neural Nets

In this exercise we will implement fully-connected networks using a modular approach. For each layer we will implement a **forward** and a **backward** function. The **forward** function will receive inputs, weights, and other parameters and will return both an output and a **cache** object storing data needed for the backward pass, like this:

```
def layer_forward(x, w):
    """ Receive inputs x and weights w """
    # Do some computations ...
    z = # ... some intermediate value
    # Do some more computations ...
    out = # the output

    cache = (x, w, z, out) # Values we need to compute gradients

    return out, cache
```

The backward pass will receive upstream derivatives and the **cache** object, and will return gradients with respect to the inputs and weights, like this:

```
def layer_backward(dout, cache):
    """
    Receive dout (derivative of loss with respect to outputs) and cache,
    and compute derivative with respect to inputs.
    """
    # Unpack cache values
    x, w, z, out = cache

    # Use values in cache to compute derivatives
    dx = # Derivative of loss with respect to x
    dw = # Derivative of loss with respect to w

    return dx, dw
```

After implementing a bunch of layers this way, we will be able to easily combine them to build classifiers with different architectures.

```
[2]: # As usual, a bit of setup
from __future__ import print_function
import time
import numpy as np
import matplotlib.pyplot as plt
from cs231n.classifiers.fc_net import *
from cs231n.data_utils import get_CIFAR10_data
from cs231n.gradient_check import eval_numerical_gradient, \
    eval_numerical_gradient_array
from cs231n.solver import Solver

%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# for auto-reloading external modules
# see http://stackoverflow.com/questions/1907993/
# autoreload-of-modules-in-ipython
%load_ext autoreload
%autoreload 2

def rel_error(x, y):
    """ returns relative error """
    return np.max(np.abs(x - y) / (np.maximum(1e-8, np.abs(x) + np.abs(y))))
```

```
[3]: # Load the (preprocessed) CIFAR10 data.
```

```
data = get_CIFAR10_data()
for k, v in list(data.items()):
    print('%s: ' % k, v.shape)
```

```
('X_train: ', (49000, 3, 32, 32))
('y_train: ', (49000,))
('X_val: ', (1000, 3, 32, 32))
('y_val: ', (1000,))
('X_test: ', (1000, 3, 32, 32))
('y_test: ', (1000,))
```

2 Affine layer: forward

Open the file `cs231n/layers.py` and implement the `affine_forward` function.

Once you are done you can test your implementaion by running the following:

```
[4]: # Test the affine_forward function
```

```

num_inputs = 2
input_shape = (4, 5, 6)
output_dim = 3

input_size = num_inputs * np.prod(input_shape)
weight_size = output_dim * np.prod(input_shape)

x = np.linspace(-0.1, 0.5, num=input_size).reshape(num_inputs, *input_shape)
w = np.linspace(-0.2, 0.3, num=weight_size).reshape(np.prod(input_shape),
    ↪output_dim)
b = np.linspace(-0.3, 0.1, num=output_dim)

out, _ = affine_forward(x, w, b)
correct_out = np.array([[ 1.49834967,  1.70660132,  1.91485297],
    [ 3.25553199,  3.5141327,  3.77273342]])

# Compare your output with ours. The error should be around e-9 or less.
print('Testing affine_forward function:')
print('difference: ', rel_error(out, correct_out))

```

Testing affine_forward function:
 difference: 9.769849468192957e-10

3 Affine layer: backward

Now implement the `affine_backward` function and test your implementation using numeric gradient checking.

```

[5]: # Test the affine_backward function
np.random.seed(231)
x = np.random.randn(10, 2, 3)
w = np.random.randn(6, 5)
b = np.random.randn(5)
dout = np.random.randn(10, 5)

dx_num = eval_numerical_gradient_array(lambda x: affine_forward(x, w, b)[0], x,
    ↪dout)
dw_num = eval_numerical_gradient_array(lambda w: affine_forward(x, w, b)[0], w,
    ↪dout)
db_num = eval_numerical_gradient_array(lambda b: affine_forward(x, w, b)[0], b,
    ↪dout)

# print(dx_num, dw_num, db_num)
_, cache = affine_forward(x, w, b)
dx, dw, db = affine_backward(dout, cache)

```

```

# The error should be around e-10 or less
print('Testing affine_backward function:')
print('dx error: ', rel_error(dx_num, dx))
print('dw error: ', rel_error(dw_num, dw))
print('db error: ', rel_error(db_num, db))

```

```

Testing affine_backward function:
dx error:  5.399100368651805e-11
dw error:  9.904211865398145e-11
db error:  2.4122867568119087e-11

```

4 ReLU activation: forward

Implement the forward pass for the ReLU activation function in the `relu_forward` function and test your implementation using the following:

```

[6]: # Test the relu_forward function

x = np.linspace(-0.5, 0.5, num=12).reshape(3, 4)

out, _ = relu_forward(x)
correct_out = np.array([[ 0.,          0.,          0.,          0.,          ],
                        [ 0.,          0.,          0.04545455,  0.13636364, ],
                        [ 0.22727273,  0.31818182,  0.40909091,  0.5,          ]])

# Compare your output with ours. The error should be on the order of e-8
print('Testing relu_forward function:')
print('difference: ', rel_error(out, correct_out))

```

```

Testing relu_forward function:
difference:  4.999999798022158e-08

```

5 ReLU activation: backward

Now implement the backward pass for the ReLU activation function in the `relu_backward` function and test your implementation using numeric gradient checking:

```

[7]: np.random.seed(231)
x = np.random.randn(10, 10)
dout = np.random.randn(*x.shape)

dx_num = eval_numerical_gradient_array(lambda x: relu_forward(x)[0], x, dout)

_, cache = relu_forward(x)
dx = relu_backward(dout, cache)
# print(dout)

```

```
# The error should be on the order of e-12
print('Testing relu_backward function:')
# print(dx_num)
print('dx error: ', rel_error(dx_num, dx))
```

Testing relu_backward function:
dx error: 3.2756349136310288e-12

5.1 Inline Question 1:

We've only asked you to implement ReLU, but there are a number of different activation functions that one could use in neural networks, each with its pros and cons. In particular, an issue commonly seen with activation functions is getting zero (or close to zero) gradient flow during backpropagation. Which of the following activation functions have this problem? If you consider these functions in the one dimensional case, what types of input would lead to this behaviour? 1. Sigmoid 2. ReLU 3. Leaky ReLU

Your Answer : Fill this in

Hàm Sigmoid, ReLU sẽ bị vấn đề này.

Ta có $\text{sigmoid}'(x) = \text{sigmoid}(x) * (1 - \text{sigmoid}(x))$, nên khi $x \rightarrow +\infty$ thì $\text{sigmoid}(x) \rightarrow 1$ thì $\text{sigmoid}'(x) \rightarrow 0$, còn khi $x \rightarrow -\infty$ thì $\text{sigmoid}(x) \rightarrow 0$ nên $\text{sigmoid}'(x) \rightarrow 0$.

Với $\text{ReLU}(x) = \max(x, 0)$, khi $x \leq 0$ thì $\text{ReLU}'(x) = 0$

Còn Leaky ReLU cải tiến ReLU bằng việc $\text{LeakyReLU}'(x) = \alpha$ với $x < 0$.

6 “Sandwich” layers

There are some common patterns of layers that are frequently used in neural nets. For example, affine layers are frequently followed by a ReLU nonlinearity. To make these common patterns easy, we define several convenience layers in the file `cs231n/layer_utils.py`.

For now take a look at the `affine_relu_forward` and `affine_relu_backward` functions, and run the following to numerically gradient check the backward pass:

```
[8]: from cs231n.layer_utils import affine_relu_forward, affine_relu_backward
np.random.seed(231)
x = np.random.randn(2, 3, 4)
w = np.random.randn(12, 10)
b = np.random.randn(10)
dout = np.random.randn(2, 10)

out, cache = affine_relu_forward(x, w, b)
dx, dw, db = affine_relu_backward(dout, cache)

dx_num = eval_numerical_gradient_array(lambda x: affine_relu_forward(x, w, b), x, dout)
```



```

dw_num = eval_numerical_gradient_array(lambda w: affine_relu_forward(x, w,
    ↪b)[0], w, dout)
db_num = eval_numerical_gradient_array(lambda b: affine_relu_forward(x, w,
    ↪b)[0], b, dout)

# Relative error should be around e-10 or less
print('Testing affine_relu_forward and affine_relu_backward:')
print('dx error: ', rel_error(dx_num, dx))
print('dw error: ', rel_error(dw_num, dw))
print('db error: ', rel_error(db_num, db))

```

```

Testing affine_relu_forward and affine_relu_backward:
dx error:  2.299579177309368e-11
dw error:  8.162011105764925e-11
db error:  7.826724021458994e-12

```

7 Loss layers: Softmax

Now implement the loss and gradient for softmax in the `softmax_loss` function in `cs231n/layers.py`. These should be similar to what you implemented in `cs231n/classifiers/softmax.py`. Other loss functions (e.g. `svm_loss`) can also be implemented in a modular way, however, it is not required for this assignment.

You can make sure that the implementations are correct by running the following:

```

[9]: np.random.seed(231)
num_classes, num_inputs = 10, 50
x = 0.001 * np.random.randn(num_inputs, num_classes)
y = np.random.randint(num_classes, size=num_inputs)

# softmax_loss(x,y)
dx_num = eval_numerical_gradient(lambda x: softmax_loss(x, y)[0], x,
    ↪verbose=False)
loss, dx = softmax_loss(x, y)

# Test softmax_loss function. Loss should be close to 2.3 and dx error should
    ↪be around e-8
print('\nTesting softmax_loss:')
print('loss: ', loss)
# print(dx.shape)
# print(dx_num.shape)
print('dx error: ', rel_error(dx_num, dx))

# if rel_error(dx_num, dx) > 1e-8:
#     print("dung")

```

```

Testing softmax_loss:

```

```
loss: 2.302545844500738
dx error: 7.454884968628367e-09
```

8 Two-layer network

Open the file `cs231n/classifiers/fc_net.py` and complete the implementation of the `TwoLayerNet` class. Read through it to make sure you understand the API. You can run the cell below to test your implementation.

```
[10]: np.random.seed(231)
N, D, H, C = 3, 5, 50, 7
X = np.random.randn(N, D)
y = np.random.randint(C, size=N)

std = 1e-3
model = TwoLayerNet(input_dim=D, hidden_dim=H, num_classes=C, weight_scale=std)

print('Testing initialization ... ')
W1_std = abs(model.params['W1'].std() - std)
b1 = model.params['b1']
W2_std = abs(model.params['W2'].std() - std)
b2 = model.params['b2']
assert W1_std < std / 10, 'First layer weights do not seem right'
assert np.all(b1 == 0), 'First layer biases do not seem right'
assert W2_std < std / 10, 'Second layer weights do not seem right'
assert np.all(b2 == 0), 'Second layer biases do not seem right'

# model.reg= 0.5

print('Testing test-time forward pass ... ')
model.params['W1'] = np.linspace(-0.7, 0.3, num=D*H).reshape(D, H)
model.params['b1'] = np.linspace(-0.1, 0.9, num=H)
model.params['W2'] = np.linspace(-0.3, 0.4, num=H*C).reshape(H, C)
model.params['b2'] = np.linspace(-0.9, 0.1, num=C)
X = np.linspace(-5.5, 4.5, num=N*D).reshape(D, N).T
scores = model.loss(X)
correct_scores = np.asarray(
    [[11.53165108, 12.2917344, 13.05181771, 13.81190102, 14.57198434, 15.
     ↪33206765, 16.09215096],
     [12.05769098, 12.74614105, 13.43459113, 14.1230412, 14.81149128, 15.
     ↪49994135, 16.18839143],
     [12.58373087, 13.20054771, 13.81736455, 14.43418138, 15.05099822, 15.
     ↪66781506, 16.2846319 ]])
scores_diff = np.abs(scores - correct_scores).sum()
assert scores_diff < 1e-6, 'Problem with test-time forward pass'

print('Testing training loss (no regularization)')
```

```

y = np.asarray([0, 5, 1])
loss, grads = model.loss(X, y)
# print("loss k co regularization la : ")
# print(loss)
correct_loss = 3.4702243556
assert abs(loss - correct_loss) < 1e-10, 'Problem with training-time loss'

print('Pass testing training loss (no regularization)')

model.reg = 1.0
loss, grads = model.loss(X, y)

# print('==== Loss goc la')

# print(loss)
correct_loss = 26.5948426952
assert abs(loss - correct_loss) < 1e-10, 'Problem with regularization loss'

# Errors should be around e-7 or less
for reg in [0.0, 0.7]:
    print('Running numeric gradient check with reg = ', reg)
    model.reg = reg
    loss, grads = model.loss(X, y)

    for name in sorted(grads):
        f = lambda _: model.loss(X, y)[0]
        grad_num = eval_numerical_gradient(f, model.params[name], verbose=False)
        print('%s relative error: %.2e' % (name, rel_error(grad_num, grads[name])))

```

```

Testing initialization ...
Testing test-time forward pass ...
Testing training loss (no regularization)
Pass testing training loss (no regularization)
Running numeric gradient check with reg = 0.0
W1 relative error: 1.53e-08
W2 relative error: 3.28e-10
b1 relative error: 8.01e-09
b2 relative error: 4.33e-10
Running numeric gradient check with reg = 0.7
W1 relative error: 3.12e-07
W2 relative error: 7.98e-08
b1 relative error: 1.09e-09
b2 relative error: 7.76e-10

```

9 Solver

Open the file `cs231n/solver.py` and read through it to familiarize yourself with the API. After doing so, use a `Solver` instance to train a `TwoLayerNet` that achieves about 36% accuracy on the validation set.

```
[11]: input_size = 32 * 32 * 3
      hidden_size = 50
      num_classes = 10
      model = TwoLayerNet(input_size, hidden_size, num_classes)
      solver = None

      #####
      # TODO: Use a Solver instance to train a TwoLayerNet that achieves about 36% #
      # accuracy on the validation set.                                           #
      #####
      solver = Solver(model, data,
                      update_rule='sgd',
                      optim_config={
                          'learning_rate': 1e-4,
                      },
                      lr_decay=0.95,
                      num_epochs=5, batch_size=200,
                      print_every=100)

      solver.train()

      #####
      #                                     END OF YOUR CODE                       #
      #####
```

```
(Iteration 1 / 1225) loss: 2.301725
(Epoch 0 / 5) train acc: 0.145000; val_acc: 0.140000
(Iteration 101 / 1225) loss: 2.241923
(Iteration 201 / 1225) loss: 2.187425
(Epoch 1 / 5) train acc: 0.267000; val_acc: 0.243000
(Iteration 301 / 1225) loss: 2.056790
(Iteration 401 / 1225) loss: 1.937978
(Epoch 2 / 5) train acc: 0.294000; val_acc: 0.303000
(Iteration 501 / 1225) loss: 1.924555
(Iteration 601 / 1225) loss: 1.933743
(Iteration 701 / 1225) loss: 1.832777
(Epoch 3 / 5) train acc: 0.336000; val_acc: 0.315000
(Iteration 801 / 1225) loss: 1.960827
(Iteration 901 / 1225) loss: 1.832752
(Epoch 4 / 5) train acc: 0.340000; val_acc: 0.350000
(Iteration 1001 / 1225) loss: 1.739182
(Iteration 1101 / 1225) loss: 1.940517
(Iteration 1201 / 1225) loss: 1.848443
(Epoch 5 / 5) train acc: 0.355000; val_acc: 0.373000
```

10 Debug the training

With the default parameters we provided above, you should get a validation accuracy of about 0.36 on the validation set. This isn't very good.

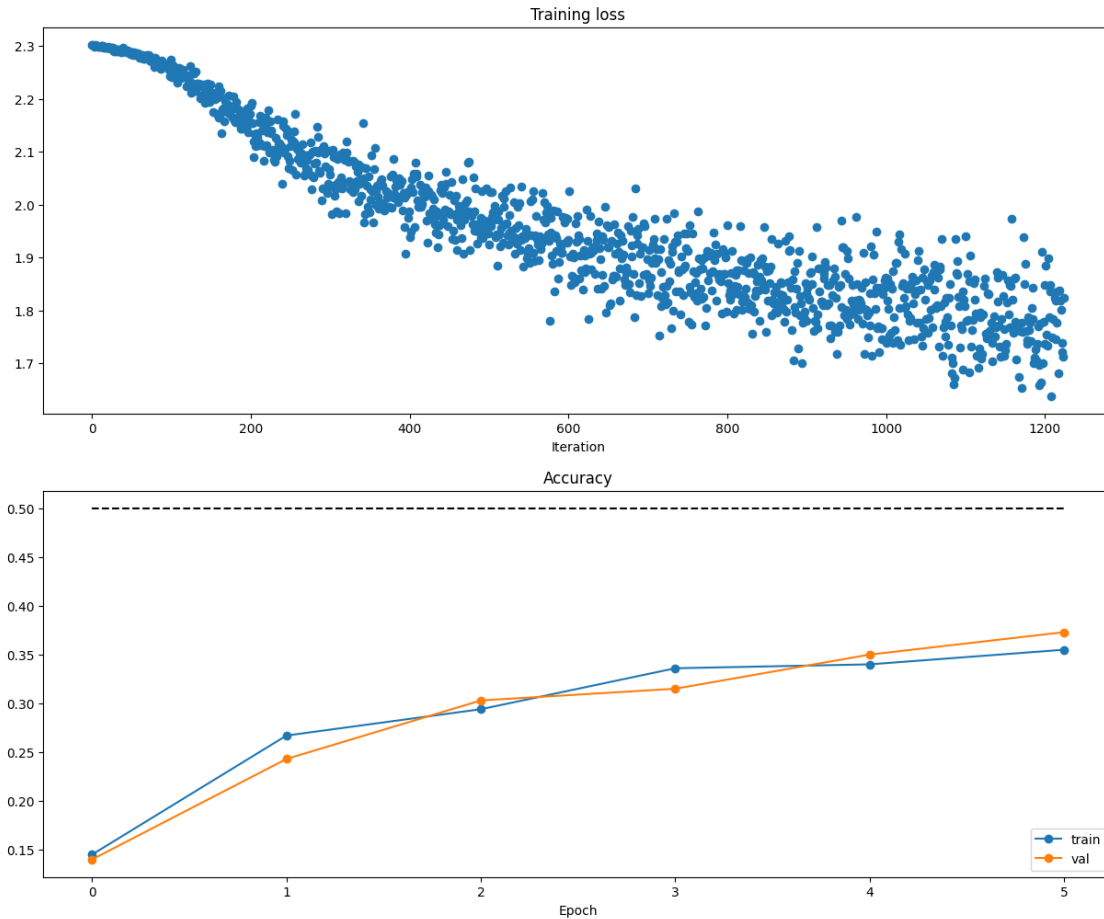
One strategy for getting insight into what's wrong is to plot the loss function and the accuracies on the training and validation sets during optimization.

Another strategy is to visualize the weights that were learned in the first layer of the network. In most neural networks trained on visual data, the first layer weights typically show some visible structure when visualized.

[12]: *# Run this cell to visualize training loss and train / val accuracy*

```
plt.subplot(2, 1, 1)
plt.title('Training loss')
plt.plot(solver.loss_history, 'o')
plt.xlabel('Iteration')

plt.subplot(2, 1, 2)
plt.title('Accuracy')
plt.plot(solver.train_acc_history, '-o', label='train')
plt.plot(solver.val_acc_history, '-o', label='val')
plt.plot([0.5] * len(solver.val_acc_history), 'k--')
plt.xlabel('Epoch')
plt.legend(loc='lower right')
plt.gcf().set_size_inches(15, 12)
plt.show()
```

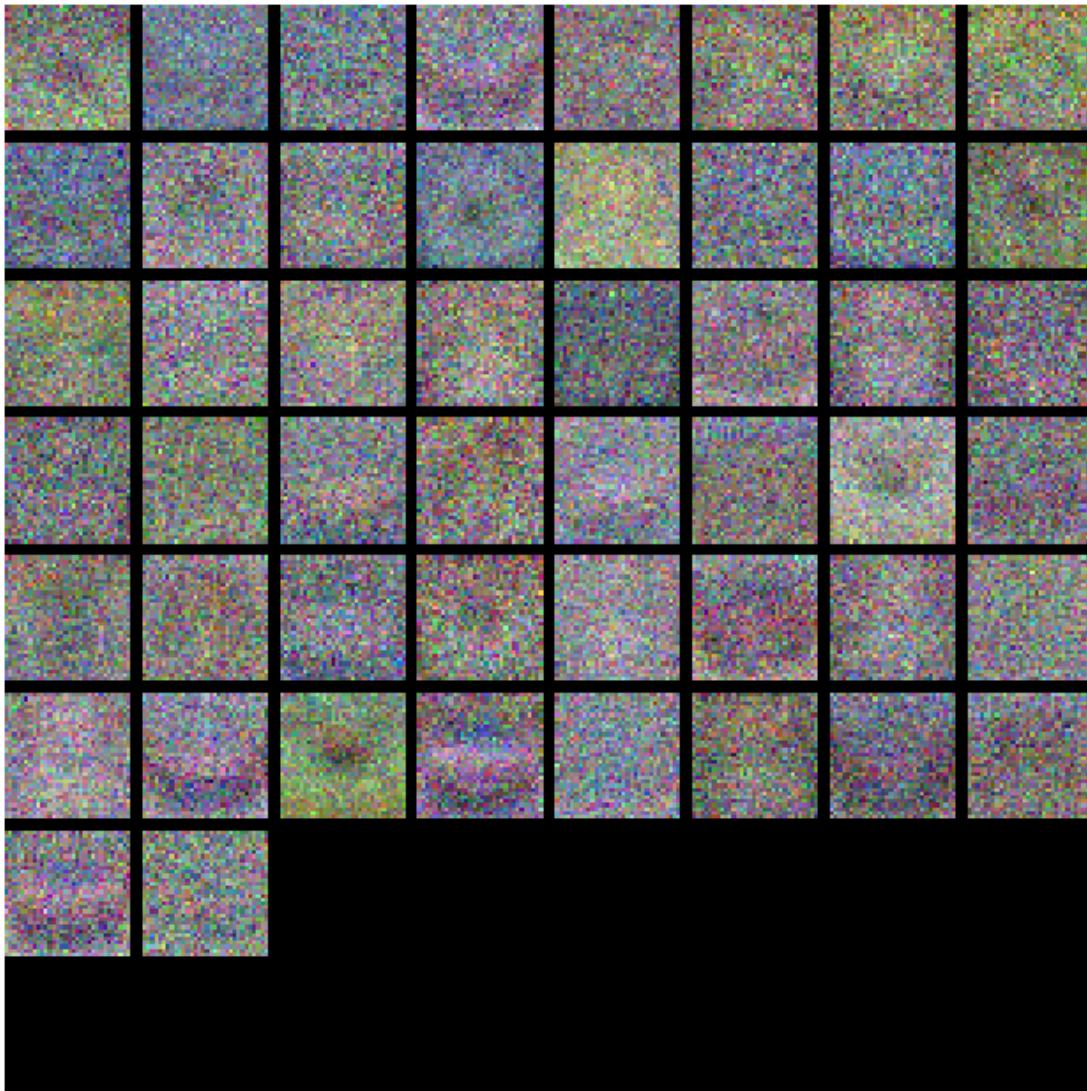


```
[13]: from cs231n.vis_utils import visualize_grid

# Visualize the weights of the network

def show_net_weights(net):
    W1 = net.params['W1']
    W1 = W1.reshape(3, 32, 32, -1).transpose(3, 1, 2, 0)
    plt.imshow(visualize_grid(W1, padding=3).astype('uint8'))
    plt.gca().axis('off')
    plt.show()

show_net_weights(model)
```



11 Tune your hyperparameters

What's wrong?. Looking at the visualizations above, we see that the loss is decreasing more or less linearly, which seems to suggest that the learning rate may be too low. Moreover, there is no gap between the training and validation accuracy, suggesting that the model we used has low capacity, and that we should increase its size. On the other hand, with a very large model we would expect to see more overfitting, which would manifest itself as a very large gap between the training and validation accuracy.

Tuning. Tuning the hyperparameters and developing intuition for how they affect the final performance is a large part of using Neural Networks, so we want you to get a lot of practice. Below, you should experiment with different values of the various hyperparameters, including hidden layer size, learning rate, number of training epochs, and regularization strength. You might also consider

tuning the learning rate decay, but you should be able to get good performance using the default value.

Approximate results. You should be aim to achieve a classification accuracy of greater than 48% on the validation set. Our best network gets over 52% on the validation set.

Experiment: Your goal in this exercise is to get as good of a result on CIFAR-10 as you can (52% could serve as a reference), with a fully-connected Neural Network. Feel free implement your own techniques (e.g. PCA to reduce dimensionality, or adding dropout, or adding features to the solver, etc.).

```
[21]: best_model = None

#####
# TODO: Tune hyperparameters using the validation set. Store your best trained
#
# model in best_model.
#
#
# To help debug your network, it may help to use visualizations similar to the
#
# ones we used above; these visualizations will have significant qualitative
#
# differences from the ones we saw above for the poorly tuned network.
#
#
# Tweaking hyperparameters by hand can be fun, but you might find it useful to
#
# write code to sweep through possible combinations of hyperparameters
#
# automatically like we did on the previous exercises.
#
#####
learning_rates = [1e-4, 5e-4, 1e-3]
learning_rates_decay = [0.95, 0.98]
hidden_sizes = [100]
regularization_strengths = [0.0, 10]
# learning_rates_decay = [0.95]
# batch_size = [200]
num_epochs = [5]
# for learning_rate in learning_rates:
#     for learning_rate_decay in learning_rates_decay:
#         for hidden_size in hidden_sizes:
```



```

#         for regularization_strength in regularization_strengths:
#             for batch in batch_size:
#                 for epoch in num_epochs:
#                     model = TwoLayerNet(input_size, hidden_size, num_classes)
#                     solver = Solver(model, data,
#                                     update_rule='sgd',
#                                     optim_config={
#                                         'learning_rate': learning_rate,
#                                     },
#                                     lr_decay=learning_rate_decay,
#                                     num_epochs=epoch, batch_size=batch,
#                                     print_every=100)
#                     solver.train()
#                     print(solver.val_acc_history[-1])
#                     if best_model is None or solver.val_acc_history[-1] > best_model.
↪ val_acc_history[-1]:
#                         best_model = solver
best_val = 0.0
best_solver = None
for lr in learning_rates:
    for lr_decay in learning_rates_decay:
        for hidden_size in hidden_sizes :
            for reg in regularization_strengths :
                for num_epoch in num_epochs :
                    model = TwoLayerNet(input_size, hidden_size, num_classes,
↪ ,reg=reg)

                    solver = Solver(model, data,
                                    update_rule='sgd',
                                    optim_config={
                                        'learning_rate': lr,
                                    },
                                    lr_decay=lr_decay,
                                    num_epochs=num_epoch, batch_size=100,
                                    print_every=100)

                    solver.train()
                    if best_solver is None :
                        best_solver = solver
                    print(solver.val_acc_history[-1])
                    if best_model is None or solver.val_acc_history[-1] >
↪ best_solver.val_acc_history[-1]:
                        best_model = model
                        best_solver = solver
                        best_val = solver.val_acc_history[-1]

#####
#                                     END OF YOUR CODE                                     #
#####

```

```

(Iteration 1 / 2450) loss: 2.301749
(Epoch 0 / 5) train acc: 0.098000; val_acc: 0.118000
(Iteration 101 / 2450) loss: 2.243806
(Iteration 201 / 2450) loss: 2.049760
(Iteration 301 / 2450) loss: 1.974670
(Iteration 401 / 2450) loss: 2.025692
(Epoch 1 / 5) train acc: 0.285000; val_acc: 0.319000
(Iteration 501 / 2450) loss: 1.928706
(Iteration 601 / 2450) loss: 1.943749
(Iteration 701 / 2450) loss: 1.896251
(Iteration 801 / 2450) loss: 1.996052
(Iteration 901 / 2450) loss: 1.756462
(Epoch 2 / 5) train acc: 0.377000; val_acc: 0.372000
(Iteration 1001 / 2450) loss: 1.853278
(Iteration 1101 / 2450) loss: 1.790235
(Iteration 1201 / 2450) loss: 1.798872
(Iteration 1301 / 2450) loss: 1.754139
(Iteration 1401 / 2450) loss: 1.782141
(Epoch 3 / 5) train acc: 0.423000; val_acc: 0.393000
(Iteration 1501 / 2450) loss: 1.573168
(Iteration 1601 / 2450) loss: 1.731085
(Iteration 1701 / 2450) loss: 1.616972
(Iteration 1801 / 2450) loss: 1.783365
(Iteration 1901 / 2450) loss: 1.515438
(Epoch 4 / 5) train acc: 0.437000; val_acc: 0.423000
(Iteration 2001 / 2450) loss: 1.734900
(Iteration 2101 / 2450) loss: 1.562339
(Iteration 2201 / 2450) loss: 1.549542
(Iteration 2301 / 2450) loss: 1.694715
(Iteration 2401 / 2450) loss: 1.652630
(Epoch 5 / 5) train acc: 0.448000; val_acc: 0.450000
0.45
(Iteration 1 / 2450) loss: 3.845529
(Epoch 0 / 5) train acc: 0.110000; val_acc: 0.101000
(Iteration 101 / 2450) loss: 3.518604
(Iteration 201 / 2450) loss: 3.202241
(Iteration 301 / 2450) loss: 2.889131
(Iteration 401 / 2450) loss: 2.791108
(Epoch 1 / 5) train acc: 0.291000; val_acc: 0.293000
(Iteration 501 / 2450) loss: 2.645189
(Iteration 601 / 2450) loss: 2.473279
(Iteration 701 / 2450) loss: 2.366527
(Iteration 801 / 2450) loss: 2.416521
(Iteration 901 / 2450) loss: 2.178868
(Epoch 2 / 5) train acc: 0.321000; val_acc: 0.339000
(Iteration 1001 / 2450) loss: 2.206790
(Iteration 1101 / 2450) loss: 2.038215
(Iteration 1201 / 2450) loss: 2.135069

```

(Iteration 1301 / 2450) loss: 2.049772
(Iteration 1401 / 2450) loss: 2.162427
(Epoch 3 / 5) train acc: 0.369000; val_acc: 0.361000
(Iteration 1501 / 2450) loss: 1.916830
(Iteration 1601 / 2450) loss: 2.094552
(Iteration 1701 / 2450) loss: 2.024583
(Iteration 1801 / 2450) loss: 2.067527
(Iteration 1901 / 2450) loss: 2.032168
(Epoch 4 / 5) train acc: 0.326000; val_acc: 0.374000
(Iteration 2001 / 2450) loss: 1.931569
(Iteration 2101 / 2450) loss: 2.029985
(Iteration 2201 / 2450) loss: 2.027078
(Iteration 2301 / 2450) loss: 1.981885
(Iteration 2401 / 2450) loss: 2.012006
(Epoch 5 / 5) train acc: 0.367000; val_acc: 0.378000
0.378
(Iteration 1 / 2450) loss: 2.306307
(Epoch 0 / 5) train acc: 0.096000; val_acc: 0.119000
(Iteration 101 / 2450) loss: 2.239455
(Iteration 201 / 2450) loss: 2.093494
(Iteration 301 / 2450) loss: 1.980060
(Iteration 401 / 2450) loss: 1.925475
(Epoch 1 / 5) train acc: 0.307000; val_acc: 0.313000
(Iteration 501 / 2450) loss: 1.929285
(Iteration 601 / 2450) loss: 1.883582
(Iteration 701 / 2450) loss: 1.855610
(Iteration 801 / 2450) loss: 1.874377
(Iteration 901 / 2450) loss: 1.786115
(Epoch 2 / 5) train acc: 0.358000; val_acc: 0.378000
(Iteration 1001 / 2450) loss: 1.776880
(Iteration 1101 / 2450) loss: 1.873830
(Iteration 1201 / 2450) loss: 1.738423
(Iteration 1301 / 2450) loss: 1.615846
(Iteration 1401 / 2450) loss: 1.782798
(Epoch 3 / 5) train acc: 0.379000; val_acc: 0.408000
(Iteration 1501 / 2450) loss: 1.757016
(Iteration 1601 / 2450) loss: 1.693303
(Iteration 1701 / 2450) loss: 1.635641
(Iteration 1801 / 2450) loss: 1.629810
(Iteration 1901 / 2450) loss: 1.618996
(Epoch 4 / 5) train acc: 0.406000; val_acc: 0.424000
(Iteration 2001 / 2450) loss: 1.706885
(Iteration 2101 / 2450) loss: 1.656841
(Iteration 2201 / 2450) loss: 1.536829
(Iteration 2301 / 2450) loss: 1.668395
(Iteration 2401 / 2450) loss: 1.594174
(Epoch 5 / 5) train acc: 0.455000; val_acc: 0.449000
0.449

(Iteration 1 / 2450) loss: 3.845093
(Epoch 0 / 5) train acc: 0.085000; val_acc: 0.108000
(Iteration 101 / 2450) loss: 3.505329
(Iteration 201 / 2450) loss: 3.229461
(Iteration 301 / 2450) loss: 2.884033
(Iteration 401 / 2450) loss: 2.737407
(Epoch 1 / 5) train acc: 0.287000; val_acc: 0.291000
(Iteration 501 / 2450) loss: 2.654550
(Iteration 601 / 2450) loss: 2.532789
(Iteration 701 / 2450) loss: 2.404880
(Iteration 801 / 2450) loss: 2.360390
(Iteration 901 / 2450) loss: 2.167015
(Epoch 2 / 5) train acc: 0.345000; val_acc: 0.335000
(Iteration 1001 / 2450) loss: 2.188855
(Iteration 1101 / 2450) loss: 1.966607
(Iteration 1201 / 2450) loss: 2.124569
(Iteration 1301 / 2450) loss: 2.081837
(Iteration 1401 / 2450) loss: 2.181445
(Epoch 3 / 5) train acc: 0.352000; val_acc: 0.358000
(Iteration 1501 / 2450) loss: 1.903247
(Iteration 1601 / 2450) loss: 2.039532
(Iteration 1701 / 2450) loss: 1.939618
(Iteration 1801 / 2450) loss: 2.029489
(Iteration 1901 / 2450) loss: 1.981105
(Epoch 4 / 5) train acc: 0.379000; val_acc: 0.378000
(Iteration 2001 / 2450) loss: 2.022385
(Iteration 2101 / 2450) loss: 1.980091
(Iteration 2201 / 2450) loss: 2.055336
(Iteration 2301 / 2450) loss: 1.907533
(Iteration 2401 / 2450) loss: 1.942509
(Epoch 5 / 5) train acc: 0.357000; val_acc: 0.385000
0.385
(Iteration 1 / 2450) loss: 2.304326
(Epoch 0 / 5) train acc: 0.098000; val_acc: 0.088000
(Iteration 101 / 2450) loss: 1.876076
(Iteration 201 / 2450) loss: 1.818885
(Iteration 301 / 2450) loss: 1.846473
(Iteration 401 / 2450) loss: 1.565334
(Epoch 1 / 5) train acc: 0.447000; val_acc: 0.429000
(Iteration 501 / 2450) loss: 1.439154
(Iteration 601 / 2450) loss: 1.620333
(Iteration 701 / 2450) loss: 1.467683
(Iteration 801 / 2450) loss: 1.533239
(Iteration 901 / 2450) loss: 1.256714
(Epoch 2 / 5) train acc: 0.458000; val_acc: 0.469000
(Iteration 1001 / 2450) loss: 1.469790
(Iteration 1101 / 2450) loss: 1.354988
(Iteration 1201 / 2450) loss: 1.278984

```

(Iteration 1301 / 2450) loss: 1.310528
(Iteration 1401 / 2450) loss: 1.480736
(Epoch 3 / 5) train acc: 0.533000; val_acc: 0.476000
(Iteration 1501 / 2450) loss: 1.408796
(Iteration 1601 / 2450) loss: 1.600700
(Iteration 1701 / 2450) loss: 1.379270
(Iteration 1801 / 2450) loss: 1.171149
(Iteration 1901 / 2450) loss: 1.401426
(Epoch 4 / 5) train acc: 0.534000; val_acc: 0.492000
(Iteration 2001 / 2450) loss: 1.329156
(Iteration 2101 / 2450) loss: 1.368661
(Iteration 2201 / 2450) loss: 1.399260
(Iteration 2301 / 2450) loss: 1.322160
(Iteration 2401 / 2450) loss: 1.261487
(Epoch 5 / 5) train acc: 0.526000; val_acc: 0.495000
0.495
(Iteration 1 / 2450) loss: 3.847991
(Epoch 0 / 5) train acc: 0.090000; val_acc: 0.112000
(Iteration 101 / 2450) loss: 2.521330
(Iteration 201 / 2450) loss: 2.125404
(Iteration 301 / 2450) loss: 2.120380
(Iteration 401 / 2450) loss: 1.999170
(Epoch 1 / 5) train acc: 0.386000; val_acc: 0.376000
(Iteration 501 / 2450) loss: 1.979077
(Iteration 601 / 2450) loss: 2.018328
(Iteration 701 / 2450) loss: 1.868861
(Iteration 801 / 2450) loss: 1.848722
(Iteration 901 / 2450) loss: 1.930097
(Epoch 2 / 5) train acc: 0.382000; val_acc: 0.390000
(Iteration 1001 / 2450) loss: 1.953699
(Iteration 1101 / 2450) loss: 2.010049
(Iteration 1201 / 2450) loss: 2.009463
(Iteration 1301 / 2450) loss: 2.048782
(Iteration 1401 / 2450) loss: 1.830786
(Epoch 3 / 5) train acc: 0.401000; val_acc: 0.402000
(Iteration 1501 / 2450) loss: 2.006929
(Iteration 1601 / 2450) loss: 1.856926
(Iteration 1701 / 2450) loss: 2.012700
(Iteration 1801 / 2450) loss: 1.987291
(Iteration 1901 / 2450) loss: 1.883578
(Epoch 4 / 5) train acc: 0.391000; val_acc: 0.410000
(Iteration 2001 / 2450) loss: 1.938213
(Iteration 2101 / 2450) loss: 1.849377
(Iteration 2201 / 2450) loss: 1.951238
(Iteration 2301 / 2450) loss: 1.977702
(Iteration 2401 / 2450) loss: 1.982016
(Epoch 5 / 5) train acc: 0.382000; val_acc: 0.393000
0.393

```

```

(Iteration 1 / 2450) loss: 2.303348
(Epoch 0 / 5) train acc: 0.143000; val_acc: 0.133000
(Iteration 101 / 2450) loss: 1.988603
(Iteration 201 / 2450) loss: 1.873307
(Iteration 301 / 2450) loss: 1.709605
(Iteration 401 / 2450) loss: 1.623701
(Epoch 1 / 5) train acc: 0.427000; val_acc: 0.447000
(Iteration 501 / 2450) loss: 1.635711
(Iteration 601 / 2450) loss: 1.700721
(Iteration 701 / 2450) loss: 1.677750
(Iteration 801 / 2450) loss: 1.651241
(Iteration 901 / 2450) loss: 1.549472
(Epoch 2 / 5) train acc: 0.470000; val_acc: 0.436000
(Iteration 1001 / 2450) loss: 1.489045
(Iteration 1101 / 2450) loss: 1.449347
(Iteration 1201 / 2450) loss: 1.267118
(Iteration 1301 / 2450) loss: 1.449496
(Iteration 1401 / 2450) loss: 1.333810
(Epoch 3 / 5) train acc: 0.490000; val_acc: 0.474000
(Iteration 1501 / 2450) loss: 1.566528
(Iteration 1601 / 2450) loss: 1.445114
(Iteration 1701 / 2450) loss: 1.312343
(Iteration 1801 / 2450) loss: 1.164462
(Iteration 1901 / 2450) loss: 1.203384
(Epoch 4 / 5) train acc: 0.536000; val_acc: 0.504000
(Iteration 2001 / 2450) loss: 1.414759
(Iteration 2101 / 2450) loss: 1.155585
(Iteration 2201 / 2450) loss: 1.443606
(Iteration 2301 / 2450) loss: 1.466080
(Iteration 2401 / 2450) loss: 1.193138
(Epoch 5 / 5) train acc: 0.541000; val_acc: 0.508000
0.508
(Iteration 1 / 2450) loss: 3.847009
(Epoch 0 / 5) train acc: 0.099000; val_acc: 0.108000
(Iteration 101 / 2450) loss: 2.695851
(Iteration 201 / 2450) loss: 2.265951
(Iteration 301 / 2450) loss: 2.065789
(Iteration 401 / 2450) loss: 1.989584
(Epoch 1 / 5) train acc: 0.381000; val_acc: 0.376000
(Iteration 501 / 2450) loss: 1.955069
(Iteration 601 / 2450) loss: 1.830184
(Iteration 701 / 2450) loss: 2.048452
(Iteration 801 / 2450) loss: 1.941986
(Iteration 901 / 2450) loss: 1.981639
(Epoch 2 / 5) train acc: 0.404000; val_acc: 0.397000
(Iteration 1001 / 2450) loss: 1.842996
(Iteration 1101 / 2450) loss: 1.966579
(Iteration 1201 / 2450) loss: 1.918521

```

(Iteration 1301 / 2450) loss: 1.956072
(Iteration 1401 / 2450) loss: 2.056145
(Epoch 3 / 5) train acc: 0.396000; val_acc: 0.409000
(Iteration 1501 / 2450) loss: 2.008561
(Iteration 1601 / 2450) loss: 1.983103
(Iteration 1701 / 2450) loss: 1.883686
(Iteration 1801 / 2450) loss: 1.963123
(Iteration 1901 / 2450) loss: 1.843112
(Epoch 4 / 5) train acc: 0.403000; val_acc: 0.405000
(Iteration 2001 / 2450) loss: 1.939682
(Iteration 2101 / 2450) loss: 2.061217
(Iteration 2201 / 2450) loss: 1.970191
(Iteration 2301 / 2450) loss: 1.943288
(Iteration 2401 / 2450) loss: 1.950226
(Epoch 5 / 5) train acc: 0.385000; val_acc: 0.406000
0.406
(Iteration 1 / 2450) loss: 2.300191
(Epoch 0 / 5) train acc: 0.110000; val_acc: 0.102000
(Iteration 101 / 2450) loss: 1.909037
(Iteration 201 / 2450) loss: 1.671444
(Iteration 301 / 2450) loss: 1.834561
(Iteration 401 / 2450) loss: 1.384903
(Epoch 1 / 5) train acc: 0.450000; val_acc: 0.446000
(Iteration 501 / 2450) loss: 1.608294
(Iteration 601 / 2450) loss: 1.347842
(Iteration 701 / 2450) loss: 1.397401
(Iteration 801 / 2450) loss: 1.435975
(Iteration 901 / 2450) loss: 1.592174
(Epoch 2 / 5) train acc: 0.480000; val_acc: 0.464000
(Iteration 1001 / 2450) loss: 1.387358
(Iteration 1101 / 2450) loss: 1.507610
(Iteration 1201 / 2450) loss: 1.468075
(Iteration 1301 / 2450) loss: 1.315154
(Iteration 1401 / 2450) loss: 1.255913
(Epoch 3 / 5) train acc: 0.504000; val_acc: 0.498000
(Iteration 1501 / 2450) loss: 1.232489
(Iteration 1601 / 2450) loss: 1.182163
(Iteration 1701 / 2450) loss: 1.164944
(Iteration 1801 / 2450) loss: 1.244483
(Iteration 1901 / 2450) loss: 1.383576
(Epoch 4 / 5) train acc: 0.558000; val_acc: 0.498000
(Iteration 2001 / 2450) loss: 1.059229
(Iteration 2101 / 2450) loss: 1.267327
(Iteration 2201 / 2450) loss: 1.381446
(Iteration 2301 / 2450) loss: 1.487697
(Iteration 2401 / 2450) loss: 1.257390
(Epoch 5 / 5) train acc: 0.573000; val_acc: 0.488000
0.488

```

(Iteration 1 / 2450) loss: 3.847184
(Epoch 0 / 5) train acc: 0.147000; val_acc: 0.147000
(Iteration 101 / 2450) loss: 2.121443
(Iteration 201 / 2450) loss: 1.932640
(Iteration 301 / 2450) loss: 1.980204
(Iteration 401 / 2450) loss: 1.971472
(Epoch 1 / 5) train acc: 0.386000; val_acc: 0.367000
(Iteration 501 / 2450) loss: 2.074934
(Iteration 601 / 2450) loss: 1.923750
(Iteration 701 / 2450) loss: 1.994239
(Iteration 801 / 2450) loss: 1.996819
(Iteration 901 / 2450) loss: 1.980302
(Epoch 2 / 5) train acc: 0.395000; val_acc: 0.356000
(Iteration 1001 / 2450) loss: 1.984379
(Iteration 1101 / 2450) loss: 2.020597
(Iteration 1201 / 2450) loss: 2.004712
(Iteration 1301 / 2450) loss: 1.888158
(Iteration 1401 / 2450) loss: 1.922352
(Epoch 3 / 5) train acc: 0.378000; val_acc: 0.385000
(Iteration 1501 / 2450) loss: 2.000217
(Iteration 1601 / 2450) loss: 1.956929
(Iteration 1701 / 2450) loss: 1.839352
(Iteration 1801 / 2450) loss: 2.080973
(Iteration 1901 / 2450) loss: 2.202207
(Epoch 4 / 5) train acc: 0.405000; val_acc: 0.394000
(Iteration 2001 / 2450) loss: 1.995613
(Iteration 2101 / 2450) loss: 2.018219
(Iteration 2201 / 2450) loss: 1.892949
(Iteration 2301 / 2450) loss: 1.868557
(Iteration 2401 / 2450) loss: 1.976136
(Epoch 5 / 5) train acc: 0.381000; val_acc: 0.391000
0.391
(Iteration 1 / 2450) loss: 2.307005
(Epoch 0 / 5) train acc: 0.129000; val_acc: 0.133000
(Iteration 101 / 2450) loss: 1.964923
(Iteration 201 / 2450) loss: 1.610894
(Iteration 301 / 2450) loss: 1.808313
(Iteration 401 / 2450) loss: 1.553470
(Epoch 1 / 5) train acc: 0.433000; val_acc: 0.449000
(Iteration 501 / 2450) loss: 1.536548
(Iteration 601 / 2450) loss: 1.525503
(Iteration 701 / 2450) loss: 1.513647
(Iteration 801 / 2450) loss: 1.493670
(Iteration 901 / 2450) loss: 1.626963
(Epoch 2 / 5) train acc: 0.468000; val_acc: 0.452000
(Iteration 1001 / 2450) loss: 1.487103
(Iteration 1101 / 2450) loss: 1.368489
(Iteration 1201 / 2450) loss: 1.542403

```



```

(Iteration 1301 / 2450) loss: 1.516782
(Iteration 1401 / 2450) loss: 1.381809
(Epoch 3 / 5) train acc: 0.505000; val_acc: 0.479000
(Iteration 1501 / 2450) loss: 1.355015
(Iteration 1601 / 2450) loss: 1.422214
(Iteration 1701 / 2450) loss: 1.405394
(Iteration 1801 / 2450) loss: 1.265019
(Iteration 1901 / 2450) loss: 1.374771
(Epoch 4 / 5) train acc: 0.546000; val_acc: 0.485000
(Iteration 2001 / 2450) loss: 1.474281
(Iteration 2101 / 2450) loss: 1.455896
(Iteration 2201 / 2450) loss: 1.433912
(Iteration 2301 / 2450) loss: 1.239538
(Iteration 2401 / 2450) loss: 1.356995
(Epoch 5 / 5) train acc: 0.540000; val_acc: 0.505000
0.505
(Iteration 1 / 2450) loss: 3.841490
(Epoch 0 / 5) train acc: 0.137000; val_acc: 0.156000
(Iteration 101 / 2450) loss: 2.199176
(Iteration 201 / 2450) loss: 2.208428
(Iteration 301 / 2450) loss: 1.932301
(Iteration 401 / 2450) loss: 2.078875
(Epoch 1 / 5) train acc: 0.411000; val_acc: 0.381000
(Iteration 501 / 2450) loss: 2.027477
(Iteration 601 / 2450) loss: 1.981302
(Iteration 701 / 2450) loss: 1.929610
(Iteration 801 / 2450) loss: 2.133233
(Iteration 901 / 2450) loss: 1.968993
(Epoch 2 / 5) train acc: 0.355000; val_acc: 0.365000
(Iteration 1001 / 2450) loss: 2.043226
(Iteration 1101 / 2450) loss: 1.961948
(Iteration 1201 / 2450) loss: 1.932726
(Iteration 1301 / 2450) loss: 2.027675
(Iteration 1401 / 2450) loss: 1.984492
(Epoch 3 / 5) train acc: 0.372000; val_acc: 0.377000
(Iteration 1501 / 2450) loss: 1.979330
(Iteration 1601 / 2450) loss: 2.008644
(Iteration 1701 / 2450) loss: 1.991986
(Iteration 1801 / 2450) loss: 2.023756
(Iteration 1901 / 2450) loss: 1.923758
(Epoch 4 / 5) train acc: 0.359000; val_acc: 0.391000
(Iteration 2001 / 2450) loss: 1.955289
(Iteration 2101 / 2450) loss: 2.005906
(Iteration 2201 / 2450) loss: 1.889467
(Iteration 2301 / 2450) loss: 1.957606
(Iteration 2401 / 2450) loss: 2.034266
(Epoch 5 / 5) train acc: 0.368000; val_acc: 0.397000
0.397

```

12 Test your model!

Run your best model on the validation and test sets. You should achieve above 48% accuracy on the validation set and the test set.

```
[22]: print(best_val)
```

0.508

```
[23]: y_val_pred = np.argmax(best_model.loss(data['X_val']), axis=1)
print('Validation set accuracy: ', (y_val_pred == data['y_val']).mean())
```

Validation set accuracy: 0.508

```
[24]: y_test_pred = np.argmax(best_model.loss(data['X_test']), axis=1)
print('Test set accuracy: ', (y_test_pred == data['y_test']).mean())
```

Test set accuracy: 0.513

```
[25]: # Save best model
best_model.save("best_two_layer_net.npy")
```

best_two_layer_net.npy saved.

12.1 Inline Question 2:

Now that you have trained a Neural Network classifier, you may find that your testing accuracy is much lower than the training accuracy. In what ways can we decrease this gap? Select all that apply.

1. Train on a larger dataset.
2. Add more hidden units.
3. Increase the regularization strength.
4. None of the above.

Your Answer :

Cách các có thể làm là 1,3.

Your Explanation :

Vấn đề trên là overfit.

Cách 1 làm mô hình buộc phải học các đặc trưng tổng quát hơn của data, tránh học vẹt.

Cách 2 làm càng khiến cho model bị overfit, do model càng phức tạp.

Cách 3 là một phương pháp chống overfit thông dụng, tăng tham số regularization để W không quá phức tạp.

```
[ ]:
```

features

April 21, 2026

```
[1]: # This mounts your Google Drive to the Colab VM.
from google.colab import drive
drive.mount('/content/drive')

# TODO: Enter the foldername in your Drive where you have saved the unzipped
# assignment folder, e.g. 'cs231n/assignments/assignment1/'
FOLDERNAME = 'cs231n/assignments/assignment1/'
assert FOLDERNAME is not None, "[!] Enter the foldername."

# Now that we've mounted your Drive, this ensures that
# the Python interpreter of the Colab VM can load
# python files from within it.
import sys
sys.path.append('/content/drive/My Drive/{}'.format(FOLDERNAME))

# This downloads the CIFAR-10 dataset to your Drive
# if it doesn't already exist.
# %cd /content/drive/My\ Drive/$FOLDERNAME/cs231n/datasets/
# !bash get_datasets.sh
%cd /content/drive/My\ Drive/$FOLDERNAME
```

Mounted at /content/drive
/content/drive/.shortcut-targets-by-
id/16YEzBI86ON8oXEabllZ2vheRhnpKWioQ/cs231n/assignments/assignment1

```
[ ]: # !pip install --upgrade IPython
```

Requirement already satisfied: IPython in /usr/local/lib/python3.11/dist-packages (7.34.0)
Collecting IPython
 Downloading ipython-9.10.1-py3-none-any.whl.metadata (4.6 kB)
Requirement already satisfied: decorator>=4.3.2 in /usr/local/lib/python3.11/dist-packages (from IPython) (4.4.2)
Collecting ipython-pygments-lexers>=1.0.0 (from IPython)
 Downloading ipython_pygments_lexers-1.1.1-py3-none-any.whl.metadata (1.1 kB)
Collecting jedi>=0.18.1 (from IPython)
 Downloading jedi-0.19.2-py2.py3-none-any.whl.metadata (22 kB)
Requirement already satisfied: matplotlib-inline>=0.1.5 in

```

/usr/local/lib/python3.11/dist-packages (from IPython) (0.1.7)
Requirement already satisfied: pexpect>4.3 in /usr/local/lib/python3.11/dist-
packages (from IPython) (4.9.0)
Requirement already satisfied: prompt_toolkit<3.1.0,>=3.0.41 in
/usr/local/lib/python3.11/dist-packages (from IPython) (3.0.51)
Requirement already satisfied: pygments>=2.11.0 in
/usr/local/lib/python3.11/dist-packages (from IPython) (2.19.2)
Collecting stack_data>=0.6.0 (from IPython)
  Downloading stack_data-0.6.3-py3-none-any.whl.metadata (18 kB)
Collecting traitlets>=5.13.0 (from IPython)
  Downloading traitlets-5.14.3-py3-none-any.whl.metadata (10 kB)
Requirement already satisfied: typing_extensions>=4.6 in
/usr/local/lib/python3.11/dist-packages (from IPython) (4.14.1)
Requirement already satisfied: parso<0.9.0,>=0.8.4 in
/usr/local/lib/python3.11/dist-packages (from jedi>=0.18.1->IPython) (0.8.4)
Requirement already satisfied: ptyprocess>=0.5 in
/usr/local/lib/python3.11/dist-packages (from pexpect>4.3->IPython) (0.7.0)
Requirement already satisfied: wcwidth in /usr/local/lib/python3.11/dist-
packages (from prompt_toolkit<3.1.0,>=3.0.41->IPython) (0.2.13)
Collecting executing>=1.2.0 (from stack_data>=0.6.0->IPython)
  Downloading executing-2.2.1-py2.py3-none-any.whl.metadata (8.9 kB)
Collecting asttokens>=2.1.0 (from stack_data>=0.6.0->IPython)
  Downloading asttokens-3.0.1-py3-none-any.whl.metadata (4.9 kB)
Collecting pure-eval (from stack_data>=0.6.0->IPython)
  Downloading pure_eval-0.2.3-py3-none-any.whl.metadata (6.3 kB)
Downloading ipython-9.10.1-py3-none-any.whl (622 kB)
      622.8/622.8 kB
13.3 MB/s eta 0:00:00
Downloading ipython_pygments_lexers-1.1.1-py3-none-any.whl (8.1 kB)
Downloading jedi-0.19.2-py2.py3-none-any.whl (1.6 MB)
      1.6/1.6 MB
54.8 MB/s eta 0:00:00
Downloading stack_data-0.6.3-py3-none-any.whl (24 kB)
Downloading traitlets-5.14.3-py3-none-any.whl (85 kB)
      85.4/85.4 kB
11.1 MB/s eta 0:00:00
Downloading asttokens-3.0.1-py3-none-any.whl (27 kB)
Downloading executing-2.2.1-py2.py3-none-any.whl (28 kB)
Downloading pure_eval-0.2.3-py3-none-any.whl (11 kB)
Installing collected packages: pure-eval, traitlets, jedi, ipython-pygments-
lexers, executing, asttokens, stack_data, IPython
  Attempting uninstall: traitlets
    Found existing installation: traitlets 5.7.1
    Uninstalling traitlets-5.7.1:
      Successfully uninstalled traitlets-5.7.1
  Attempting uninstall: IPython
    Found existing installation: ipython 7.34.0
    Uninstalling ipython-7.34.0:

```

```
Successfully uninstalled ipython-7.34.0
ERROR: pip's dependency resolver does not currently take into account all
the packages that are installed. This behaviour is the source of the following
dependency conflicts.

google-colab 1.0.0 requires ipython==7.34.0, but you have ipython 9.10.1 which
is incompatible.

Successfully installed IPython-9.10.1 asttokens-3.0.1 executing-2.2.1
ipython-pygments-lexers-1.1.1 jedi-0.19.2 pure-eval-0.2.3 stack_data-0.6.3
traitlets-5.14.3
```

1 Image features exercise

Complete and hand in this completed worksheet (including its outputs and any supporting code outside of the worksheet) with your assignment submission. For more details see the [assignments page](#) on the course website.

We have seen that we can achieve reasonable performance on an image classification task by training a linear classifier on the pixels of the input image. In this exercise we will show that we can improve our classification performance by training linear classifiers not on raw pixels but on features that are computed from the raw pixels.

All of your work for this exercise will be done in this notebook.

```
[2]: import random
import numpy as np
from cs231n.data_utils import load_CIFAR10
import matplotlib.pyplot as plt

%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# for auto-reloading external modules
# see http://stackoverflow.com/questions/1907993/
# ↪ autoreload-of-modules-in-ipython
%load_ext autoreload
%autoreload 2
```

1.1 Load data

Similar to previous exercises, we will load CIFAR-10 data from disk.

```
[3]: from cs231n.features import color_histogram_hsv, hog_feature

def get_CIFAR10_data(num_training=49000, num_validation=1000, num_test=1000):
    # Load the raw CIFAR-10 data
    cifar10_dir = 'cs231n/datasets/cifar-10-batches-py'

    # Cleaning up variables to prevent loading data multiple times (which may
    # cause memory issue)
    try:
        del X_train, y_train
        del X_test, y_test
        print('Clear previously loaded data.')
    except:
        pass

    X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

    # Subsample the data
    mask = list(range(num_training, num_training + num_validation))
    X_val = X_train[mask]
    y_val = y_train[mask]
    mask = list(range(num_training))
    X_train = X_train[mask]
    y_train = y_train[mask]
    mask = list(range(num_test))
    X_test = X_test[mask]
    y_test = y_test[mask]

    return X_train, y_train, X_val, y_val, X_test, y_test

X_train, y_train, X_val, y_val, X_test, y_test = get_CIFAR10_data()
```

1.2 Extract Features

For each image we will compute a Histogram of Oriented Gradients (HOG) as well as a color histogram using the hue channel in HSV color space. We form our final feature vector for each image by concatenating the HOG and color histogram feature vectors.

Roughly speaking, HOG should capture the texture of the image while ignoring color information, and the color histogram represents the color of the input image while ignoring texture. As a result, we expect that using both together ought to work better than using either alone. Verifying this assumption would be a good thing to try for your own interest.

The `hog_feature` and `color_histogram_hsv` functions both operate on a single image and return a feature vector for that image. The `extract_features` function takes a set of images and a list of feature functions and evaluates each feature function on each image, storing the results in a matrix where each column is the concatenation of all feature vectors for a single image.

```
[4]: from cs231n.features import *

# num_color_bins = 10 # Number of bins in the color histogram
num_color_bins = 25 # Number of bins in the color histogram
feature_fns = [hog_feature, lambda img: color_histogram_hsv(img,
    ↪nbin=num_color_bins)]
X_train_feats = extract_features(X_train, feature_fns, verbose=True)
X_val_feats = extract_features(X_val, feature_fns)
X_test_feats = extract_features(X_test, feature_fns)

# Preprocessing: Subtract the mean feature
mean_feat = np.mean(X_train_feats, axis=0, keepdims=True)
X_train_feats -= mean_feat
X_val_feats -= mean_feat
X_test_feats -= mean_feat

# Preprocessing: Divide by standard deviation. This ensures that each feature
# has roughly the same scale.
std_feat = np.std(X_train_feats, axis=0, keepdims=True)
X_train_feats /= std_feat
X_val_feats /= std_feat
X_test_feats /= std_feat

# Preprocessing: Add a bias dimension
X_train_feats = np.hstack([X_train_feats, np.ones((X_train_feats.shape[0], 1))])
X_val_feats = np.hstack([X_val_feats, np.ones((X_val_feats.shape[0], 1))])
X_test_feats = np.hstack([X_test_feats, np.ones((X_test_feats.shape[0], 1))])
```

```
Done extracting features for 1000 / 49000 images
Done extracting features for 2000 / 49000 images
Done extracting features for 3000 / 49000 images
Done extracting features for 4000 / 49000 images
Done extracting features for 5000 / 49000 images
Done extracting features for 6000 / 49000 images
Done extracting features for 7000 / 49000 images
Done extracting features for 8000 / 49000 images
Done extracting features for 9000 / 49000 images
Done extracting features for 10000 / 49000 images
Done extracting features for 11000 / 49000 images
Done extracting features for 12000 / 49000 images
Done extracting features for 13000 / 49000 images
Done extracting features for 14000 / 49000 images
Done extracting features for 15000 / 49000 images
Done extracting features for 16000 / 49000 images
Done extracting features for 17000 / 49000 images
Done extracting features for 18000 / 49000 images
Done extracting features for 19000 / 49000 images
```

```
Done extracting features for 20000 / 49000 images
Done extracting features for 21000 / 49000 images
Done extracting features for 22000 / 49000 images
Done extracting features for 23000 / 49000 images
Done extracting features for 24000 / 49000 images
Done extracting features for 25000 / 49000 images
Done extracting features for 26000 / 49000 images
Done extracting features for 27000 / 49000 images
Done extracting features for 28000 / 49000 images
Done extracting features for 29000 / 49000 images
Done extracting features for 30000 / 49000 images
Done extracting features for 31000 / 49000 images
Done extracting features for 32000 / 49000 images
Done extracting features for 33000 / 49000 images
Done extracting features for 34000 / 49000 images
Done extracting features for 35000 / 49000 images
Done extracting features for 36000 / 49000 images
Done extracting features for 37000 / 49000 images
Done extracting features for 38000 / 49000 images
Done extracting features for 39000 / 49000 images
Done extracting features for 40000 / 49000 images
Done extracting features for 41000 / 49000 images
Done extracting features for 42000 / 49000 images
Done extracting features for 43000 / 49000 images
Done extracting features for 44000 / 49000 images
Done extracting features for 45000 / 49000 images
Done extracting features for 46000 / 49000 images
Done extracting features for 47000 / 49000 images
Done extracting features for 48000 / 49000 images
Done extracting features for 49000 / 49000 images
```

1.3 Train Softmax classifier on features

Using the Softmax code developed earlier in the assignment, train Softmax classifiers on top of the features extracted above; this should achieve better results than training them directly on top of raw pixels.

```
[5]: # Use the validation set to tune the learning rate and regularization strength

from cs231n.classifiers.linear_classifier import Softmax

learning_rates = [1e-7, 1e-6]
# regularization_strengths = [5e5, 5e6]
regularization_strengths = [1e4, 1e5]

results = {}
best_val = -1
best_softmax = None
```



```
#####
# TODO: #
# Use the validation set to set the learning rate and regularization strength. #
# This should be identical to the validation that you did for the Softmax; save #
# the best trained classifier in best_softmax. If you carefully tune the model, #
# you should be able to get accuracy of above 0.42 on the validation set. #
#####
for lr in learning_rates :
    for rg in regularization_strengths :
        softmax = Softmax()
        softmax.train(X_train_feats, y_train,
            learning_rate=lr, reg=rg, num_iters=1500, batch_size=200, verbose=True)
        y_train_pred = softmax.predict(X_train_feats)
        train_accuracy = np.mean(y_train == y_train_pred)
        y_val_pred = softmax.predict(X_val_feats)
        val_accuracy = np.mean(y_val == y_val_pred)
        results[(lr, rg)] = (train_accuracy, val_accuracy)
        if val_accuracy > best_val:
            best_val = val_accuracy
            best_softmax = softmax

# Print out results.
for lr, reg in sorted(results):
    train_accuracy, val_accuracy = results[(lr, reg)]
    print('lr %e reg %e train accuracy: %f val accuracy: %f' % (
        lr, reg, train_accuracy, val_accuracy))

print('best validation accuracy achieved: %f' % best_val)
```

```
iteration 0 / 1500: loss 18.933224
iteration 100 / 1500: loss 13.446171
iteration 200 / 1500: loss 9.769468
iteration 300 / 1500: loss 7.306252
iteration 400 / 1500: loss 5.655153
iteration 500 / 1500: loss 4.548723
iteration 600 / 1500: loss 3.807559
iteration 700 / 1500: loss 3.311106
iteration 800 / 1500: loss 2.978113
iteration 900 / 1500: loss 2.755326
iteration 1000 / 1500: loss 2.605919
iteration 1100 / 1500: loss 2.505898
iteration 1200 / 1500: loss 2.438772
iteration 1300 / 1500: loss 2.393810
iteration 1400 / 1500: loss 2.363655
iteration 0 / 1500: loss 172.237095
iteration 100 / 1500: loss 5.291266
```

```
iteration 200 / 1500: loss 2.355154
iteration 300 / 1500: loss 2.303506
iteration 400 / 1500: loss 2.302599
iteration 500 / 1500: loss 2.302583
iteration 600 / 1500: loss 2.302584
iteration 700 / 1500: loss 2.302583
iteration 800 / 1500: loss 2.302583
iteration 900 / 1500: loss 2.302583
iteration 1000 / 1500: loss 2.302583
iteration 1100 / 1500: loss 2.302583
iteration 1200 / 1500: loss 2.302583
iteration 1300 / 1500: loss 2.302584
iteration 1400 / 1500: loss 2.302583
iteration 0 / 1500: loss 18.909257
iteration 100 / 1500: loss 2.594697
iteration 200 / 1500: loss 2.307697
iteration 300 / 1500: loss 2.302663
iteration 400 / 1500: loss 2.302572
iteration 500 / 1500: loss 2.302565
iteration 600 / 1500: loss 2.302562
iteration 700 / 1500: loss 2.302568
iteration 800 / 1500: loss 2.302565
iteration 900 / 1500: loss 2.302565
iteration 1000 / 1500: loss 2.302568
iteration 1100 / 1500: loss 2.302565
iteration 1200 / 1500: loss 2.302567
iteration 1300 / 1500: loss 2.302567
iteration 1400 / 1500: loss 2.302572
iteration 0 / 1500: loss 168.621340
iteration 100 / 1500: loss 2.302583
iteration 200 / 1500: loss 2.302583
iteration 300 / 1500: loss 2.302583
iteration 400 / 1500: loss 2.302583
iteration 500 / 1500: loss 2.302583
iteration 600 / 1500: loss 2.302584
iteration 700 / 1500: loss 2.302583
iteration 800 / 1500: loss 2.302583
iteration 900 / 1500: loss 2.302583
iteration 1000 / 1500: loss 2.302583
iteration 1100 / 1500: loss 2.302584
iteration 1200 / 1500: loss 2.302583
iteration 1300 / 1500: loss 2.302583
iteration 1400 / 1500: loss 2.302583
lr 1.000000e-07 reg 1.000000e+04 train accuracy: 0.105449 val accuracy: 0.098000
lr 1.000000e-07 reg 1.000000e+05 train accuracy: 0.420143 val accuracy: 0.429000
lr 1.000000e-06 reg 1.000000e+04 train accuracy: 0.422592 val accuracy: 0.426000
lr 1.000000e-06 reg 1.000000e+05 train accuracy: 0.397429 val accuracy: 0.406000
best validation accuracy achieved: 0.429000
```

```
[6]: # Evaluate your trained Softmax on the test set: you should be able to get at
      ↪ least 0.42
y_test_pred = best_softmax.predict(X_test_feats)
test_accuracy = np.mean(y_test == y_test_pred)
print(test_accuracy)
```

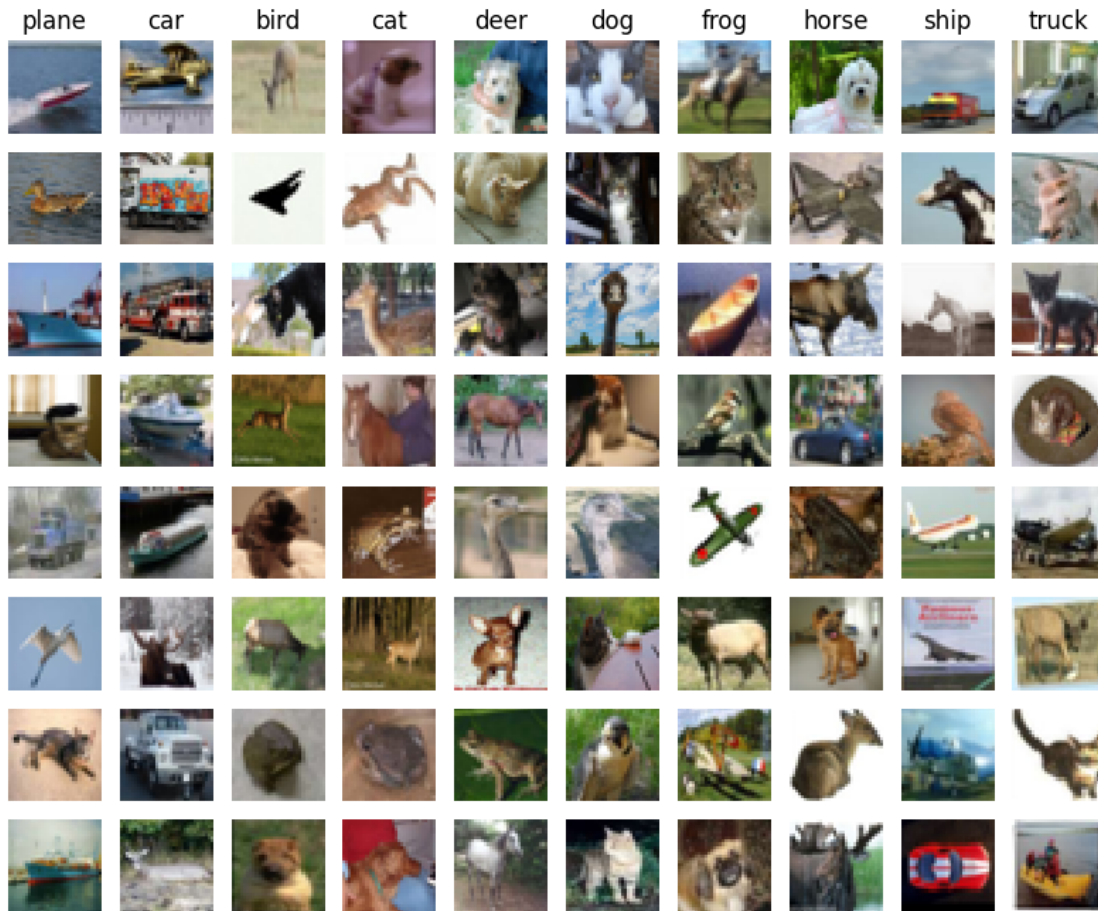
0.425

```
[7]: # Save best softmax model
best_softmax.save("best_softmax_features.npy")
```

best_softmax_features.npy saved.

```
[8]: # An important way to gain intuition about how an algorithm works is to
      # visualize the mistakes that it makes. In this visualization, we show examples
      # of images that are misclassified by our current system. The first column
      # shows images that our system labeled as "plane" but whose true label is
      # something other than "plane".

examples_per_class = 8
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse',
          ↪ 'ship', 'truck']
for cls, cls_name in enumerate(classes):
    idxs = np.where((y_test != cls) & (y_test_pred == cls))[0]
    idxs = np.random.choice(idxs, examples_per_class, replace=False)
    for i, idx in enumerate(idxs):
        plt.subplot(examples_per_class, len(classes), i * len(classes) + cls +
          ↪ 1)
        plt.imshow(X_test[idx].astype('uint8'))
        plt.axis('off')
        if i == 0:
            plt.title(cls_name)
plt.show()
```



1.3.1 Inline question 1:

Describe the misclassification results that you see. Do they make sense?

Your Answer :

vì những ảnh này có các viền của object tương tự như ảnh nhân , mà thuật toán HOG rất chú ý vào những đặc trưng viền cạnh này, thông qua tính gradient của các pixel

1.4 Neural Network on image features

Earlier in this assignment we saw that training a two-layer neural network on raw pixels achieved better classification performance than linear classifiers on raw pixels. In this notebook we have seen that linear classifiers on image features outperform linear classifiers on raw pixels.

For completeness, we should also try training a neural network on image features. This approach should outperform all previous approaches: you should easily be able to achieve over 55% classification accuracy on the test set; our best model achieves about 60% classification accuracy.

```
[9]: # Preprocessing: Remove the bias dimension
# Make sure to run this cell only ONCE
print(X_train_feats.shape)
X_train_feats = X_train_feats[:, :-1]
X_val_feats = X_val_feats[:, :-1]
X_test_feats = X_test_feats[:, :-1]

print(X_train_feats.shape)
```

(49000, 170)

(49000, 169)

```
[11]: from cs231n.classifiers.fc_net import TwoLayerNet
from cs231n.solver import Solver

input_dim = X_train_feats.shape[1]
hidden_dim = 500
num_classes = 10

data = {
    'X_train': X_train_feats,
    'y_train': y_train,
    'X_val': X_val_feats,
    'y_val': y_val,
    'X_test': X_test_feats,
    'y_test': y_test,
}

net = TwoLayerNet(input_dim, hidden_dim, num_classes)
best_net = None

#####
# TODO: Train a two-layer neural network on image features. You may want to #
# cross-validate various parameters as in previous sections. Store your best #
# model in the best_net variable. #
# num_folds = 3
# k_choices = [1, 3, 5, 8, 10, 12, 15, 20, 50, 100]

# X_train_folds = np.array_split(data['X_train'], num_folds)
# y_train_folds = np.array_split(data['y_train'], num_folds)
# X_train_merged = np.vstack([data['X_train'], data['X_val']])
# y_train_merged = np.concatenate([data['y_train'], data['y_val']])

# X_train_folds = np.array_split(X_train_merged, num_folds)
# y_train_folds = np.array_split(y_train_merged, num_folds)
# learning_rates = [1e-7, 1e-5, 1e-4, 1e-3, 2e-3]
```

```

learning_rates = [1e-3,1e-2,1e-1,3e-1,5e-1]
# regularization_strengths = [5e5, 5e6]
regularization_strengths = [0.0,10]
learning_rates_decay = [0.92,0.95]

results = {}
best_val = -1
best_net = None
best_params = None

for lr in learning_rates:
    for rg in regularization_strengths:
        for lr_decay in learning_rates_decay:
            print(f"eval lr= {lr}, reg= {rg}, lr_decay ={lr_decay}")
            # val_acc_sum = 0
            # for i in range(num_folds):
            #     X_val_fold = X_train_folds[i]
            #     y_val_fold = y_train_folds[i]
            #     X_train_fold = np.concatenate(X_train_folds[:i] +
↪X_train_folds[i+1:])
            #     y_train_fold = np.concatenate(y_train_folds[:i] +
↪y_train_folds[i+1:])
            #     fold_data = {
            #         'X_train': X_train_fold,
            #         'y_train': y_train_fold,
            #         'X_val': X_val_fold,
            #         'y_val': y_val_fold,
            #     }

            net = TwoLayerNet(input_dim, hidden_dim, num_classes, reg=rg)

            solver = Solver(net, data,
                            update_rule='sgd',
                            optim_config={'learning_rate': lr},
                            lr_decay=lr_decay,
                            num_epochs=5,
                            batch_size=100,
                            verbose=False)

            solver.train()
            val_acc = solver.best_val_acc

            # avg_val_acc = val_acc_sum / num_folds
            results[(lr, rg)] = val_acc
            print(f"val accuracy: {val_acc:f}")
            if val_acc > best_val:
                best_val = val_acc

```

```

        best_params = (lr, rg,lr_decay)
        best_net = net

print(30*"=")
print(f"best cross-val accuracy: {best_val:f} with lr={best_params[0]} ,
      ↪reg={best_params[1]}")

#####

```

```

eval lr= 0.001, reg= 0.0, lr_decay =0.92
val accuracy: 0.222000
eval lr= 0.001, reg= 0.0, lr_decay =0.95
val accuracy: 0.231000
eval lr= 0.001, reg= 10, lr_decay =0.92
val accuracy: 0.113000
eval lr= 0.001, reg= 10, lr_decay =0.95
val accuracy: 0.115000
eval lr= 0.01, reg= 0.0, lr_decay =0.92
val accuracy: 0.378000
eval lr= 0.01, reg= 0.0, lr_decay =0.95
val accuracy: 0.411000
eval lr= 0.01, reg= 10, lr_decay =0.92
val accuracy: 0.113000
eval lr= 0.01, reg= 10, lr_decay =0.95
val accuracy: 0.102000
eval lr= 0.1, reg= 0.0, lr_decay =0.92
val accuracy: 0.560000
eval lr= 0.1, reg= 0.0, lr_decay =0.95
val accuracy: 0.568000
eval lr= 0.1, reg= 10, lr_decay =0.92
val accuracy: 0.113000
eval lr= 0.1, reg= 10, lr_decay =0.95
val accuracy: 0.119000
eval lr= 0.3, reg= 0.0, lr_decay =0.92
val accuracy: 0.585000
eval lr= 0.3, reg= 0.0, lr_decay =0.95
val accuracy: 0.593000
eval lr= 0.3, reg= 10, lr_decay =0.92

/content/drive/.shortcut-targets-by-id/16YEzBI86ON8oXEabl1Z2vheRhnpKWi0Q/cs231n/
assignments/assignment1/cs231n/layers.py:819: RuntimeWarning: divide by zero
encountered in log
    log_probs = np.log(probs)
/content/drive/.shortcut-targets-by-id/16YEzBI86ON8oXEabl1Z2vheRhnpKWi0Q/cs231n/
assignments/assignment1/cs231n/layers.py:33: RuntimeWarning: overflow
encountered in matmul
    out = x_trans @ w + b
/content/drive/.shortcut-targets-by-id/16YEzBI86ON8oXEabl1Z2vheRhnpKWi0Q/cs231n/

```

```

assignments/assignment1/cs231n/layers.py:814: RuntimeWarning: overflow
encountered in subtract
    expo = x - np.max(x,axis=1,keepdims=True)
/content/drive/.shortcut-targets-by-id/16YEzBI860N8oXEabllZ2vheRhnpKWi0Q/cs231n/
assignments/assignment1/cs231n/layers.py:814: RuntimeWarning: invalid value
encountered in subtract
    expo = x - np.max(x,axis=1,keepdims=True)
/content/drive/.shortcut-targets-by-id/16YEzBI860N8oXEabllZ2vheRhnpKWi0Q/cs231n/
assignments/assignment1/cs231n/classifiers/fc_net.py:131: RuntimeWarning:
overflow encountered in scalar multiply
    loss += self.reg*0.5*np.sum(self.params[item] * self.params[item])

val accuracy: 0.105000
eval lr= 0.3, reg= 10, lr_decay =0.95
val accuracy: 0.087000
eval lr= 0.5, reg= 0.0, lr_decay =0.92
val accuracy: 0.565000
eval lr= 0.5, reg= 0.0, lr_decay =0.95
val accuracy: 0.576000
eval lr= 0.5, reg= 10, lr_decay =0.92
val accuracy: 0.111000
eval lr= 0.5, reg= 10, lr_decay =0.95

/usr/local/lib/python3.11/dist-packages/numpy/_core/fromnumeric.py:86:
RuntimeWarning: overflow encountered in reduce
    return ufunc.reduce(obj, axis, dtype, out, **passkwargs)

val accuracy: 0.101000
=====
best cross-val accuracy: 0.593000 with lr=0.3 , reg=0.0

```

```

[12]: # Run your best neural net classifier on the test set. You should be able
# to get more than 58% accuracy. It is also possible to get >60% accuracy
# with careful tuning.

```

```

y_test_pred = np.argmax(best_net.loss(data['X_test']), axis=1)
test_acc = (y_test_pred == data['y_test']).mean()
print(test_acc)

```

0.579

```

[13]: # Save best model
best_net.save("best_two_layer_net_features.npy")

```

best_two_layer_net_features.npy saved.

```

[ ]:

```


FullyConnectedNets

April 21, 2026

```
[ ]: # This mounts your Google Drive to the Colab VM.
from google.colab import drive
drive.mount('/content/drive')

# TODO: Enter the foldername in your Drive where you have saved the unzipped
# assignment folder, e.g. 'cs231n/assignments/assignment1/'
FOLDERNAME = 'cs231n/assignments/assignment1/'
assert FOLDERNAME is not None, "[!] Enter the foldername."

# Now that we've mounted your Drive, this ensures that
# the Python interpreter of the Colab VM can load
# python files from within it.
import sys
sys.path.append('/content/drive/My Drive/{}'.format(FOLDERNAME))

# This downloads the CIFAR-10 dataset to your Drive
# if it doesn't already exist.
# %cd /content/drive/My\ Drive/$FOLDERNAME/cs231n/datasets/
# %cd /content/drive/My\ Drive/$FOLDERNAME
# !bash get_datasets.sh
%cd /content/drive/My\ Drive/$FOLDERNAME
```

Mounted at /content/drive
/content/drive/My Drive/cs231n/assignments/assignment1

```
[ ]: # !pip install --upgrade IPython
```

```
Requirement already satisfied: IPython in /usr/local/lib/python3.11/dist-
packages (7.34.0)
Collecting IPython
  Downloading ipython-9.10.1-py3-none-any.whl.metadata (4.6 kB)
Requirement already satisfied: decorator>=4.3.2 in
/usr/local/lib/python3.11/dist-packages (from IPython) (4.4.2)
Collecting ipython-pygments-lexers>=1.0.0 (from IPython)
  Downloading ipython_pygments_lexers-1.1.1-py3-none-any.whl.metadata (1.1 kB)
Collecting jedi>=0.18.1 (from IPython)
  Downloading jedi-0.19.2-py2.py3-none-any.whl.metadata (22 kB)
Requirement already satisfied: matplotlib-inline>=0.1.5 in
```

```

/usr/local/lib/python3.11/dist-packages (from IPython) (0.1.7)
Requirement already satisfied: pexpect>4.3 in /usr/local/lib/python3.11/dist-
packages (from IPython) (4.9.0)
Requirement already satisfied: prompt_toolkit<3.1.0,>=3.0.41 in
/usr/local/lib/python3.11/dist-packages (from IPython) (3.0.51)
Requirement already satisfied: pygments>=2.11.0 in
/usr/local/lib/python3.11/dist-packages (from IPython) (2.19.2)
Collecting stack_data>=0.6.0 (from IPython)
  Downloading stack_data-0.6.3-py3-none-any.whl.metadata (18 kB)
Collecting traitlets>=5.13.0 (from IPython)
  Downloading traitlets-5.14.3-py3-none-any.whl.metadata (10 kB)
Requirement already satisfied: typing_extensions>=4.6 in
/usr/local/lib/python3.11/dist-packages (from IPython) (4.14.1)
Requirement already satisfied: parso<0.9.0,>=0.8.4 in
/usr/local/lib/python3.11/dist-packages (from jedi>=0.18.1->IPython) (0.8.4)
Requirement already satisfied: ptyprocess>=0.5 in
/usr/local/lib/python3.11/dist-packages (from pexpect>4.3->IPython) (0.7.0)
Requirement already satisfied: wcwidth in /usr/local/lib/python3.11/dist-
packages (from prompt_toolkit<3.1.0,>=3.0.41->IPython) (0.2.13)
Collecting executing>=1.2.0 (from stack_data>=0.6.0->IPython)
  Downloading executing-2.2.1-py2.py3-none-any.whl.metadata (8.9 kB)
Collecting asttokens>=2.1.0 (from stack_data>=0.6.0->IPython)
  Downloading asttokens-3.0.1-py3-none-any.whl.metadata (4.9 kB)
Collecting pure-eval (from stack_data>=0.6.0->IPython)
  Downloading pure_eval-0.2.3-py3-none-any.whl.metadata (6.3 kB)
Downloading ipython-9.10.1-py3-none-any.whl (622 kB)
      622.8/622.8 kB
4.0 MB/s eta 0:00:0000:0100:01
Downloading ipython_pygments_lexers-1.1.1-py3-none-any.whl (8.1 kB)
Downloading jedi-0.19.2-py2.py3-none-any.whl (1.6 MB)
      1.6/1.6 MB
22.0 MB/s eta 0:00:0000:0100:01
Downloading stack_data-0.6.3-py3-none-any.whl (24 kB)
Downloading traitlets-5.14.3-py3-none-any.whl (85 kB)
      85.4/85.4 kB
10.5 MB/s eta 0:00:00
Downloading asttokens-3.0.1-py3-none-any.whl (27 kB)
Downloading executing-2.2.1-py2.py3-none-any.whl (28 kB)
Downloading pure_eval-0.2.3-py3-none-any.whl (11 kB)
Installing collected packages: pure-eval, traitlets, jedi, ipython-pygments-
lexers, executing, asttokens, stack_data, IPython
  Attempting uninstall: traitlets
    Found existing installation: traitlets 5.7.1
    Uninstalling traitlets-5.7.1:
      Successfully uninstalled traitlets-5.7.1
  Attempting uninstall: IPython
    Found existing installation: ipython 7.34.0
    Uninstalling ipython-7.34.0:

```

Successfully uninstalled ipython-7.34.0
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.
google-colab 1.0.0 requires ipython==7.34.0, but you have ipython 9.10.1 which is incompatible.

Successfully installed IPython-9.10.1 asttokens-3.0.1 executing-2.2.1
ipython-pygments-lexers-1.1.1 jedi-0.19.2 pure-eval-0.2.3 stack_data-0.6.3
traitlets-5.14.3

1 Multi-Layer Fully Connected Network

In this exercise, you will implement a fully connected network with an arbitrary number of hidden layers.

```
[ ]: # from google.colab import drive
      # drive.mount('/content/drive')
```

Read through the FullyConnectedNet class in the file cs231n/classifiers/fc_net.py.

Implement the network initialization, forward pass, and backward pass. Throughout this assignment, you will be implementing layers in cs231n/layers.py. You can re-use your implementations for affine_forward, affine_backward, relu_forward, relu_backward, and softmax_loss from before. For right now, don't worry about implementing dropout or batch/layer normalization yet, as you will add those features later.

```
[ ]: # Setup cell.
import time
import numpy as np
import matplotlib.pyplot as plt
from cs231n.classifiers.fc_net import *
from cs231n.data_utils import get_CIFAR10_data
from cs231n.gradient_check import eval_numerical_gradient, \
    eval_numerical_gradient_array
from cs231n.solver import Solver

%matplotlib inline
plt.rcParams["figure.figsize"] = (10.0, 8.0) # Set default size of plots.
plt.rcParams["image.interpolation"] = "nearest"
plt.rcParams["image.cmap"] = "gray"

%load_ext autoreload
%autoreload 2

def rel_error(x, y):
```

```

"""Returns relative error."""
return np.max(np.abs(x - y) / (np.maximum(1e-8, np.abs(x) + np.abs(y))))

```

```

[ ]: # Load the (preprocessed) CIFAR-10 data.
data = get_CIFAR10_data()
for k, v in list(data.items()):
    print(f"{k}: {v.shape}")

```

```

X_train: (49000, 3, 32, 32)
y_train: (49000,)
X_val: (1000, 3, 32, 32)
y_val: (1000,)
X_test: (1000, 3, 32, 32)
y_test: (1000,)

```

1.1 Initial Loss and Gradient Check

As a sanity check, run the following to check the initial loss and to gradient check the network both with and without regularization. This is a good way to see if the initial losses seem reasonable.

For gradient checking, you should expect to see errors around $1e-7$ or less.

```

[ ]: np.random.seed(231)
N, D, H1, H2, C = 2, 15, 20, 30, 10
X = np.random.randn(N, D)
y = np.random.randint(C, size=(N,))

for reg in [0, 3.14]:
    print("Running check with reg = ", reg)
    model = FullyConnectedNet(
        [H1, H2],
        input_dim=D,
        num_classes=C,
        reg=reg,
        weight_scale=5e-2,
        dtype=np.float64
    )
    # print(model.params)
    loss, grads = model.loss(X, y)
    # for name in model.params:
    #     print(name + ' wrong')

    # Most of the errors should be on the order of e-7 or smaller.
    # NOTE: It is fine however to see an error for W2 on the order of e-5
    # for the check when reg = 0.0
    for name in sorted(grads):
        # print("chay vao day")
        f = lambda _: model.loss(X, y)[0]

```

```

        grad_num = eval_numerical_gradient(f, model.params[name], ▯
↪ verbose=False, h=1e-5)
        print(f"{name} relative error: {rel_error(grad_num, grads[name])}")

```

```

Running check with reg = 0
W1 relative error: 1.4839894979053162e-07
W2 relative error: 0.00020570303548494203
W3 relative error: 5.134310517772522e-07
b1 relative error: 5.376386325179258e-09
b2 relative error: 9.899294796427697e-10
b3 relative error: 8.53836872706499e-11
Running check with reg = 3.14
W1 relative error: 3.904541941902138e-09
W2 relative error: 6.86942277940646e-08
W3 relative error: 3.483989247437803e-08
b1 relative error: 1.4752428222134868e-08
b2 relative error: 1.7223751746766738e-09
b3 relative error: 1.5702707897417976e-10

```

As another sanity check, make sure your network can overfit on a small dataset of 50 images. First, we will try a three-layer network with 100 units in each hidden layer. In the following cell, tweak the **learning rate** and **weight initialization scale** to overfit and achieve 100% training accuracy within 20 epochs.

```

[ ]: # TODO: Use a three-layer Net to overfit 50 training examples by
      # tweaking just the learning rate and initialization scale.

```

```

num_train = 50
small_data = {
    "X_train": data["X_train"][:num_train],
    "y_train": data["y_train"][:num_train],
    "X_val": data["X_val"],
    "y_val": data["y_val"],
}

weight_scale = 2e-2  # Experiment with this!
learning_rate = 2e-3  # Experiment with this!

model = FullyConnectedNet(
    [100, 100],
    weight_scale=weight_scale,
    dtype=np.float64
)
solver = Solver(
    model,
    small_data,
    print_every=10,

```

```

    num_epochs=20,
    batch_size=25,
    update_rule="sgd",
    optim_config={"learning_rate": learning_rate},
)
solver.train()

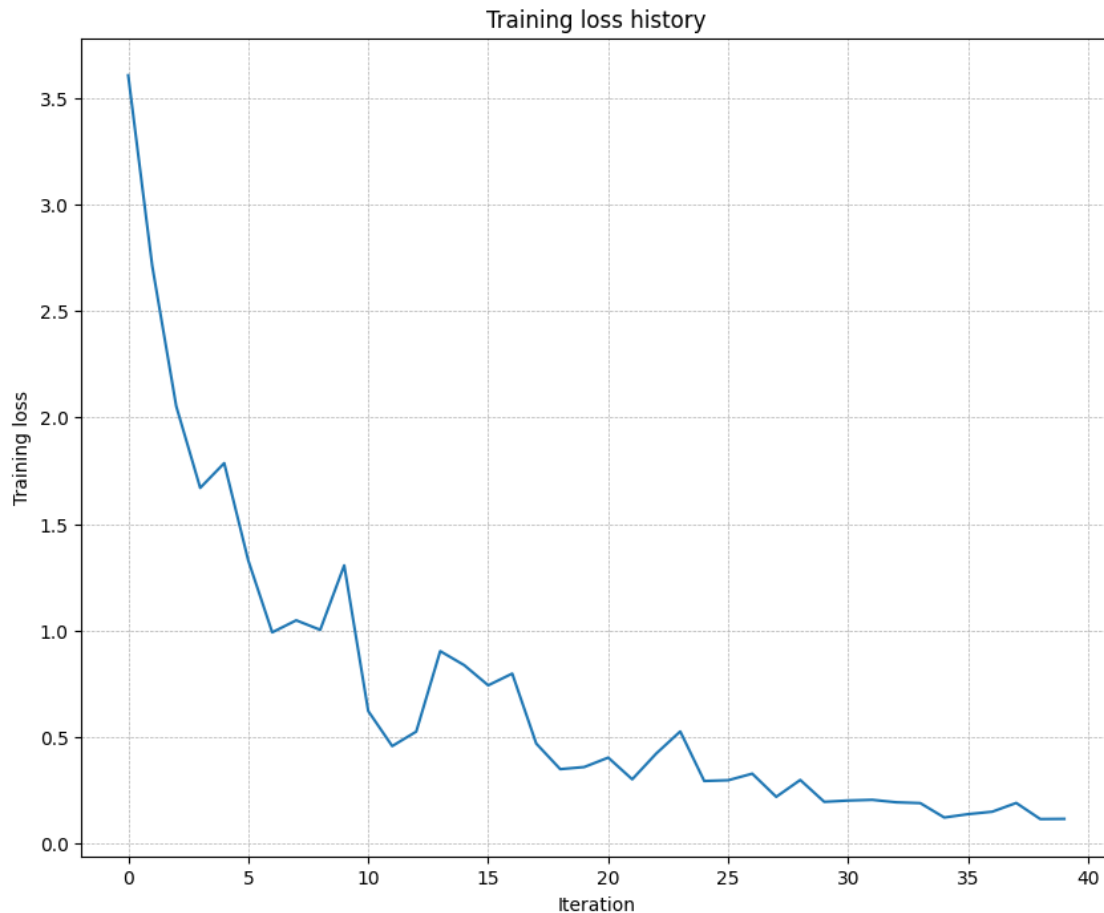
plt.plot(solver.loss_history)
plt.title("Training loss history")
plt.xlabel("Iteration")
plt.ylabel("Training loss")
plt.grid(linestyle='--', linewidth=0.5)
plt.show()

```

```

(Iteration 1 / 40) loss: 3.604568
(Epoch 0 / 20) train acc: 0.140000; val_acc: 0.116000
(Epoch 1 / 20) train acc: 0.300000; val_acc: 0.137000
(Epoch 2 / 20) train acc: 0.540000; val_acc: 0.150000
(Epoch 3 / 20) train acc: 0.640000; val_acc: 0.162000
(Epoch 4 / 20) train acc: 0.760000; val_acc: 0.168000
(Epoch 5 / 20) train acc: 0.840000; val_acc: 0.175000
(Iteration 11 / 40) loss: 0.622784
(Epoch 6 / 20) train acc: 0.880000; val_acc: 0.188000
(Epoch 7 / 20) train acc: 0.900000; val_acc: 0.184000
(Epoch 8 / 20) train acc: 0.920000; val_acc: 0.190000
(Epoch 9 / 20) train acc: 0.980000; val_acc: 0.187000
(Epoch 10 / 20) train acc: 0.960000; val_acc: 0.179000
(Iteration 21 / 40) loss: 0.404246
(Epoch 11 / 20) train acc: 0.980000; val_acc: 0.204000
(Epoch 12 / 20) train acc: 1.000000; val_acc: 0.183000
(Epoch 13 / 20) train acc: 1.000000; val_acc: 0.189000
(Epoch 14 / 20) train acc: 1.000000; val_acc: 0.188000
(Epoch 15 / 20) train acc: 1.000000; val_acc: 0.190000
(Iteration 31 / 40) loss: 0.203004
(Epoch 16 / 20) train acc: 1.000000; val_acc: 0.203000
(Epoch 17 / 20) train acc: 1.000000; val_acc: 0.198000
(Epoch 18 / 20) train acc: 1.000000; val_acc: 0.203000
(Epoch 19 / 20) train acc: 1.000000; val_acc: 0.190000
(Epoch 20 / 20) train acc: 1.000000; val_acc: 0.191000

```



Now, try to use a five-layer network with 100 units on each layer to overfit on 50 training examples. Again, you will have to adjust the learning rate and weight initialization scale, but you should be able to achieve 100% training accuracy within 20 epochs.

```
[ ]: print(small_data['X_train'].shape)
```

```
(50, 3, 32, 32)
```

```
[ ]: # TODO: Use a five-layer Net to overfit 50 training examples by
      # tweaking just the learning rate and initialization scale.
```

```
num_train = 50
small_data = {
    'X_train': data['X_train'][:num_train],
    'y_train': data['y_train'][:num_train],
    'X_val': data['X_val'],
    'y_val': data['y_val'],
}
```

```

learning_rate = 8e-3 # Experiment with this!
weight_scale = 4e-2  # Experiment with this!

model = FullyConnectedNet(
    [100, 100, 100, 100],
    weight_scale=weight_scale,
    dtype=np.float64
)
solver = Solver(
    model,
    small_data,
    print_every=10,
    num_epochs=20,
    batch_size=25,
    update_rule='sgd',
    optim_config={'learning_rate': learning_rate},
)
solver.train()
# learning_rates = [5e-4, 1e-3, 2e-3, 4e-3, 8e-3]
# weight_scales = [1e-2, 1.8e-2, 2e-2, 4e-2, 8e-2]

# for learning_rate in learning_rates:
#     for weight_scale in weight_scales:
#         print("Running check with lr = ", learning_rate)
#         print("Running check with w_c = ", weight_scale)
#         model = FullyConnectedNet(
#             [100, 100, 100, 100],
#             weight_scale=weight_scale,
#             dtype=np.float64
#         )
#         solver = Solver(
#             model,
#             small_data,
#             print_every=10,
#             num_epochs=20,
#             batch_size=25,
#             update_rule='sgd',
#             optim_config={'learning_rate': learning_rate},
#         )
#         solver.train()

# Running check with lr = 0.008
# Running check with w_c = 0.04

# Running check with lr = 0.004
# Running check with w_c = 0.08

```



```

# Running check with lr = 0.004
# Running check with w_c = 0.04

# Running check with lr = 0.002
# Running check with w_c = 0.08

# Running check with lr = 0.001
# Running check with w_c = 0.08

# Running check with lr = 0.0005
# Running check with w_c = 0.08

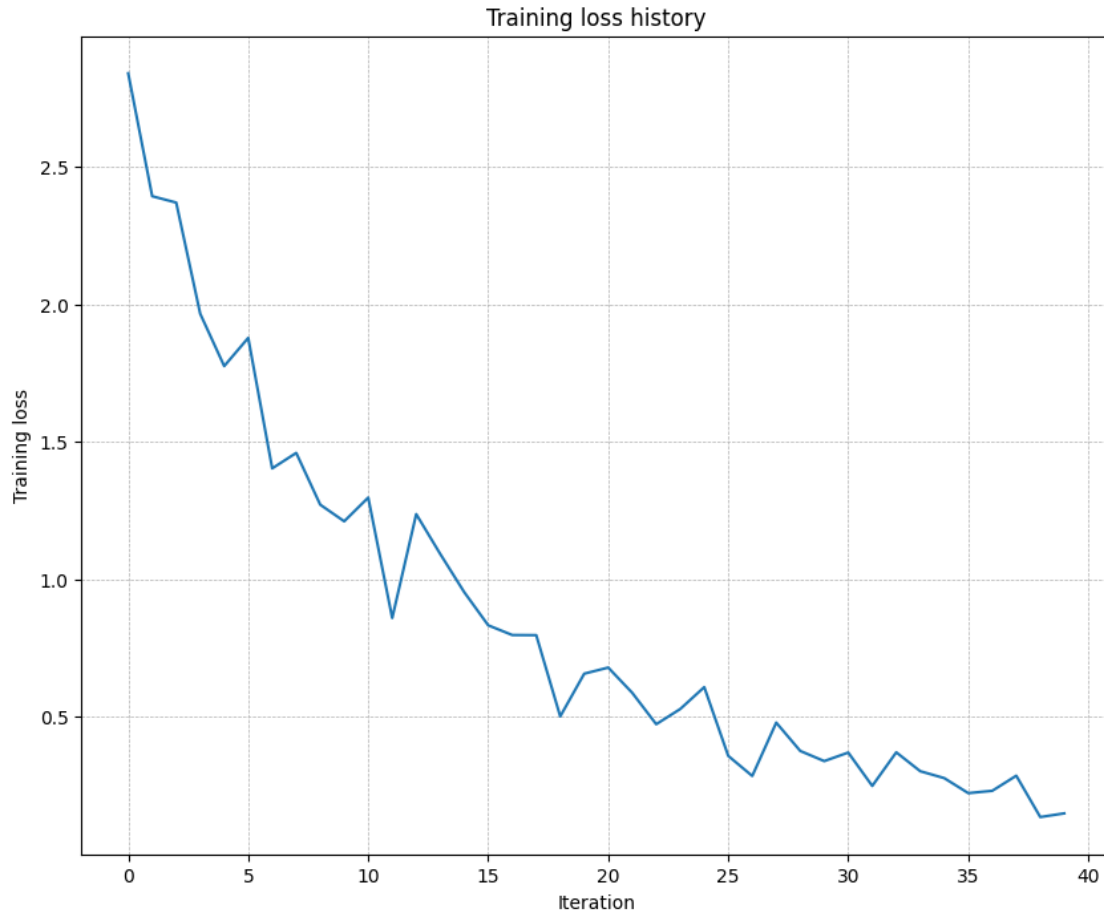
plt.plot(solver.loss_history)
plt.title('Training loss history')
plt.xlabel('Iteration')
plt.ylabel('Training loss')
plt.grid(linestyle='--', linewidth=0.5)
plt.show()

```

```

(Iteration 1 / 40) loss: 2.839886
(Epoch 0 / 20) train acc: 0.340000; val_acc: 0.121000
(Epoch 1 / 20) train acc: 0.220000; val_acc: 0.090000
(Epoch 2 / 20) train acc: 0.500000; val_acc: 0.121000
(Epoch 3 / 20) train acc: 0.500000; val_acc: 0.111000
(Epoch 4 / 20) train acc: 0.680000; val_acc: 0.138000
(Epoch 5 / 20) train acc: 0.660000; val_acc: 0.135000
(Iteration 11 / 40) loss: 1.297475
(Epoch 6 / 20) train acc: 0.720000; val_acc: 0.154000
(Epoch 7 / 20) train acc: 0.780000; val_acc: 0.155000
(Epoch 8 / 20) train acc: 0.880000; val_acc: 0.169000
(Epoch 9 / 20) train acc: 0.880000; val_acc: 0.176000
(Epoch 10 / 20) train acc: 0.880000; val_acc: 0.178000
(Iteration 21 / 40) loss: 0.679632
(Epoch 11 / 20) train acc: 0.980000; val_acc: 0.168000
(Epoch 12 / 20) train acc: 0.980000; val_acc: 0.178000
(Epoch 13 / 20) train acc: 0.960000; val_acc: 0.168000
(Epoch 14 / 20) train acc: 0.940000; val_acc: 0.169000
(Epoch 15 / 20) train acc: 0.960000; val_acc: 0.167000
(Iteration 31 / 40) loss: 0.370673
(Epoch 16 / 20) train acc: 1.000000; val_acc: 0.175000
(Epoch 17 / 20) train acc: 1.000000; val_acc: 0.172000
(Epoch 18 / 20) train acc: 1.000000; val_acc: 0.185000
(Epoch 19 / 20) train acc: 1.000000; val_acc: 0.168000
(Epoch 20 / 20) train acc: 0.960000; val_acc: 0.168000

```



1.2 Inline Question 1:

Did you notice anything about the comparative difficulty of training the three-layer network vs. training the five-layer network? In particular, based on your experience, which network seemed more sensitive to the initialization scale? Why do you think that is the case?

1.3 Answer:

[FILL THIS IN]

5-layer network nhạy cảm với khởi tạo hơn. Vì :

- Càng nhiều layer $\rightarrow$ Càng nhiều tham số $\rightarrow$ Không gian tham số phức tạp $\rightarrow$ Có thể nhiều vùng phẳng hơn $\rightarrow$ không học được.
- Càng nhiều layer $\rightarrow$ khi backprop càng nhiều tích $\rightarrow$ dễ dẫn đến gradient vanishing/exploding, vanishing khi weight_scale nhỏ, exploding khi weight_scale lớn

2 Update rules

So far we have used vanilla stochastic gradient descent (SGD) as our update rule. More sophisticated update rules can make it easier to train deep networks. We will implement a few of the most commonly used update rules and compare them to vanilla SGD.

2.1 SGD+Momentum

Stochastic gradient descent with momentum is a widely used update rule that tends to make deep networks converge faster than vanilla stochastic gradient descent. See the Momentum Update section at <http://cs231n.github.io/neural-networks-3/#sgd> for more information.

Open the file `cs231n/optim.py` and read the documentation at the top of the file to make sure you understand the API. Implement the SGD+momentum update rule in the function `sgd_momentum` and run the following to check your implementation. You should see errors less than $e-8$.

```
[ ]: from cs231n.optim import sgd_momentum

N, D = 4, 5
w = np.linspace(-0.4, 0.6, num=N*D).reshape(N, D)
dw = np.linspace(-0.6, 0.4, num=N*D).reshape(N, D)
v = np.linspace(0.6, 0.9, num=N*D).reshape(N, D)

config = {"learning_rate": 1e-3, "velocity": v}
next_w, _ = sgd_momentum(w, dw, config=config)

expected_next_w = np.asarray([
    [ 0.1406,      0.20738947,  0.27417895,  0.34096842,  0.40775789],
    [ 0.47454737,  0.54133684,  0.60812632,  0.67491579,  0.74170526],
    [ 0.80849474,  0.87528421,  0.94207368,  1.00886316,  1.07565263],
    [ 1.14244211,  1.20923158,  1.27602105,  1.34281053,  1.4096      ]])
expected_velocity = np.asarray([
    [ 0.5406,      0.55475789,  0.56891579,  0.58307368,  0.59723158],
    [ 0.61138947,  0.62554737,  0.63970526,  0.65386316,  0.66802105],
    [ 0.68217895,  0.69633684,  0.71049474,  0.72465263,  0.73881053],
    [ 0.75296842,  0.76712632,  0.78128421,  0.79544211,  0.8096      ]])

# Should see relative errors around e-8 or less
print("next_w error: ", rel_error(next_w, expected_next_w))
print("velocity error: ", rel_error(expected_velocity, config["velocity"]))
```

```
next_w error:  8.882347033505819e-09
velocity error: 4.269287743278663e-09
```

Once you have done so, run the following to train a six-layer network with both SGD and SGD+momentum. You should see the SGD+momentum update rule converge faster.

```
[ ]: num_train = 4000
     small_data = {
```

```

    'X_train': data['X_train'][:num_train],
    'y_train': data['y_train'][:num_train],
    'X_val': data['X_val'],
    'y_val': data['y_val'],
}

solvers = {}

for update_rule in ['sgd', 'sgd_momentum']:
    print('Running with ', update_rule)
    model = FullyConnectedNet(
        [100, 100, 100, 100, 100],
        weight_scale=5e-2
    )

    solver = Solver(
        model,
        small_data,
        num_epochs=5,
        batch_size=100,
        update_rule=update_rule,
        optim_config={'learning_rate': 5e-3},
        verbose=True,
    )
    solvers[update_rule] = solver
    solver.train()

fig, axes = plt.subplots(3, 1, figsize=(15, 15))

axes[0].set_title('Training loss')
axes[0].set_xlabel('Iteration')
axes[1].set_title('Training accuracy')
axes[1].set_xlabel('Epoch')
axes[2].set_title('Validation accuracy')
axes[2].set_xlabel('Epoch')

for update_rule, solver in solvers.items():
    axes[0].plot(solver.loss_history, label=f"loss_{update_rule}")
    axes[1].plot(solver.train_acc_history, label=f"train_acc_{update_rule}")
    axes[2].plot(solver.val_acc_history, label=f"val_acc_{update_rule}")

for ax in axes:
    ax.legend(loc="best", ncol=4)
    ax.grid(linestyle='--', linewidth=0.5)

plt.show()

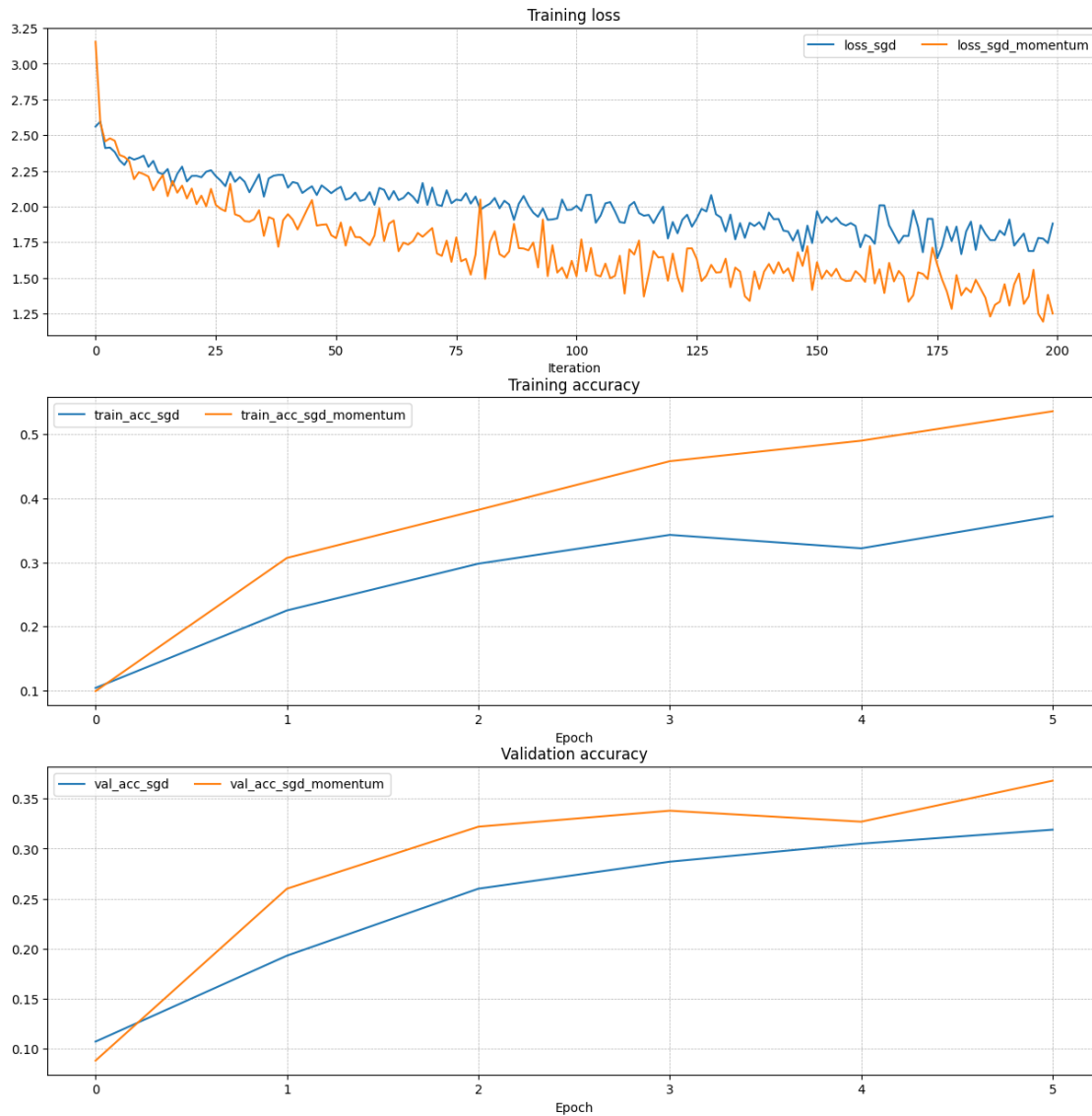
```

```
Running with  sgd
(Iteration 1 / 200) loss: 2.559978
(Epoch 0 / 5) train acc: 0.104000; val_acc: 0.107000
(Iteration 11 / 200) loss: 2.356070
(Iteration 21 / 200) loss: 2.214091
(Iteration 31 / 200) loss: 2.205928
(Epoch 1 / 5) train acc: 0.225000; val_acc: 0.193000
(Iteration 41 / 200) loss: 2.132095
(Iteration 51 / 200) loss: 2.118950
(Iteration 61 / 200) loss: 2.116443
(Iteration 71 / 200) loss: 2.132549
(Epoch 2 / 5) train acc: 0.298000; val_acc: 0.260000
(Iteration 81 / 200) loss: 1.977227
(Iteration 91 / 200) loss: 2.007528
(Iteration 101 / 200) loss: 2.004762
(Iteration 111 / 200) loss: 1.885342
(Epoch 3 / 5) train acc: 0.343000; val_acc: 0.287000
(Iteration 121 / 200) loss: 1.891516
(Iteration 131 / 200) loss: 1.923677
(Iteration 141 / 200) loss: 1.957743
(Iteration 151 / 200) loss: 1.966736
(Epoch 4 / 5) train acc: 0.322000; val_acc: 0.305000
(Iteration 161 / 200) loss: 1.801483
(Iteration 171 / 200) loss: 1.973780
(Iteration 181 / 200) loss: 1.666573
(Iteration 191 / 200) loss: 1.909494
(Epoch 5 / 5) train acc: 0.372000; val_acc: 0.319000
Running with  sgd_momentum
(Iteration 1 / 200) loss: 3.153778
(Epoch 0 / 5) train acc: 0.099000; val_acc: 0.088000
(Iteration 11 / 200) loss: 2.227203
(Iteration 21 / 200) loss: 2.125706
(Iteration 31 / 200) loss: 1.932695
(Epoch 1 / 5) train acc: 0.307000; val_acc: 0.260000
(Iteration 41 / 200) loss: 1.946488
(Iteration 51 / 200) loss: 1.778583
(Iteration 61 / 200) loss: 1.758119
(Iteration 71 / 200) loss: 1.849137
(Epoch 2 / 5) train acc: 0.382000; val_acc: 0.322000
(Iteration 81 / 200) loss: 2.048671
(Iteration 91 / 200) loss: 1.693223
(Iteration 101 / 200) loss: 1.511693
(Iteration 111 / 200) loss: 1.390754
(Epoch 3 / 5) train acc: 0.458000; val_acc: 0.338000
(Iteration 121 / 200) loss: 1.670614
(Iteration 131 / 200) loss: 1.540271
(Iteration 141 / 200) loss: 1.597365
(Iteration 151 / 200) loss: 1.609851
```

```

(Epoch 4 / 5) train acc: 0.490000; val_acc: 0.327000
(Iteration 161 / 200) loss: 1.472687
(Iteration 171 / 200) loss: 1.378620
(Iteration 181 / 200) loss: 1.378175
(Iteration 191 / 200) loss: 1.305934
(Epoch 5 / 5) train acc: 0.536000; val_acc: 0.368000

```



2.2 RMSProp and Adam

RMSProp [1] and Adam [2] are update rules that set per-parameter learning rates by using a running average of the second moments of gradients.

In the file `cs231n/optim.py`, implement the RMSProp update rule in the `rmsprop` function and

implement the Adam update rule in the `adam` function, and check your implementations using the tests below.

NOTE: Please implement the *complete* Adam update rule (with the bias correction mechanism), not the first simplified version mentioned in the course notes.

[1] Tijmen Tieleman and Geoffrey Hinton. “Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude.” COURSERA: Neural Networks for Machine Learning 4 (2012).

[2] Diederik Kingma and Jimmy Ba, “Adam: A Method for Stochastic Optimization”, ICLR 2015.

```
[ ]: # Test RMSProp implementation
from cs231n.optim import rmsprop

N, D = 4, 5
w = np.linspace(-0.4, 0.6, num=N*D).reshape(N, D)
dw = np.linspace(-0.6, 0.4, num=N*D).reshape(N, D)
cache = np.linspace(0.6, 0.9, num=N*D).reshape(N, D)

config = {'learning_rate': 1e-2, 'cache': cache}
next_w, _ = rmsprop(w, dw, config=config)

expected_next_w = np.asarray([
    [-0.39223849, -0.34037513, -0.28849239, -0.23659121, -0.18467247],
    [-0.132737,   -0.08078555, -0.02881884,  0.02316247,  0.07515774],
    [ 0.12716641,  0.17918792,  0.23122175,  0.28326742,  0.33532447],
    [ 0.38739248,  0.43947102,  0.49155973,  0.54365823,  0.59576619]])
expected_cache = np.asarray([
    [ 0.5976,      0.6126277,  0.6277108,  0.64284931,  0.65804321],
    [ 0.67329252,  0.68859723,  0.70395734,  0.71937285,  0.73484377],
    [ 0.75037008,  0.7659518,   0.78158892,  0.79728144,  0.81302936],
    [ 0.82883269,  0.84469141,  0.86060554,  0.87657507,  0.8926    ]])

# You should see relative errors around e-7 or less
print('next_w error: ', rel_error(expected_next_w, next_w))
print('cache error: ', rel_error(expected_cache, config['cache']))
```

```
next_w error:  9.524687511038133e-08
cache error:  2.6477955807156126e-09
```

```
[ ]: # Test Adam implementation
from cs231n.optim import adam

N, D = 4, 5
w = np.linspace(-0.4, 0.6, num=N*D).reshape(N, D)
dw = np.linspace(-0.6, 0.4, num=N*D).reshape(N, D)
m = np.linspace(0.6, 0.9, num=N*D).reshape(N, D)
v = np.linspace(0.7, 0.5, num=N*D).reshape(N, D)
```

```

config = {'learning_rate': 1e-2, 'm': m, 'v': v, 't': 5}
next_w, _ = adam(w, dw, config=config)

expected_next_w = np.asarray([
    [-0.40094747, -0.34836187, -0.29577703, -0.24319299, -0.19060977],
    [-0.1380274, -0.08544591, -0.03286534, 0.01971428, 0.0722929],
    [ 0.1248705, 0.17744702, 0.23002243, 0.28259667, 0.33516969],
    [ 0.38774145, 0.44031188, 0.49288093, 0.54544852, 0.59801459]])
expected_v = np.asarray([
    [ 0.69966, 0.68908382, 0.67851319, 0.66794809, 0.65738853,],
    [ 0.64683452, 0.63628604, 0.6257431, 0.61520571, 0.60467385,],
    [ 0.59414753, 0.58362676, 0.57311152, 0.56260183, 0.55209767,],
    [ 0.54159906, 0.53110598, 0.52061845, 0.51013645, 0.49966,  ]])
expected_m = np.asarray([
    [ 0.48, 0.49947368, 0.51894737, 0.53842105, 0.55789474],
    [ 0.57736842, 0.59684211, 0.61631579, 0.63578947, 0.65526316],
    [ 0.67473684, 0.69421053, 0.71368421, 0.73315789, 0.75263158],
    [ 0.77210526, 0.79157895, 0.81105263, 0.83052632, 0.85  ]])

# You should see relative errors around e-7 or less
print('next_w error: ', rel_error(expected_next_w, next_w))
print('v error: ', rel_error(expected_v, config['v']))
print('m error: ', rel_error(expected_m, config['m']))

```

```

next_w error:  1.1395691798535431e-07
v error:  4.208314038113071e-09
m error:  4.214963193114416e-09

```

Once you have debugged your RMSProp and Adam implementations, run the following to train a pair of deep networks using these new update rules:

```

[ ]: learning_rates = {'rmsprop': 1e-4, 'adam': 1e-3}
for update_rule in ['adam', 'rmsprop']:
    print('Running with ', update_rule)
    model = FullyConnectedNet(
        [100, 100, 100, 100, 100],
        weight_scale=5e-2
    )
    solver = Solver(
        model,
        small_data,
        num_epochs=5,
        batch_size=100,
        update_rule=update_rule,
        optim_config={'learning_rate': learning_rates[update_rule]},
        verbose=True
    )
    solvers[update_rule] = solver

```



```

    solver.train()
    print()

fig, axes = plt.subplots(3, 1, figsize=(15, 15))

axes[0].set_title('Training loss')
axes[0].set_xlabel('Iteration')
axes[1].set_title('Training accuracy')
axes[1].set_xlabel('Epoch')
axes[2].set_title('Validation accuracy')
axes[2].set_xlabel('Epoch')

for update_rule, solver in solvers.items():
    axes[0].plot(solver.loss_history, label=f"{update_rule}")
    axes[1].plot(solver.train_acc_history, label=f"{update_rule}")
    axes[2].plot(solver.val_acc_history, label=f"{update_rule}")

for ax in axes:
    ax.legend(loc='best', ncol=4)
    ax.grid(linestyle='--', linewidth=0.5)

plt.show()

```

```

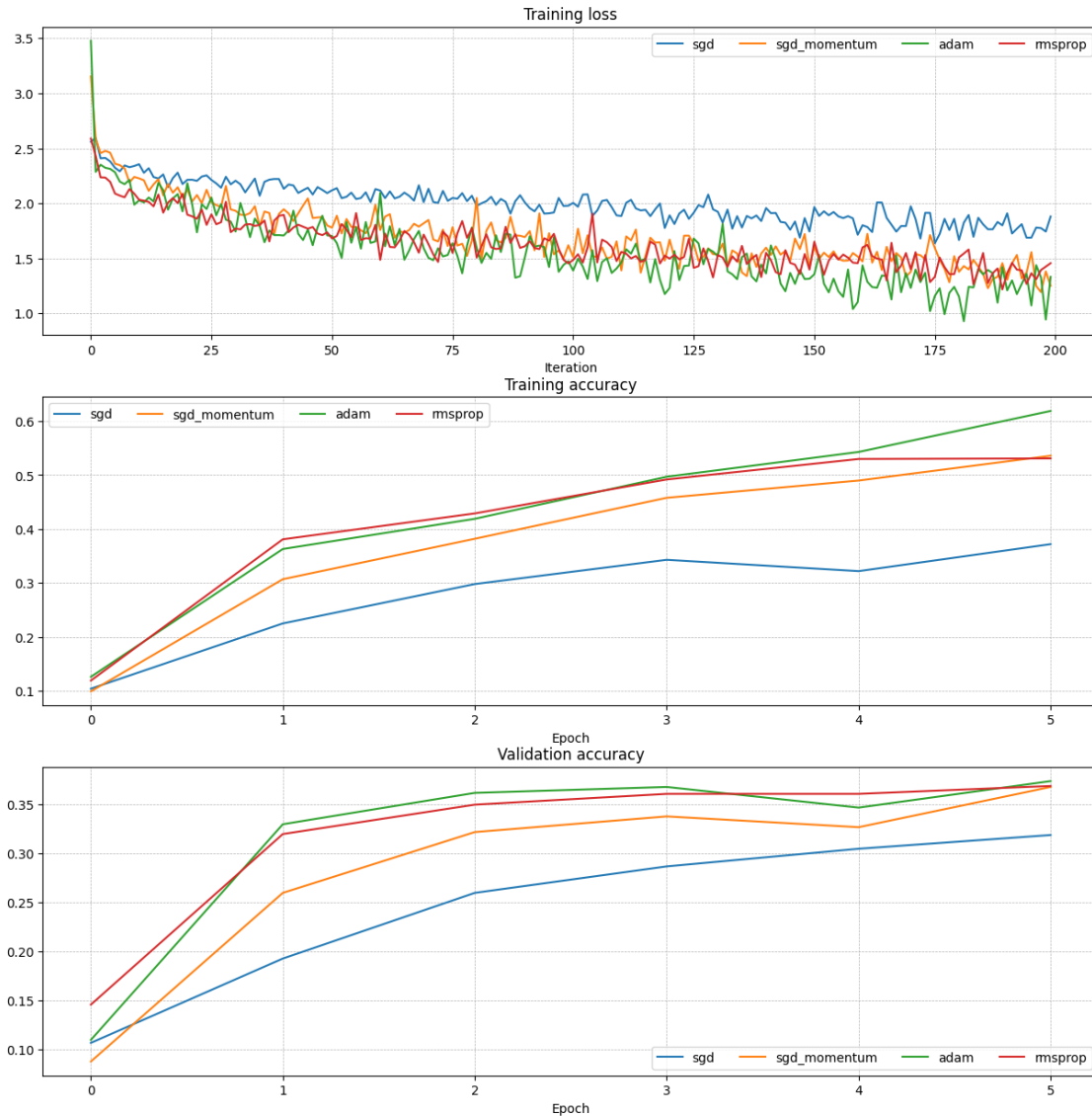
Running with adam
(Iteration 1 / 200) loss: 3.476928
(Epoch 0 / 5) train acc: 0.126000; val_acc: 0.110000
(Iteration 11 / 200) loss: 2.027712
(Iteration 21 / 200) loss: 2.183357
(Iteration 31 / 200) loss: 1.744257
(Epoch 1 / 5) train acc: 0.363000; val_acc: 0.330000
(Iteration 41 / 200) loss: 1.707951
(Iteration 51 / 200) loss: 1.703835
(Iteration 61 / 200) loss: 2.094758
(Iteration 71 / 200) loss: 1.505558
(Epoch 2 / 5) train acc: 0.419000; val_acc: 0.362000
(Iteration 81 / 200) loss: 1.594431
(Iteration 91 / 200) loss: 1.511452
(Iteration 101 / 200) loss: 1.389237
(Iteration 111 / 200) loss: 1.463575
(Epoch 3 / 5) train acc: 0.497000; val_acc: 0.368000
(Iteration 121 / 200) loss: 1.231313
(Iteration 131 / 200) loss: 1.520198
(Iteration 141 / 200) loss: 1.363221
(Iteration 151 / 200) loss: 1.355143
(Epoch 4 / 5) train acc: 0.543000; val_acc: 0.347000
(Iteration 161 / 200) loss: 1.436402
(Iteration 171 / 200) loss: 1.231426

```

(Iteration 181 / 200) loss: 1.153575
(Iteration 191 / 200) loss: 1.209479
(Epoch 5 / 5) train acc: 0.619000; val_acc: 0.374000

Running with rmsprop

(Iteration 1 / 200) loss: 2.589166
(Epoch 0 / 5) train acc: 0.119000; val_acc: 0.146000
(Iteration 11 / 200) loss: 2.032921
(Iteration 21 / 200) loss: 1.897277
(Iteration 31 / 200) loss: 1.770793
(Epoch 1 / 5) train acc: 0.381000; val_acc: 0.320000
(Iteration 41 / 200) loss: 1.895731
(Iteration 51 / 200) loss: 1.681091
(Iteration 61 / 200) loss: 1.487204
(Iteration 71 / 200) loss: 1.629973
(Epoch 2 / 5) train acc: 0.429000; val_acc: 0.350000
(Iteration 81 / 200) loss: 1.506686
(Iteration 91 / 200) loss: 1.610742
(Iteration 101 / 200) loss: 1.486124
(Iteration 111 / 200) loss: 1.559454
(Epoch 3 / 5) train acc: 0.492000; val_acc: 0.361000
(Iteration 121 / 200) loss: 1.497406
(Iteration 131 / 200) loss: 1.530736
(Iteration 141 / 200) loss: 1.550957
(Iteration 151 / 200) loss: 1.652046
(Epoch 4 / 5) train acc: 0.530000; val_acc: 0.361000
(Iteration 161 / 200) loss: 1.599574
(Iteration 171 / 200) loss: 1.401073
(Iteration 181 / 200) loss: 1.509365
(Iteration 191 / 200) loss: 1.365772
(Epoch 5 / 5) train acc: 0.531000; val_acc: 0.369000



2.3 Inline Question 2:

AdaGrad, like Adam, is a per-parameter optimization method that uses the following update rule:

```
cache += dw**2
w += - learning_rate * dw / (np.sqrt(cache) + eps)
```

John notices that when he was training a network with AdaGrad that the updates became very small, and that his network was learning slowly. Using your knowledge of the AdaGrad update rule, why do you think the updates would become very small? Would Adam have the same issue?

2.4 Answer:

[Vì AdaGrad dùng $\text{cache} = \sum dw^2 \rightarrow \text{cache tăng theo thời gian} \rightarrow \text{dẫn đến}$

biểu thức cập nhật ngày càng nhỏ, dù α có lớn. Adam cải thiện điều này bằng cách cộng có trọng số (tổng bằng 1) $v = \beta_2 * v + (1-\beta_2) * (dx^2)$

3 Train a Good Model!

Train the best fully connected model that you can on CIFAR-10, storing your best model in the `best_model` variable. We require you to get at least 50% accuracy on the validation set using a fully connected network.

If you are careful it should be possible to get accuracies above 55%, but we don't require it for this part and won't assign extra credit for doing so. Later in the next assignment, we will ask you to train the best convolutional network that you can on CIFAR-10, and we would prefer that you spend your effort working on convolutional networks rather than fully connected networks.

Note: In the next assignment, you will learn techniques like BatchNormalization and Dropout which can help you train powerful models.

```
[ ]: best_model = None

#####
# TODO: Train the best FullyConnectedNet that you can on CIFAR-10. You might #
# find batch/layer normalization and dropout useful. Store your best model in #
# the best_model variable. #####
# *****START OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
best_val = -1.0
# hidden_dims = [64, 128, 64]
hidden_dims = [512, 256, 128]
# learning_rates = [1e-5, 1e-4, 5e-4, 1e-3, 5e-3, 1e-2]
learning_rates = [1e-3, 5e-4]
weight_scales = [2e-2, 5e-2]

for lr in learning_rates:
    for ws in weight_scales:
        print(f'\n train with lr: {lr}, ws: {ws}\n')
        # model = FullyConnectedNet(
        #     [100, 100, 100, 100, 100],
        #     weight_scale=ws,
        #     normalization='layernorm',
        #     dropout_keep_ratio=0.7,
        #     reg=1e-3,
        #     dtype=np.float64
        # )
        model = FullyConnectedNet(
            hidden_dims,
            weight_scale=ws,
            normalization='batchnorm',
            dropout_keep_ratio=0.7,
```

```

        reg=1e-3,
        dtype=np.float64
    )

    # solver = Solver(
    #     model, data,
    #     num_epochs=5,
    #     batch_size=200,
    #     update_rule='adam',
    #     optim_config={'learning_rate': lr},
    #     lr_decay=0.95,
    #     print_every=100,
    #     verbose=True
    # )
    solver = Solver(
        model, data,
        num_epochs=10,
        batch_size=200,
        update_rule='adam',
        optim_config={'learning_rate': lr},
        lr_decay=0.9,
        print_every=200,
        verbose=True
    )
    solver.train()

    if solver.best_val_acc > best_val:
        best_val = solver.best_val_acc
        best_model = model
        print(f'best val accuracy: {best_val}')
# *****END OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
#####
#                                     END OF YOUR CODE                                     #
#####

```

train with lr: 0.001, ws: 0.02

```

(Iteration 1 / 2450) loss: 2.652075
(Epoch 0 / 10) train acc: 0.140000; val_acc: 0.143000
(Iteration 201 / 2450) loss: 1.824707
(Epoch 1 / 10) train acc: 0.473000; val_acc: 0.460000
(Iteration 401 / 2450) loss: 1.629882
(Epoch 2 / 10) train acc: 0.513000; val_acc: 0.485000
(Iteration 601 / 2450) loss: 1.657906
(Epoch 3 / 10) train acc: 0.532000; val_acc: 0.514000
(Iteration 801 / 2450) loss: 1.501678

```

(Epoch 4 / 10) train acc: 0.550000; val_acc: 0.519000
(Iteration 1001 / 2450) loss: 1.631724
(Iteration 1201 / 2450) loss: 1.532109
(Epoch 5 / 10) train acc: 0.557000; val_acc: 0.526000
(Iteration 1401 / 2450) loss: 1.493299
(Epoch 6 / 10) train acc: 0.544000; val_acc: 0.514000
(Iteration 1601 / 2450) loss: 1.554682
(Epoch 7 / 10) train acc: 0.607000; val_acc: 0.545000
(Iteration 1801 / 2450) loss: 1.531875
(Epoch 8 / 10) train acc: 0.575000; val_acc: 0.539000
(Iteration 2001 / 2450) loss: 1.528937
(Iteration 2201 / 2450) loss: 1.499692
(Epoch 9 / 10) train acc: 0.596000; val_acc: 0.562000
(Iteration 2401 / 2450) loss: 1.417194
(Epoch 10 / 10) train acc: 0.617000; val_acc: 0.550000
best val accuracy: 0.562

train with lr: 0.001, ws: 0.05

(Iteration 1 / 2450) loss: 4.552468
(Epoch 0 / 10) train acc: 0.145000; val_acc: 0.118000
(Iteration 201 / 2450) loss: 2.415463
(Epoch 1 / 10) train acc: 0.439000; val_acc: 0.462000
(Iteration 401 / 2450) loss: 1.837184
(Epoch 2 / 10) train acc: 0.479000; val_acc: 0.484000
(Iteration 601 / 2450) loss: 1.852124
(Epoch 3 / 10) train acc: 0.547000; val_acc: 0.494000
(Iteration 801 / 2450) loss: 1.712793
(Epoch 4 / 10) train acc: 0.548000; val_acc: 0.503000
(Iteration 1001 / 2450) loss: 1.651448
(Iteration 1201 / 2450) loss: 1.534162
(Epoch 5 / 10) train acc: 0.533000; val_acc: 0.533000
(Iteration 1401 / 2450) loss: 1.626436
(Epoch 6 / 10) train acc: 0.567000; val_acc: 0.523000
(Iteration 1601 / 2450) loss: 1.596038
(Epoch 7 / 10) train acc: 0.570000; val_acc: 0.536000
(Iteration 1801 / 2450) loss: 1.480193
(Epoch 8 / 10) train acc: 0.564000; val_acc: 0.545000
(Iteration 2001 / 2450) loss: 1.488975
(Iteration 2201 / 2450) loss: 1.481596
(Epoch 9 / 10) train acc: 0.602000; val_acc: 0.548000
(Iteration 2401 / 2450) loss: 1.407387
(Epoch 10 / 10) train acc: 0.605000; val_acc: 0.547000

train with lr: 0.0005, ws: 0.02

(Iteration 1 / 2450) loss: 2.670765
(Epoch 0 / 10) train acc: 0.128000; val_acc: 0.124000

(Iteration 201 / 2450) loss: 1.757105
(Epoch 1 / 10) train acc: 0.473000; val_acc: 0.455000
(Iteration 401 / 2450) loss: 1.784310
(Epoch 2 / 10) train acc: 0.506000; val_acc: 0.496000
(Iteration 601 / 2450) loss: 1.557049
(Epoch 3 / 10) train acc: 0.559000; val_acc: 0.506000
(Iteration 801 / 2450) loss: 1.662853
(Epoch 4 / 10) train acc: 0.541000; val_acc: 0.516000
(Iteration 1001 / 2450) loss: 1.437199
(Iteration 1201 / 2450) loss: 1.610528
(Epoch 5 / 10) train acc: 0.563000; val_acc: 0.541000
(Iteration 1401 / 2450) loss: 1.370390
(Epoch 6 / 10) train acc: 0.581000; val_acc: 0.545000
(Iteration 1601 / 2450) loss: 1.446433
(Epoch 7 / 10) train acc: 0.606000; val_acc: 0.547000
(Iteration 1801 / 2450) loss: 1.490366
(Epoch 8 / 10) train acc: 0.626000; val_acc: 0.535000
(Iteration 2001 / 2450) loss: 1.367031
(Iteration 2201 / 2450) loss: 1.393875
(Epoch 9 / 10) train acc: 0.618000; val_acc: 0.556000
(Iteration 2401 / 2450) loss: 1.211433
(Epoch 10 / 10) train acc: 0.638000; val_acc: 0.562000

train with lr: 0.0005, ws: 0.05

(Iteration 1 / 2450) loss: 4.582063
(Epoch 0 / 10) train acc: 0.122000; val_acc: 0.106000
(Iteration 201 / 2450) loss: 3.151963
(Epoch 1 / 10) train acc: 0.459000; val_acc: 0.456000
(Iteration 401 / 2450) loss: 2.488493
(Epoch 2 / 10) train acc: 0.533000; val_acc: 0.484000
(Iteration 601 / 2450) loss: 2.136898
(Epoch 3 / 10) train acc: 0.531000; val_acc: 0.526000
(Iteration 801 / 2450) loss: 2.072341
(Epoch 4 / 10) train acc: 0.546000; val_acc: 0.516000
(Iteration 1001 / 2450) loss: 1.932676
(Iteration 1201 / 2450) loss: 1.743104
(Epoch 5 / 10) train acc: 0.549000; val_acc: 0.519000
(Iteration 1401 / 2450) loss: 1.820810
(Epoch 6 / 10) train acc: 0.580000; val_acc: 0.539000
(Iteration 1601 / 2450) loss: 1.730878
(Epoch 7 / 10) train acc: 0.586000; val_acc: 0.549000
(Iteration 1801 / 2450) loss: 1.650280
(Epoch 8 / 10) train acc: 0.610000; val_acc: 0.547000
(Iteration 2001 / 2450) loss: 1.611522
(Iteration 2201 / 2450) loss: 1.563781
(Epoch 9 / 10) train acc: 0.594000; val_acc: 0.552000
(Iteration 2401 / 2450) loss: 1.516590

(Epoch 10 / 10) train acc: 0.633000; val_acc: 0.552000

```
[ ]: print(best_val)
```

0.562

4 Test Your Model!

Run your best model on the validation and test sets. You should achieve at least 50% accuracy on the validation set and the test set.

```
[ ]: y_test_pred = np.argmax(best_model.loss(data['X_test']), axis=1)
     y_val_pred = np.argmax(best_model.loss(data['X_val']), axis=1)
     print('Validation set accuracy: ', (y_val_pred == data['y_val']).mean())
     print('Test set accuracy: ', (y_test_pred == data['y_test']).mean())
```

Validation set accuracy: 0.559

Test set accuracy: 0.534